1장 도커란 무엇인가?

1절 도커란 무엇인가?

1.도커의 정체

- 도커 정의
  - "데이터 또는 프로그램을 각각 독립된 환경에 격리시키는 기능"을 제공하는 소프트웨어
  - 운영체제(유사) 통째로 격리
  - 주로 서버에서 사용

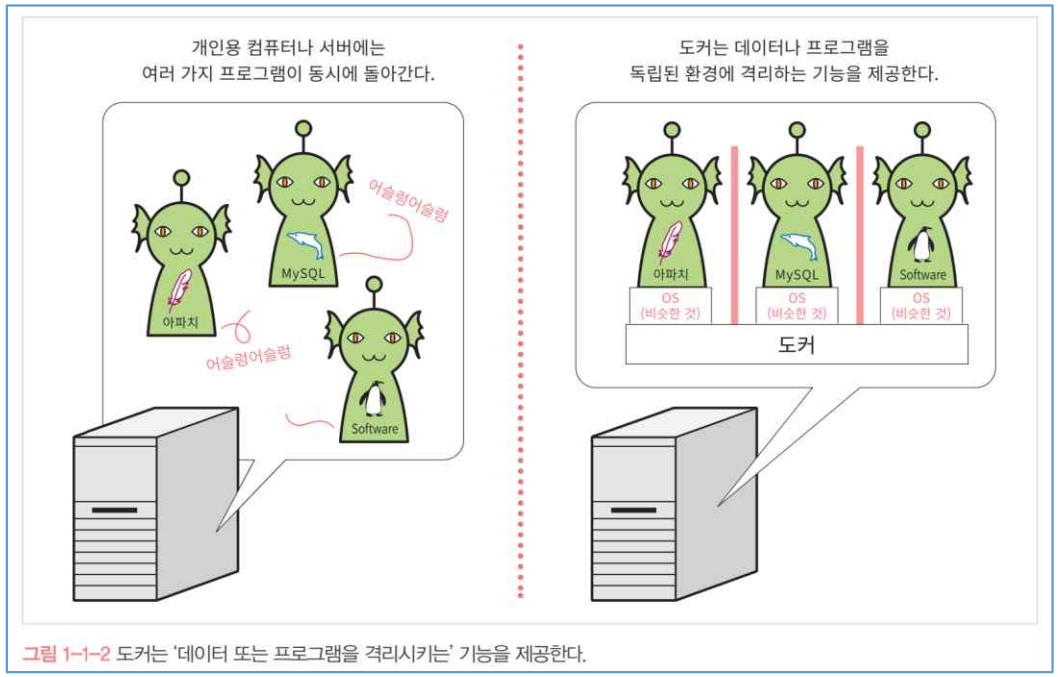


그림 1-1-2 도커는 '데이터 또는 프로그램을 격리시키는' 기능을 제공한다.

1.도커의 정체

- 도커란? (레드햇 기술문서 중)

Docker 기술은 Linux 커널과 Cgroups 및 네임스페이스 등 커널의 기능을 사용하여 프로세스를 분리함으로써 독립적으로 실행할 수 있도록 합니다. 이러한 독립성은 컨테이너의 본래 목적입니다. 다시 말해서, 여러 프로세스와 애플리케이션을 서로 개별적으로 실행하여 인프라를 더 효과적으로 활용하고 개별 시스템을 사용할 때와 동일한 보안을 유지할 수 있습니다.

<출처> https://www.redhat.com/ko/topics/containers/what-is-docker

- 컨테이너 기반 기술(Isolation)
- 리눅스 커널에서 제공하는 기능
- 하나의 시스템에서 프로세스를 격리 시킬 수 있는 가상화 기술

<리눅스 namespace 관련 주요 명령>
https://manpages.ubuntu.com/manpages/xenial/man1/unshare.1.html
https://manpages.ubuntu.com/manpages/xenial/en/man1/nsenter.1.html

1.도커의 정체

- 컨테이너와 도커 엔진

  컨테이너를 다루는 기능을 제공하는 소프트웨어 – 도커

- 도커 엔진 설치 필요

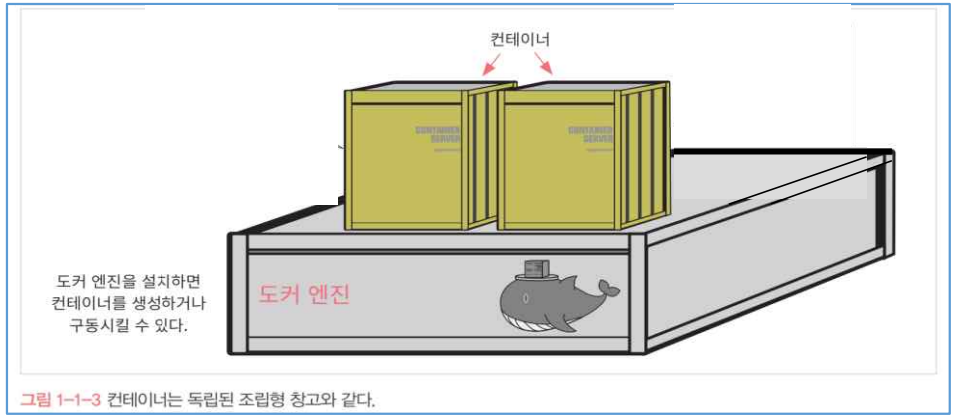


그림 1-1-3 컨테이너는 독립된 조립형 창고와 같다.

- 컨테이너 생성을 위해 이미지 필요

아파치 이미지
MySQL 이미지
. . .

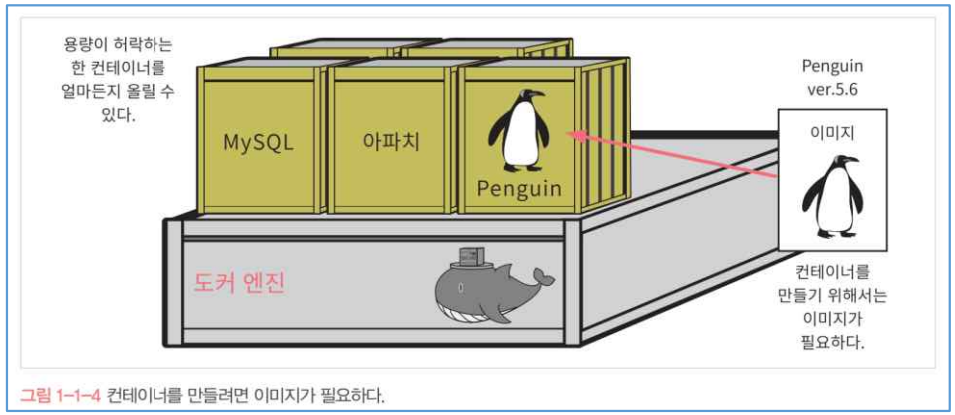


그림 1-1-4 컨테이너를 만들려면 이미지가 필요하다.

1.도커의 정체

- 도커는 리눅스에서!
  - 도커는 리눅스 운영체제 필요
  - 도커는 리눅스 기반을 전제로 만들어졌기 때문에 다음 운영체제보다는 리눅스에서 사용하기를 권장

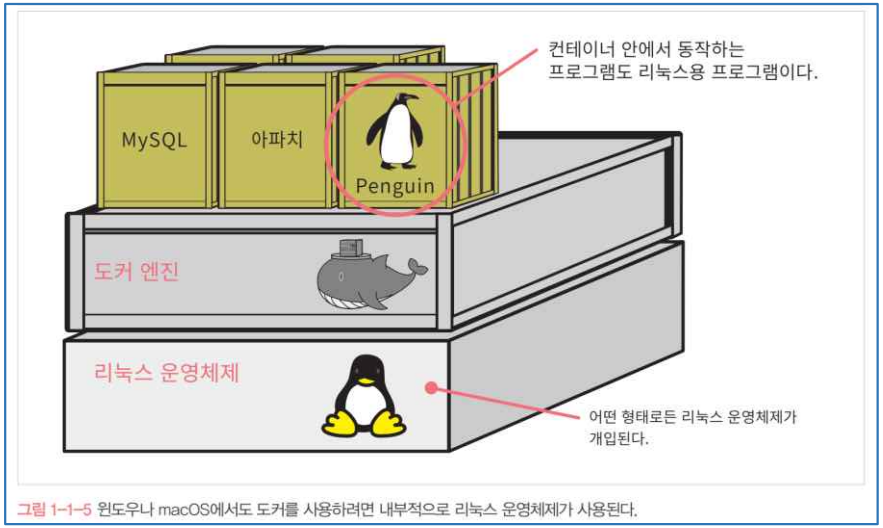


그림 1-1-5 윈도우나 macOS에서도 도커를 사용하려면 내부적으로 리눅스 운영체제가 사용된다.

2.독립된 환경이 필요한 이유

- 데이터나 프로그램을 독립된 환경에 격리해야 하는 이유
  - 프로그램이 수행될때에는 관련 환경, 라이브러리 및 다른 프로그램이 필요

그림 1-1-6 프로그램을 실행하기 위해서는 그 프로그램의 실행 환경이나 라이브러리가 필요하다.

  - 예) 워드프레스 : MySQL 필요

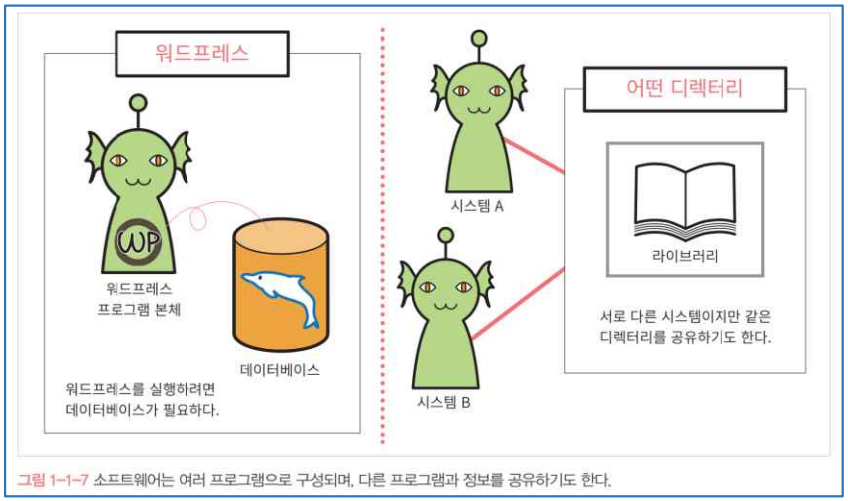


그림 1-1-7 소프트웨어는 여러 프로그램으로 구성되며, 다른 프로그램과 정보를 공유하기도 한다.

2.독립된 환경이 필요한 이유

- 특정 프로그램을 업데이트하면 다른 프로그램에도 영향을 미칠 수 있음

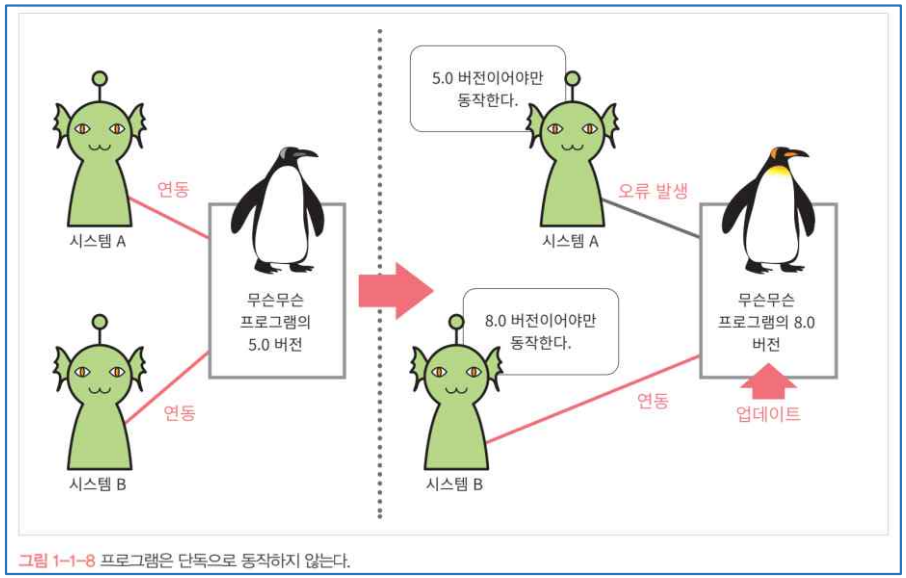


그림 1-1-8 프로그램은 단독으로 동작하지 않는다.

- 소프트웨어 업데이트 뿐만 아니라 실행환경이나 라이브러리, 디렉토리나 설정파일을 수정하는 경우가 발생할 수도 있음

- 문제의 원인은 대부분 프로그램간의 공유에 있다!

3. 프로그램의 격리

- 프로그램의 격리란?
  - 컨테이너 안의 프로그램은 다른 프로그램과 격리된 상태

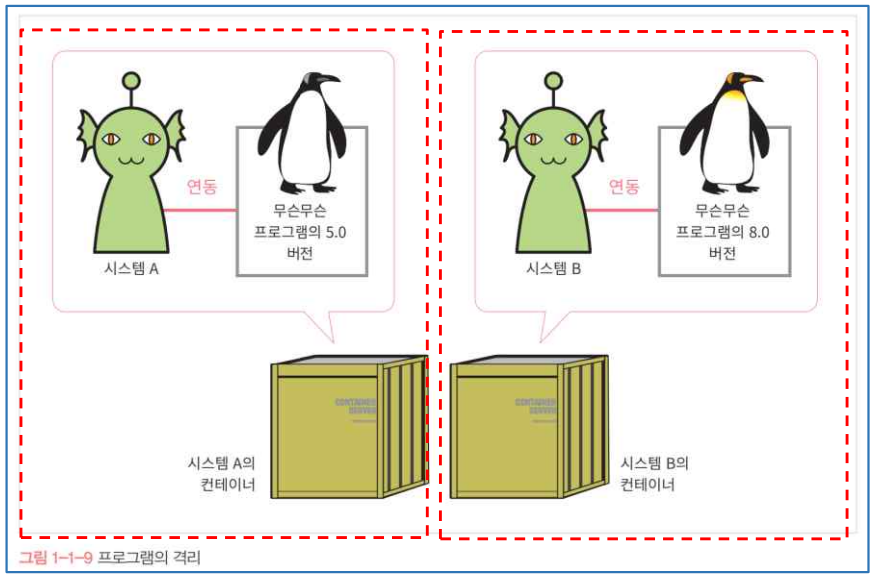


그림 1-1-9 프로그램의 격리

  - 완전히 독립된 환경이므로 동일한 버전 또는 다른 버전의 같은 프로그램 수행 가능

1장 도커란 무엇인가?

2절 서버와 도커

1.서버의 두가지 의미

- 서버란 '어떤 서비스를 제공하는 것'을 의미

그림 1-2-1 서버란 무엇인가?

- 기능적 서버와 물리적 서버

그림 1-2-2 서버의 역할

1.서버의 두가지 의미

- 서버의 기능은 소프트웨어에서 제공

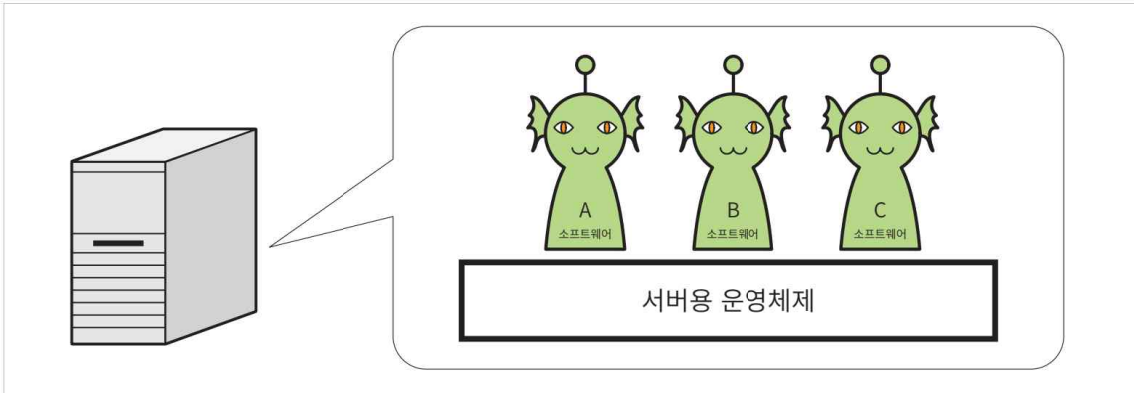


그림 1-2-6 서버 역시 운영체제가 동작하고 그 위에서 소프트웨어가 실행된다.

웹 서버용 소프트웨어를 설치 ➡ 웹 서버가 된다.
메일 서버용 소프트웨어를 설치 ➡ 메일 서버가 된다.

여러 기능적의미의 서버가 하나의 물리적 컴퓨터에 함께 존재할 수 있다 **도커의 장점과 관련!**

1.서버의 두가지 의미

• 서버의 대표적인 예

서버의 대표적인 예

서버의 종류	설명
웹 서버	웹 사이트 기능을 제공하는 서버. HTML 파일 또는 이미지 파일, 프로그램을 배치하고, 클라이언트 컴퓨터의 브라우저에서 접근해오면 이들 파일을 제공한다. 대표적인 소프트웨어로 아파치, Nginx, IIS 등이 있다.
메일 서버	메일 송수신을 담당하는 SMTP 서버와 클라이언트에 메일을 전달하는 POP 서버로 나뉜다. 이들을 합쳐 메일 서버라고 부르는 경우가 많다. 메일을 내려받지 않고 서버에 둔 채로 읽는 IMAP4 서버도 있다. 대표적인 소프트웨어로 Sendmail, Postfix, Dovecot이 있다.
데이터베이스 서버	데이터를 저장하거나 검색하는 데이터베이스 기능을 제공하는 서버다. 대표적인 소프트웨어로 MySQL, PostgreSQL, MariaDB, SQL Server, Oracle Database가 있다.
파일 서버	파일을 저장하고 다른 사람과 공유하기 위한 서버다. 대표적인 소프트웨어로 Samba가 있다.
DNS 서버	IP 주소와 도메인을 연결해주는 DNS 기능을 제공하는 서버.
DHCP 서버	IP 주소를 자동으로 할당하는 기능을 제공하는 서버.
FTP 서버	FTP 프로토콜을 사용해 파일 송수신 기능을 제공하는 서버. 웹 서버와 함께 설치하는 경우가 많으며, 파일을 제공하는 데 사용한다.
프락시 서버	통신을 중계하는 역할을 맡는 서버의 통칭. 사내 로컬 네트워크 등 인터넷과 격리된 장소에서 인터넷 상의 서버에 접속할 때 사용한다. 프락시 서버를 경유하면 접속 대상 컴퓨터에 자신의 정체를 숨길 수 있으므로 정체를 드러내고 싶지 않을 때도 사용한다.
인증 서버	사용자 인증을 위한 서버. 윈도우 네트워크에 로그인하기 위한 Active Directory 서버 또는 무선 LAN이나 원격 접속에서 사용자 인증을 맡는 Radius 서버 등이 있다. 대표적인 소프트웨어로 OpenLDAP, Active Directory가 있다.

1.서버의 두가지 의미

- 서버의 운영체제로는 주로 리눅스 사용

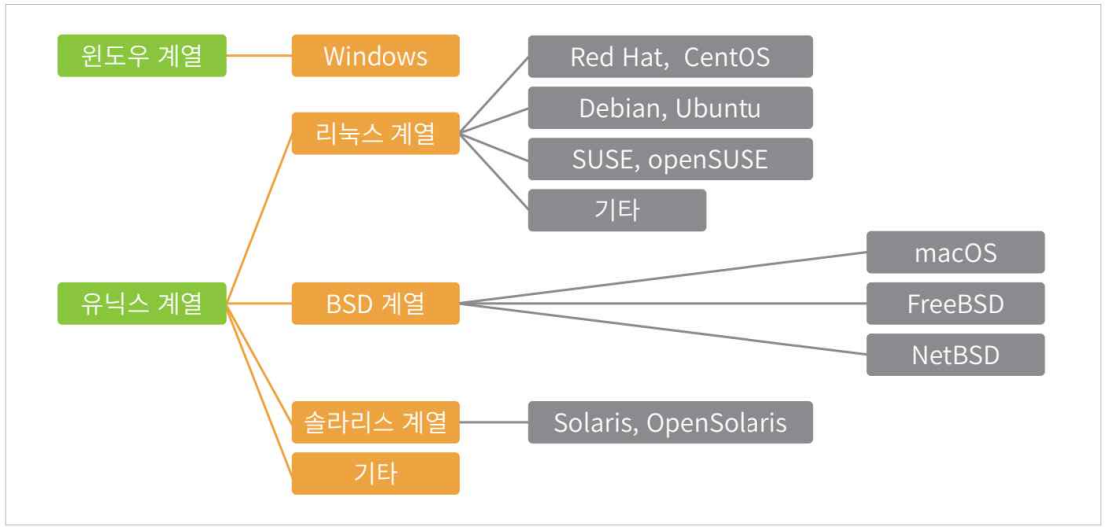


그림 1-2-7 서버 운영체제의 종류

2.여러 서버의 기능을 안전하게 실행하기

- 컨테이너 기술을 이용한 여러 서버의 안전한 실행이 가능

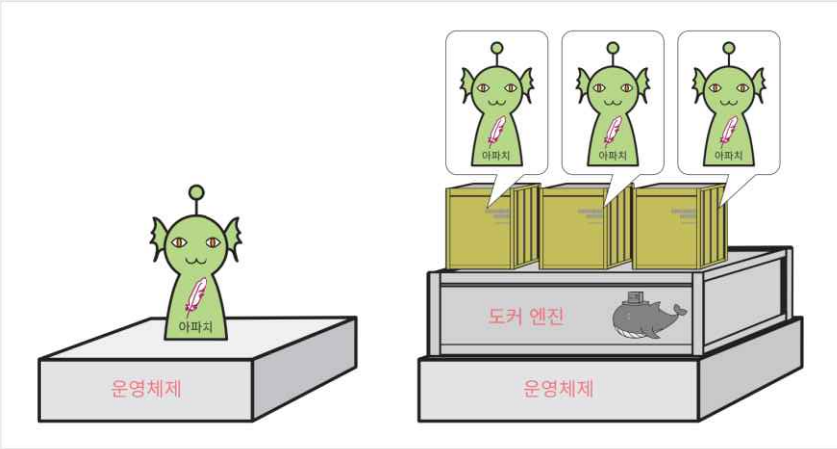


그림 1-2-8 컨테이너 기술을 활용하면 하나의 물리 서버에 여러 개의 웹 서버를 띄울 수 있다.

- 물리적인 1대의 컴퓨터에 웹서버, 메일서버, DB서버를 각 독립적인 환경에서 수행
- 1대의 서버에서 여러 개의 웹서버 구동 가능
- 물리 서버의 수를 줄일 수 있으므로 비용 절감
- 개발 환경을 갖추거나 운영환경으로 쉽게 옮길 수 있다는 특성

3.자유로이 옮길 수 있는 컨테이너

- 다른 도커 엔진으로의 이동이 용이
  - 동일한 상태로 튜닝한 컨테이너를 팀원 전체에게 배포하여 동일한 개발 환경 사용이 용이
  - 운영체제 무관, 도커만 설치되어 있으면 가능
  - 물리적 환경의 차이나 서버 구성의 차이 등 운영서버와 개발서버 사이의 환경 차이로 인한 문제를 근본적으로 방지할 수 있음

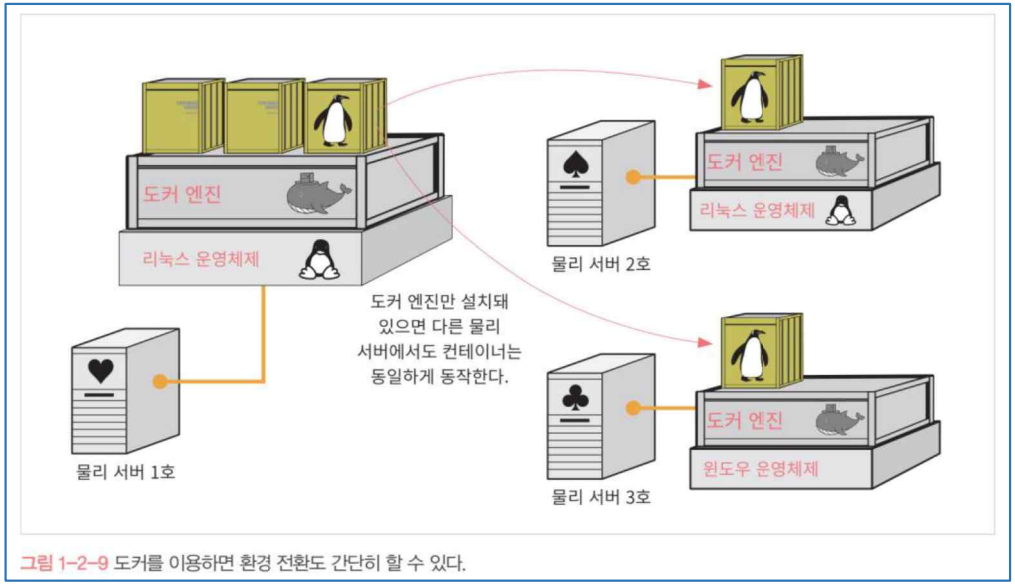


그림 1-2-9 도커를 이용하면 환경 전환도 간단히 할 수 있다.

2장 도커의 동작 원리

1절 도커의 동작원리

1.도커의 구조

• 도커의 구조

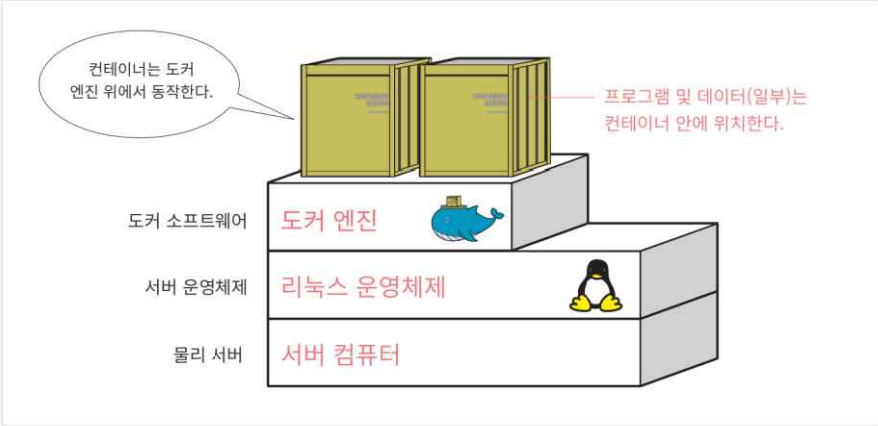


그림 2-1-1 컨테이너는 도커 엔진 위에서 동작한다.

- 운영체제 위에 도커 엔진이 동작하고 그 위에서 컨테이너가 동작
- 프로그램과 데이터는 컨테이너 안에 위치

1.도커의 구조

- 모든 컨테이너 안에는 리눅스 운영체제와 유사한 환경 제공

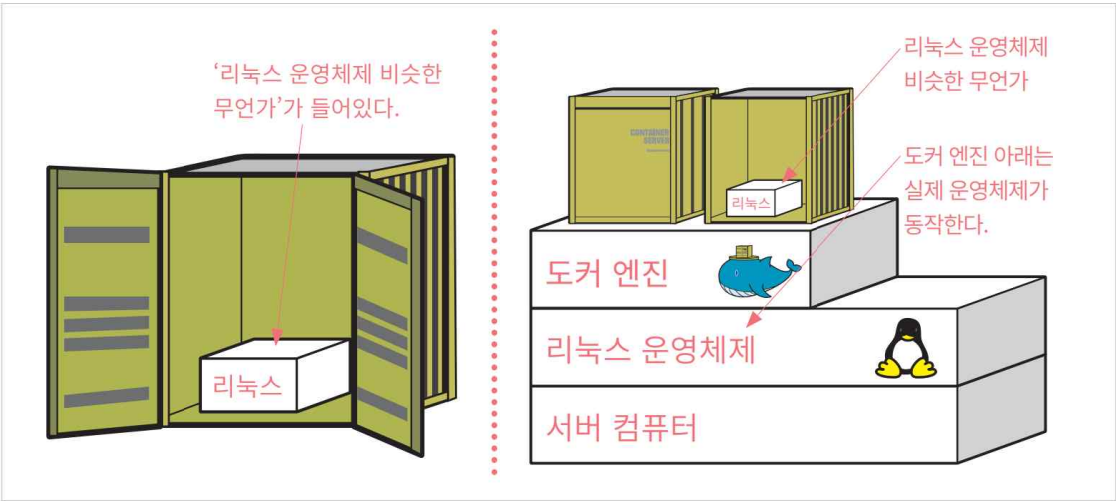


그림 2-1-2 컨테이너 속에는 '리눅스 운영체제 비슷한 무언가'가 들어 있다.

1.도커의 구조

• 운영체제의 구조

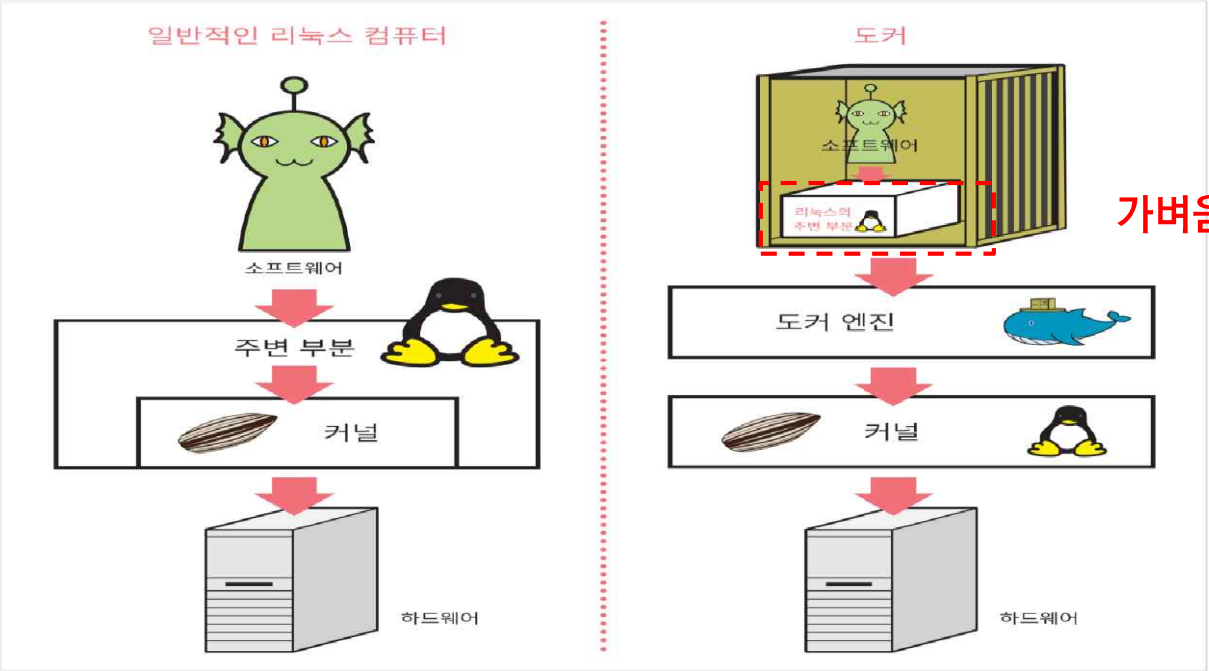


그림 2-1-4 컨테이너는 외부와 완전히 분리돼 있으므로 컨테이너 속에 운영체제의 주변 부분이 따로 들어 있다.

2.리눅스용 도커

- 도커는 기본적으로 리눅스 운영체제에서만 동작
- 컨테이너에서 실행할 소프트웨어 역시 리눅스용 소프트웨어
- 즉, 도커는 리눅스 컴퓨터에 독립된 격리 환경을 만드는 것

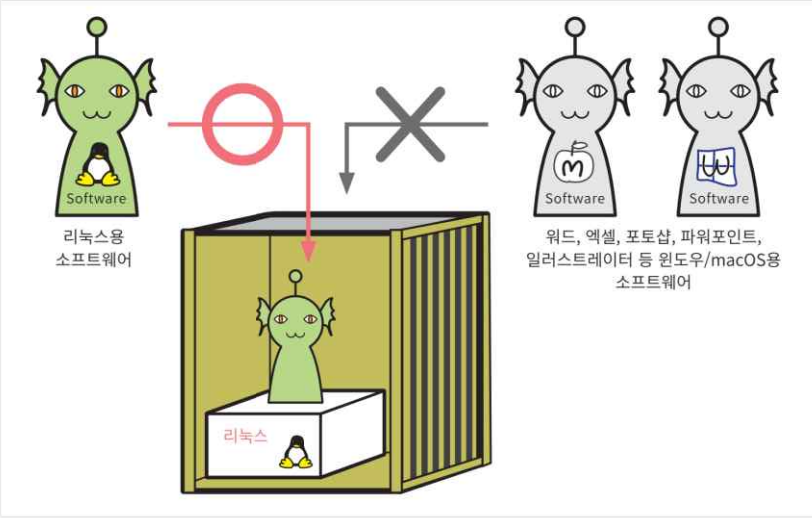


그림 2-1-5 컨테이너에 들어가는 프로그램은 리눅스용이어야 한다.

2.리눅스용 도커

- 윈도우와 macOS에서 도커 구동
- 윈도우나 macOS위에서 도커를 수행하려면 리눅스 운영체제를 갖추어야 함.

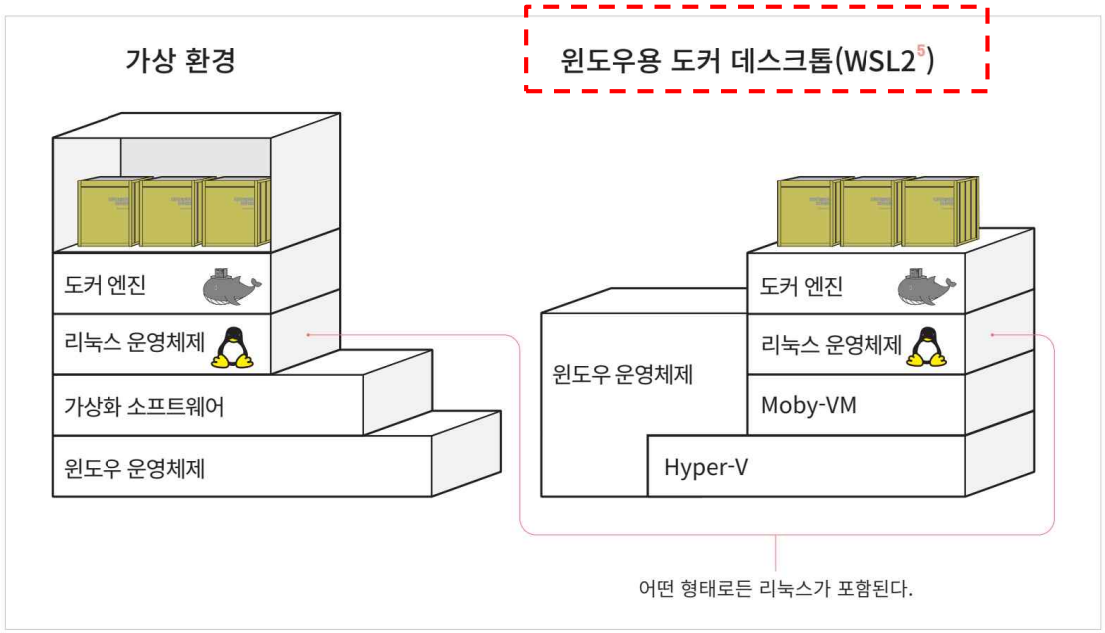


그림 2-1-6 도커는 기본적으로 리눅스용이다.

2장 도커의 동작 원리

2절 도커 허브와 이미지, 그리고 컨테이너

1.이미지와 컨테이너

• 이미지 : 컨테이너의 설계도

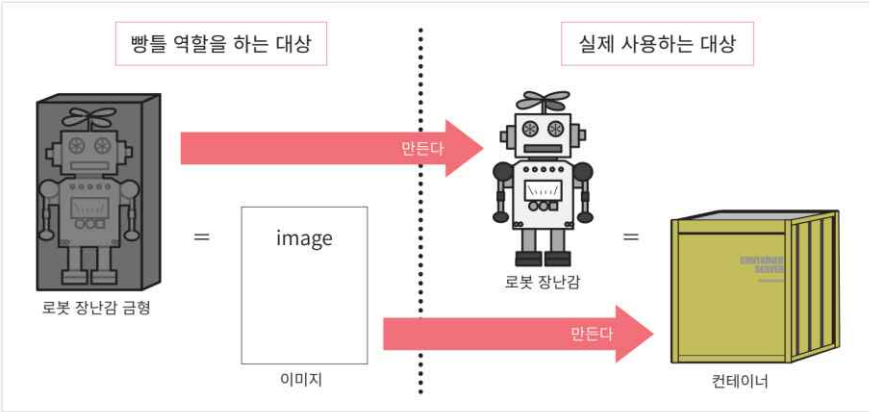


그림 2-2-1 이미지는 컨테이너를 만드는 데 사용한다.

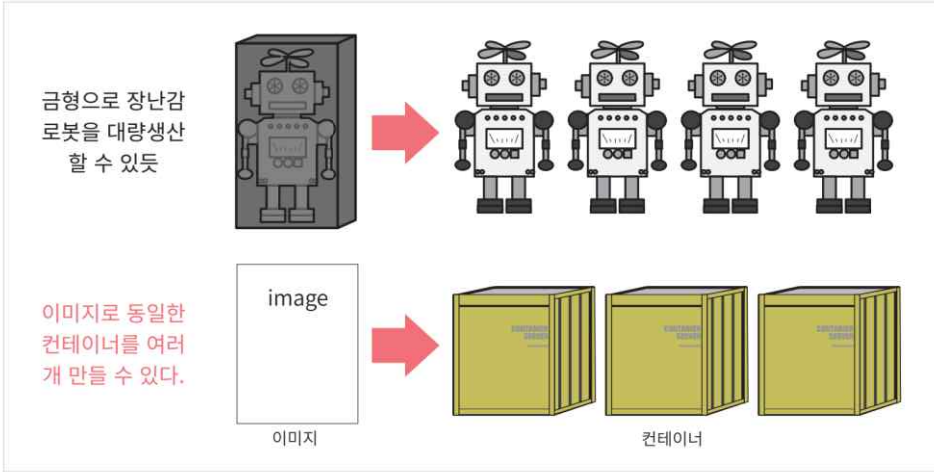


그림 2-2-2 하나의 이미지로 동일한 컨테이너를 여러 개 만들 수 있다.

1.이미지와 컨테이너

- 컨테이너로 이미지를 만들 수 있다.

그림 2-2-4 새로운 이미지를 사용해 '개조된' 컨테이너를 대량으로 만들 수 있다.

- 도커 엔진간 이동이 가능하다

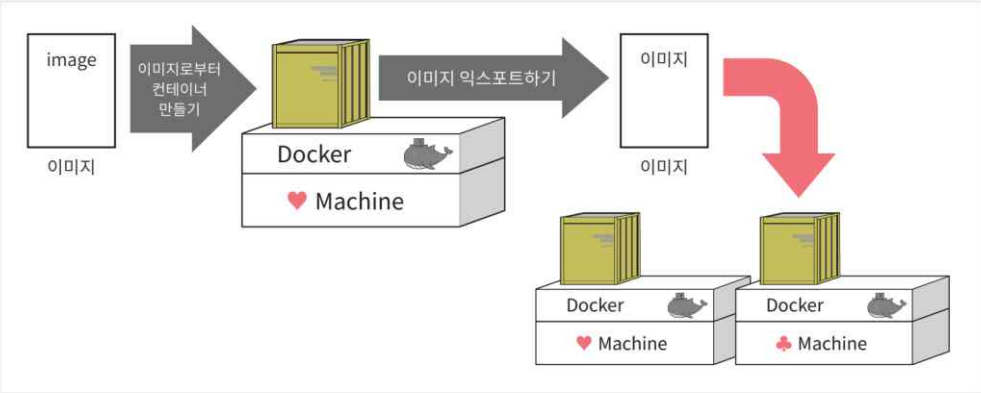


그림 2-2-5 컨테이너로부터 만든 이미지를 이용하면 다른 환경에서 원래의 컨테이너를 복원할 수 있다.

2.도커 허브와 도커 이미지

- 도커 허브(https://hub.docker.com)
  - 도커 레지스터리(이미지를 배포하는 사이트)
  - 공개된 컨테이너 이미지가 모여있는 사이트

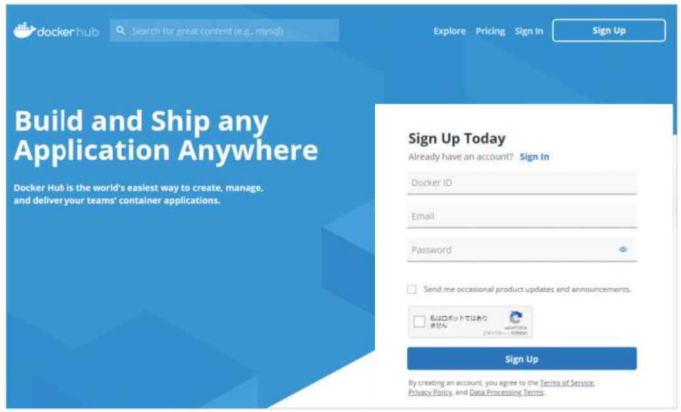
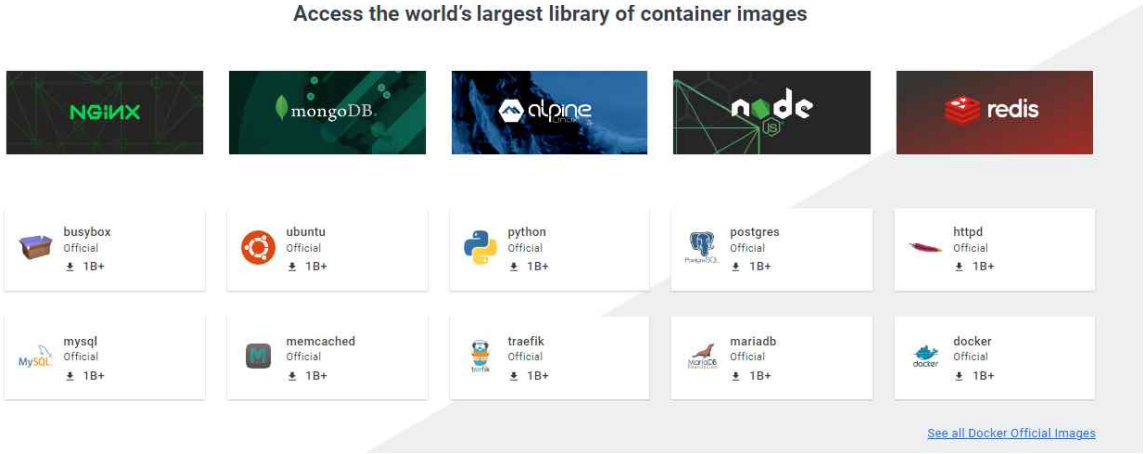


그림 2-2-6 도커 허브



2.도커 허브와 도커 이미지

• 도커허브에서 제공하는 이미지

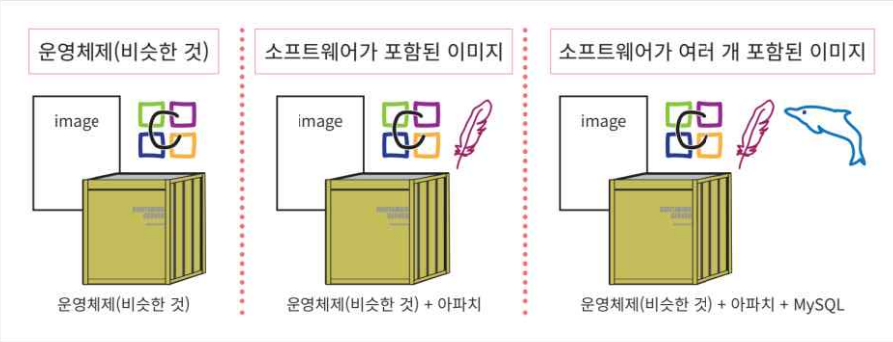


그림 2-2-8 도커 허브에는 다양한 이미지가 제공된다.

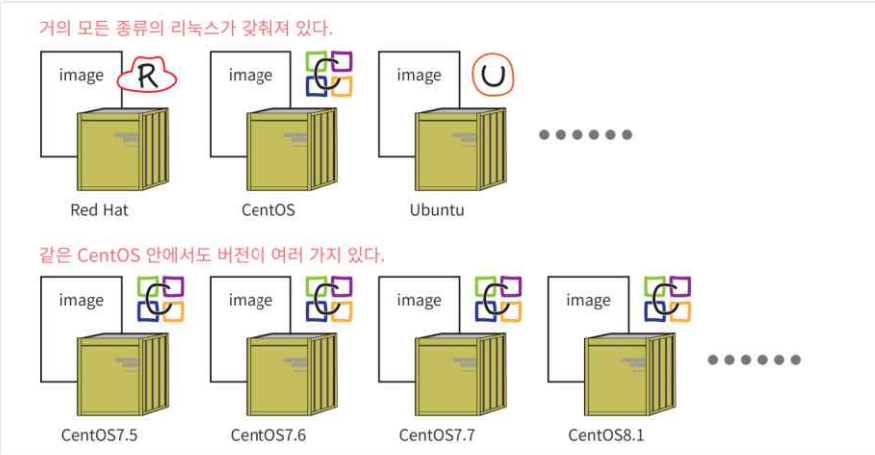


그림 2-2-9 운영체제(비슷한 것)만 들어있는 컨테이너도 여러 가지가 존재한다.

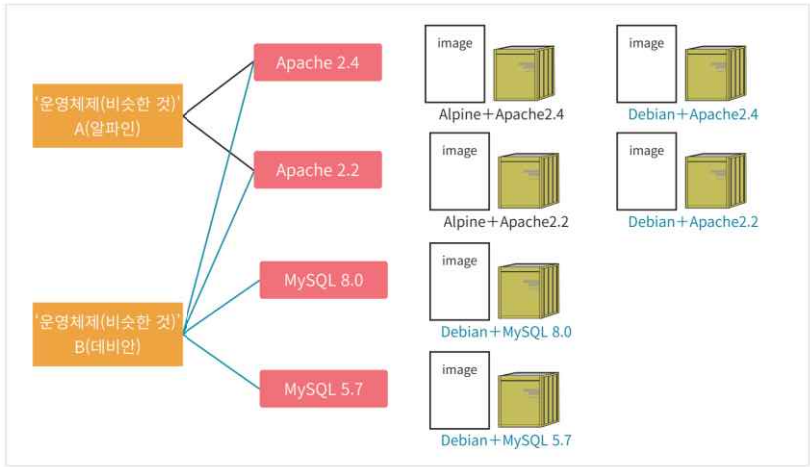


그림 2-2-10 '운영체제(비슷한 것)'과 소프트웨어의 조합은 버전에 따라 더 다양해진다.

2.도커 허브와 도커 이미지

- 안전한 컨테이너 이미지 고르는 방법
  - 공식 이미지를 사용한다
  - 커스텀 이미지를 직접 만들어 사용한다.

- 다양한 형태로 조합이 가능한 컨테이너
  - 도커 사용 원칙 '한 컨테이너에 한 프로그램'
  - 보안 및 유지 관리 측면에서 유리
  - 한 컨테이너에 필요한 프로그램을 모두 설치해도 가능

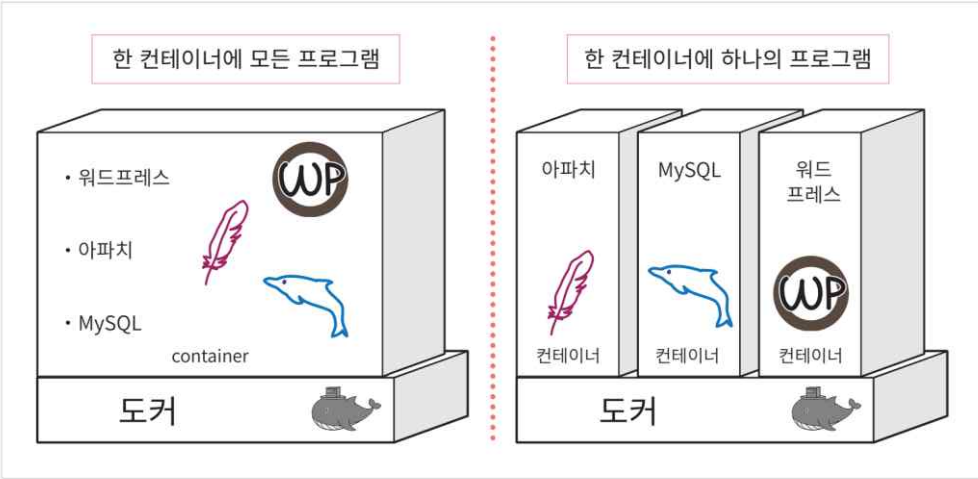


그림 2-2-11 한 컨테이너에 하나의 프로그램 또는 여러 개의 프로그램을 넣을 수도 있다.

2장 도커의 동작원리

3절 도커 컨테이너의 생애주기와 데이터 저장

1.도커 컨테이너의 생애주기와 데이터 저장

- 도커 컨테이너는 **일회용품**으로 생각

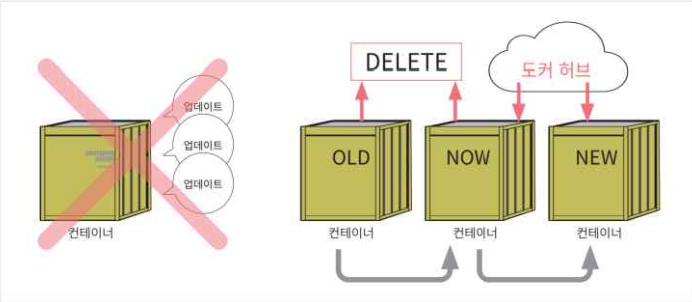


그림 2-3-1 컨테이너는 '쓰고 버리는' 일회용품에 가깝다.

- 컨테이너는 일반적으로 여러 개를 동시에 가동하는 경우가 많으므로 유지보수가 용이하지 않을 수 있다.
- 따라서 업데이트시 새로운 이미지로부터 새로운 컨테이너를 만들어 배포하는 방식을 이용

컨테이너의 생애주기

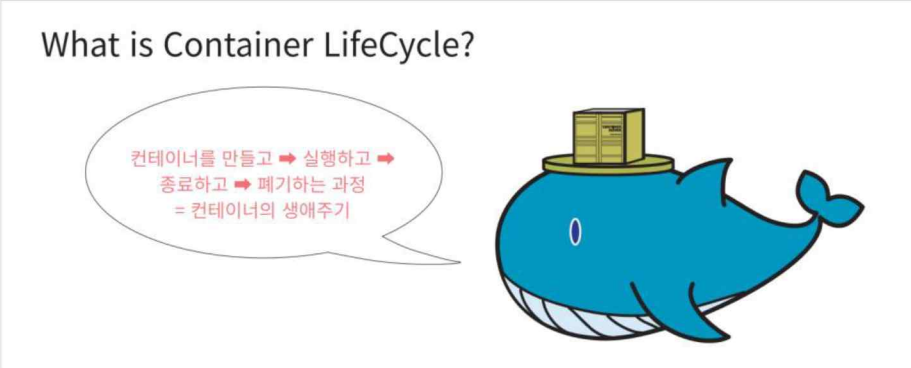


그림 2-3-2 컨테이너의 생애주기

2.데이터 저장

- 컨테이너 삭제시 그 안에 있던 파일도 함께 삭제
- 이를 방지하기위해 일반적으로 물리적 서버의 디스크를 마운트해 저장

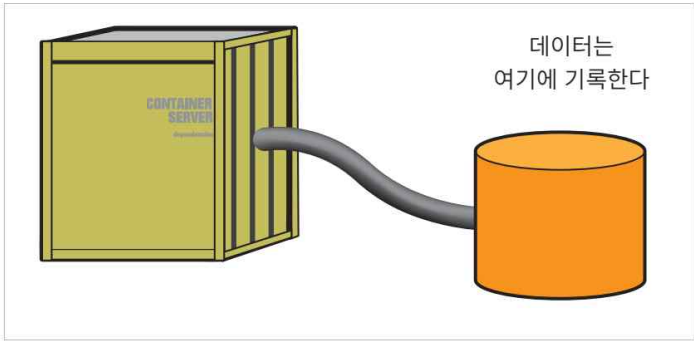


그림 2-3-3 물리적 컴퓨터의 일부를 마운트하고 여기에 데이터를 저장한다.

- 컨테이너를 삭제해도 데이터를 외부에 안전하게 보관
- 도커 엔진에 장애가 발생해도 데이터는 그대로 보존

2.데이터 저장

• 데이터의 외부 저장

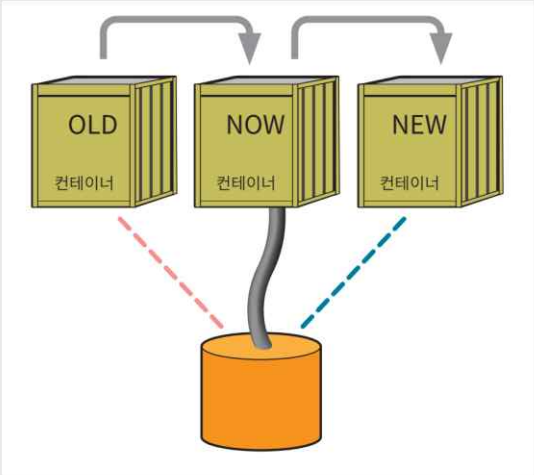


그림 2-3-4 컨테이너가 바뀌어도 외부에 저장된 데이터는 그대로 사용할 수 있다.

- 컨테이너가 바뀌어도 데이터는 계속 이용할 수 있다.
- 데이터나 설정 파일등은 외부에 저장
- 프로그램 개발시 외부에 저장하지않는 경우도 있으니 컨테이너 삭제시 확인 필요

2장 도커의 동작 원리

4절 도커의 장점과 단점

1. 도커의 구조와 특성 및 장단점

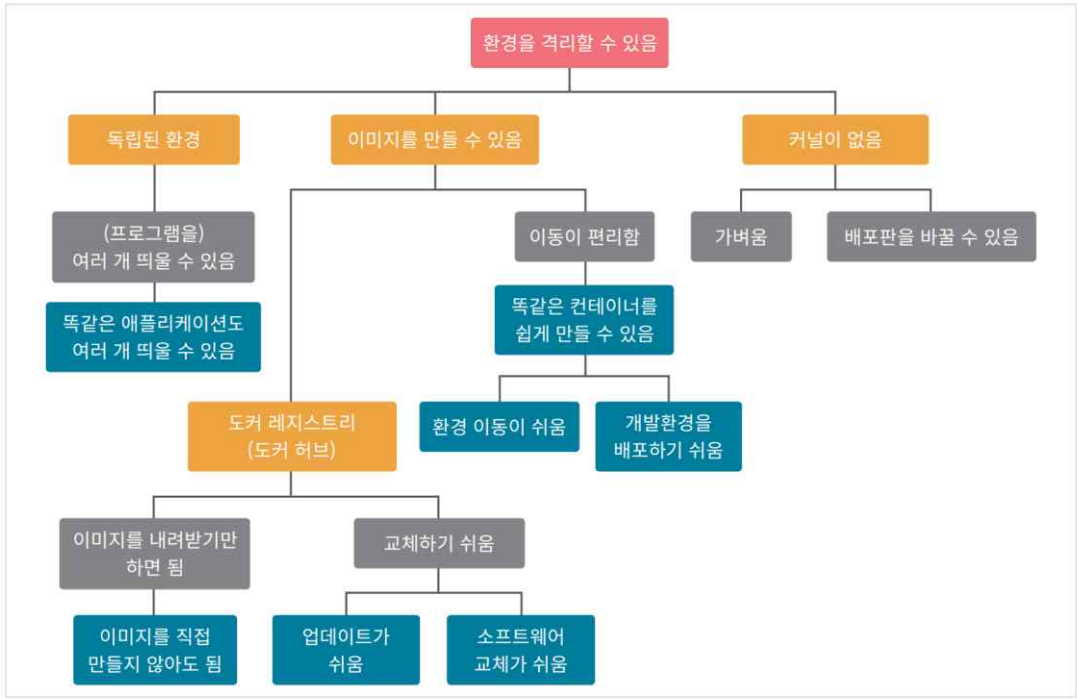


그림 2-4-1 도커의 성질

- 독립된 환경
- 이미지 생성 가능
- 컨테이너에 커널 포함되지않아 가벼움

⇨ 격리 가능

2.도커의 장점과 단점

- 장점
  - 1대의 물리적인 서버에 여러 대의 서버를 구동할 수 있다.
  - 서버 관리가 용이하다
    - 다른 소프트웨어에 영향을 미치지 않음
    - 컨테이너 교체나 수정이 쉬워 환경 이전도 수월
  - 서버 고수가 아니어도 다루기 수월
    - 명령 한줄로 서버 구축 완료

- 도커의 단점
  - 리눅스 소프트웨어만 지원
  - 호스트 서버에 문제발생시 모든 컨테이너에 영향을 미침
  - 컨테이너를 하나만 사용하는 경우에는 비효율적

3.도커의 주용도

- 팀원에게 동일한 개발환경 제공
  - 같은 프로젝트 팀원에게 개발환경을 동일하게 제공
  - 프로젝트별 컨테이너를 따로 사용할 수 있음
  - 개발환경과 운영환경의 차이가 없음

- 새로운 버전의 테스트
  - 새로운 운영체제나 라이브러리 버전을 개발환경에서 먼저 테스트하고 운영환경에 컨테이너로 적용

- 동일한 서버가 여러 대 필요한 경우
  - 1대의 물리 서버에 여러 개의 컨테이너를 이용할 수 있으므로 비용 절약
  - 명령 한 줄로 원하는 만큼 서버 구동 가능
  - 스케일링에 유리하므로 웹 서버나 API 서버로 많이 활용

3장 도커를 사용해보자

1절 도커 준비

1.도커는 모든 운영체제에서 사용 가능

• 도커 환경을 구축하는 방법

1. 리눅스 컴퓨터에서 도커 사용하기
2. 가상 머신이나 렌탈 환경에 도커를 설치하고 윈도우나 macOS를 통해 사용하기
3. 윈도우용/macOS용 도커 사용하기

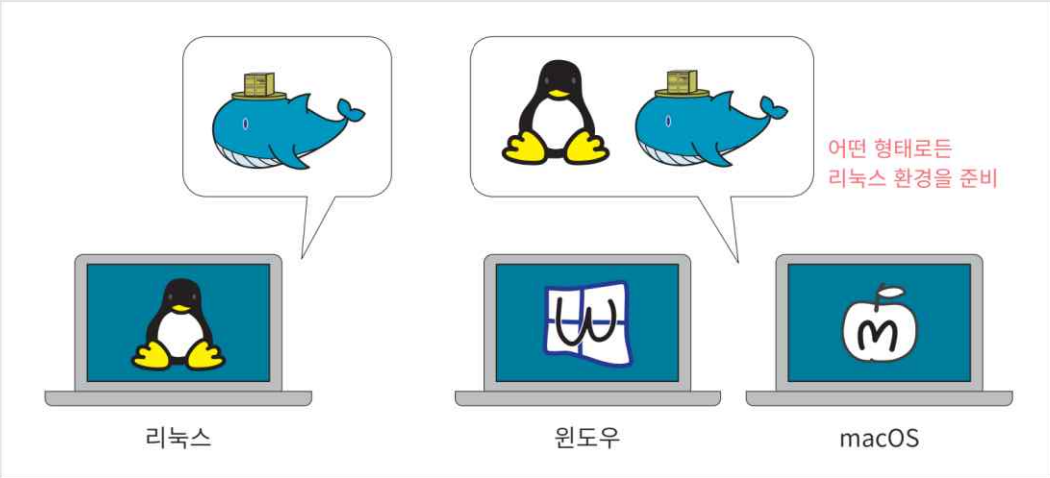


그림 3-1-1 리눅스, 윈도우, macOS 각각의 환경에 맞춰 도커 환경을 준비한다.

명령어는 공통적으로 사용

2.윈도우용/macOS용 도커 사용하기

- 리눅스 운영체제가 들어있는 패키지, '도커 데스크톱' 이용

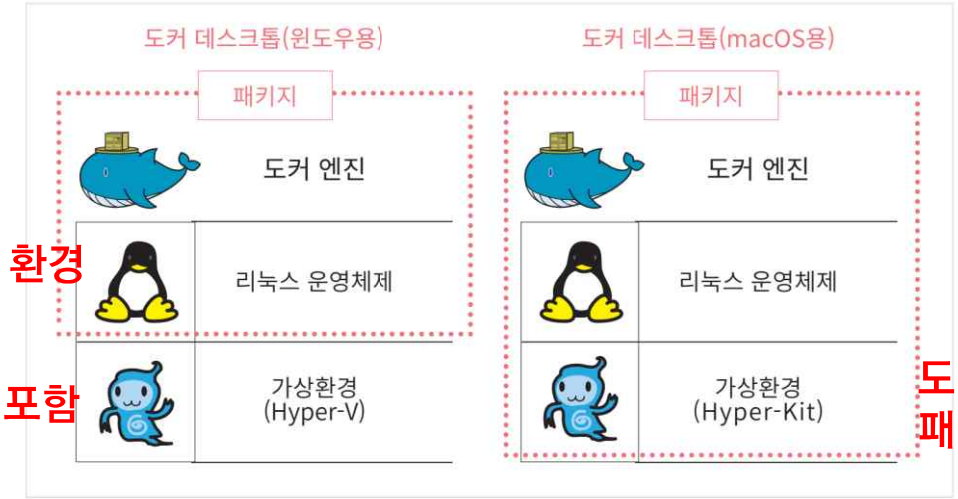


그림 3-1-2 도커 데스크톱에는 도커 엔진 외에 실행환경이 함께 패키징돼 있다.

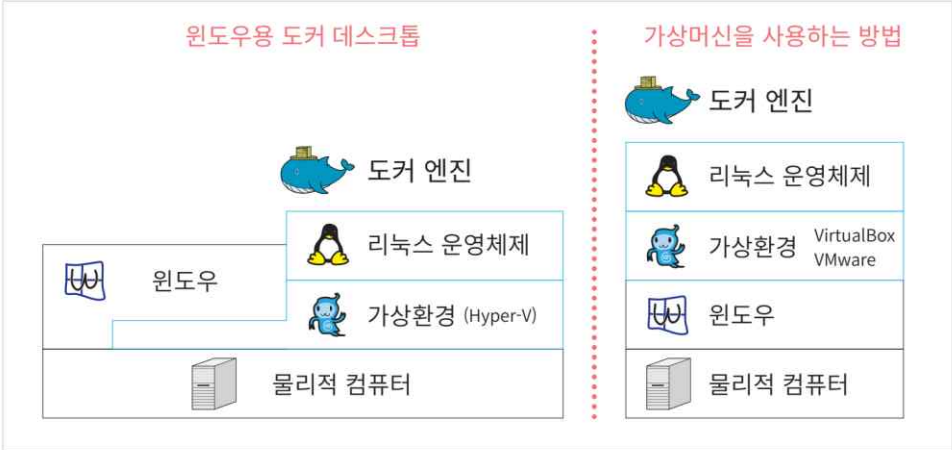


그림 3-1-3 도커 데스크톱과 가상화 소프트웨어를 사용한 방법의 차이점

2.윈도우용/macOS용 도커 사용하기

- 도커 데스크톱 일반 소프트웨어처럼 사용
- 도커 엔진을 조작할 때는 CLI(Command Line Interface) 방식 이용
- 리눅스는 도커 전용 OS로만 사용

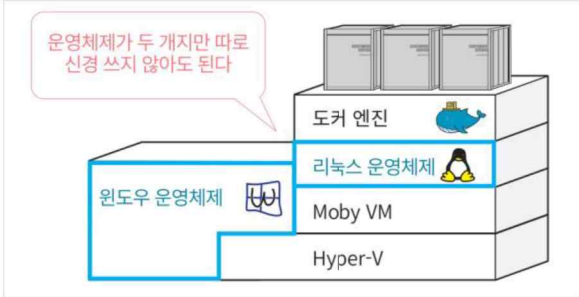


그림 3-1-4 도커 데스크톱을 사용할 때 운영체제가 두 개라는 점에 신경 쓰지 않아도 된다(그림은 윈도우용 도커 데스크톱과 뒤에 설명할 WSL2의 조합을 나타낸다).

- 윈도우용 가상환경(Hyper-V) 이용시 속도가 느려지거나 다른 가상화 SW와 충돌 발생 경우가 있음
- macOS에서는 약간 무겁게 느껴짐, M1,M2 칩으로 전환되면서 안정화 기간 필요

2. 윈도우용/macOS용 도커 사용하기

· 도커 사전 준비

윈도우용/macOS용 도커 데스크톱을 사용하기 위해 필요한 것

· 사용 조건을 만족하는 윈도우 또는 macOS

주의할 점

· 평소에 가상화 소프트웨어를 사용했다면 윈도우와 가상화 소프트웨어를 모두 최신 버전으로 업데이트해야 한다.

교재 55페이지 WSL2(Windows Subsystem for Linux2) COLUMN 참조

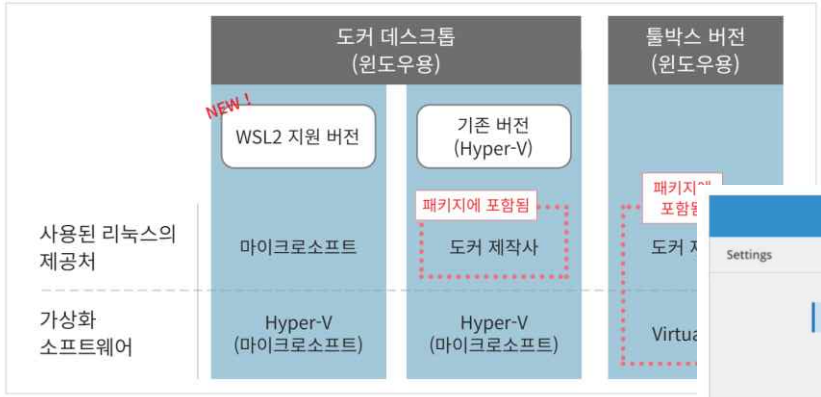


그림 3-1-6 도커 데스크톱의 두 가지 종류

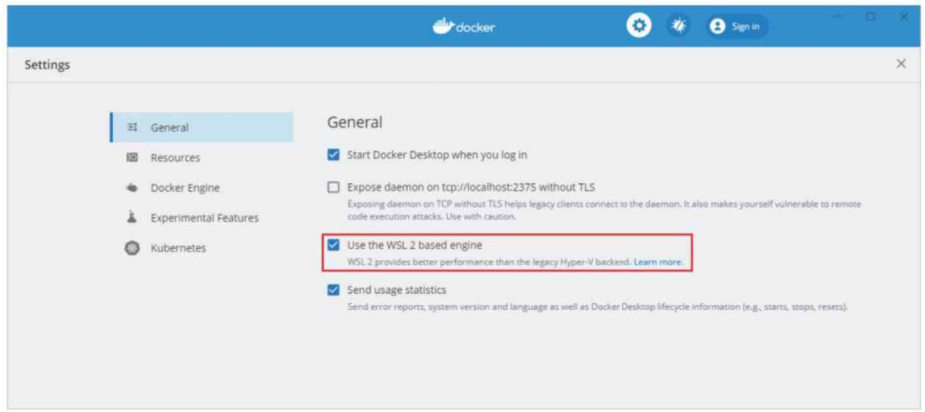


그림 3-1-7 도커 데스크톱의 설정(Settings) 화면(이 화면을 띄우는 방법은 3.2절을 참조)

3.도커 실행 조건

- 도커엔진 설치 조건
- 64비트 운영체제
- 윈도우
  - 윈도우10 64비트 버 이후 버전)
  - 윈도우10 64비트 버
  - Hyper-V 및 Cont
  - CPU : SLAT 기능을
  - 메모리 4GB 이상
  - BIOS에서 Virtualization 활성화
- masOS
  - 2010년 이후 발매된 모델
  - macOS 10.13(하이 시에라)이후 버전
  - 메모리 4GB 이상

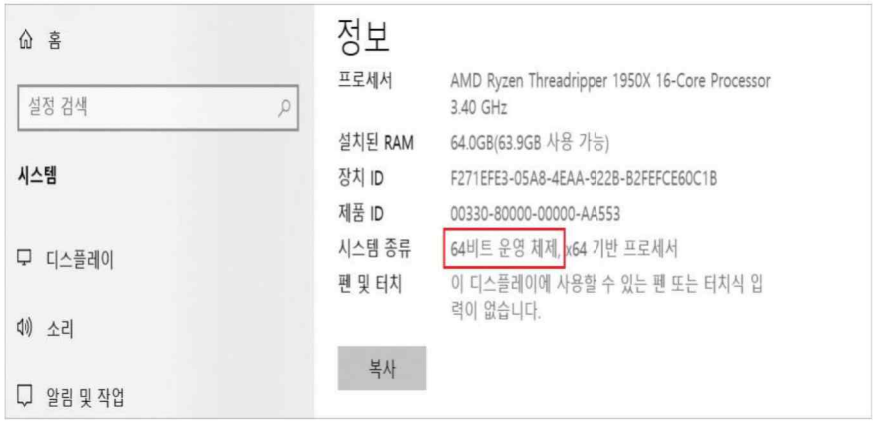


그림 3-1-9 시스템 정보 확인

3.도커 실행 조건

- 리눅스
  - 배포판 및 버전

지원하는 리눅스 배포본 및 버전

배포본	버전
CentOS	CentOS 7 이후
우분투	우분투 16.04 이후
데비안	데비안 9(스트레치) 이후
페도라	페도라 30 이후

  - 리눅스 커널 3.10 이후
  - Iptables 1.4 이후
  - Git 1.7 이후
  - XZ 도구 4.9 이후
  - procps 와 cgroup 계층 준수

3장 도커를 사용해보자

2절 도커 설치

부록2 macOS용 도커 설치

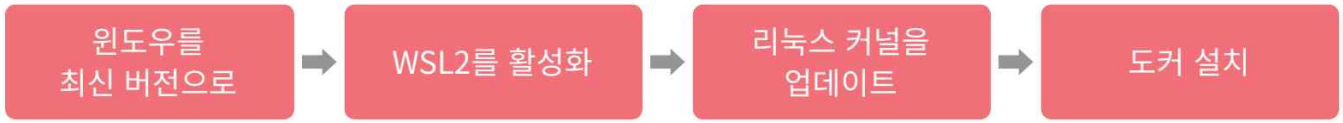
부록3 리눅스용 도커 설치

1.도커 설치

- 윈도우용 도커 데스크톱 설치

  WSL2 사용 기준 (윈도우 1903/19019 이후 버전) 부록 1 참조

- 설치 순서



1) 윈도우 최신버전으로 업데이트(1903/1909이후 버전 확인)
2) Windows 기능 켜기/끄기 창에서
   Linux용 Windows 하위시스템과 가상머신 플랫폼 체크

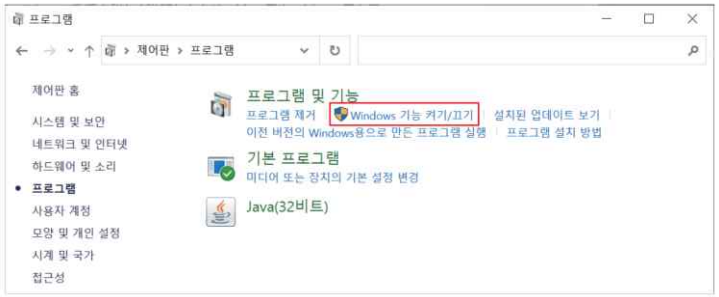


그림 3-2-3 [Windows 기능 켜기/끄기]를 클릭한다.

3) 컴퓨터 재기동

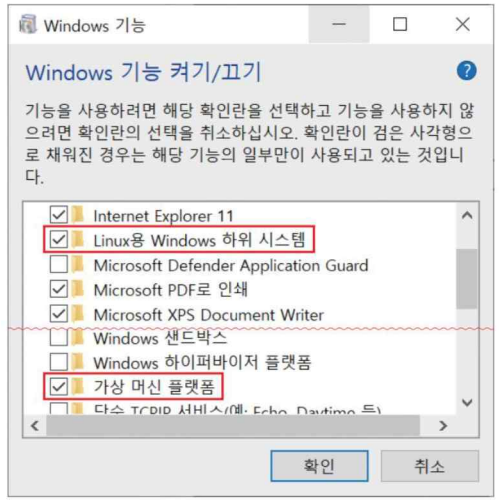


그림 3-2-4 두 가지 기능을 체크한다.

1. 도커 설치

4) 리눅스 커널 다운로드 및 업데이트

https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi

다운로드 후 실행

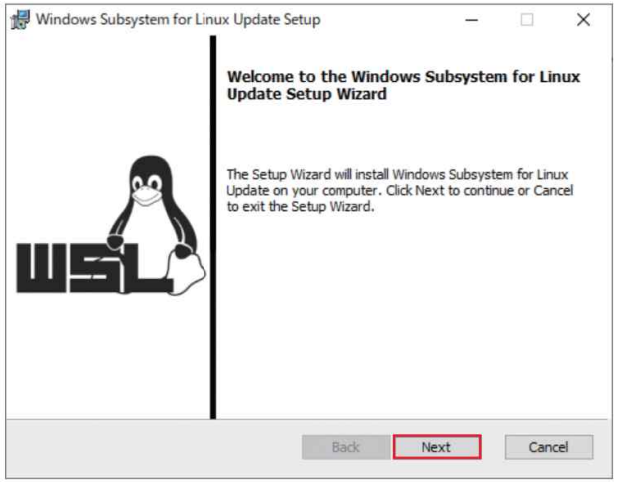


그림 3-2-6 리눅스 커널 업데이트

참고 : https://learn.microsoft.com/ko-kr/windows/wsl/install

1.도커 설치

5) 윈도우용 데스크톱 다운로드 및 설치

https://docs.docker.com/desktop/install/windows-install/

그림 3-2-8 설치 파일을 더블클릭해 실행

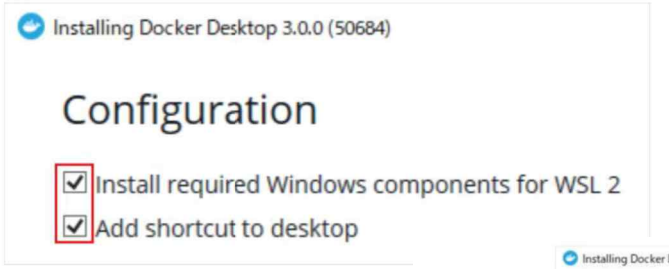


그림 3-2-9 윈도우용 도커 데스크톱의 설치 진행 화면

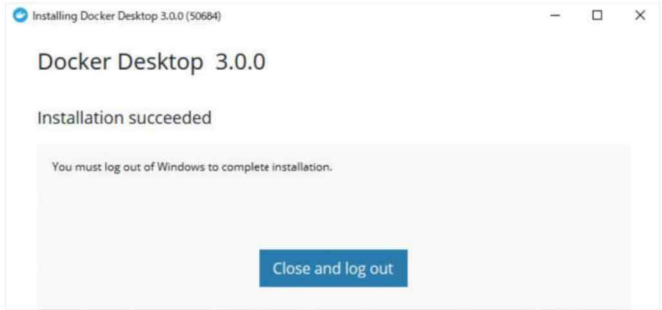
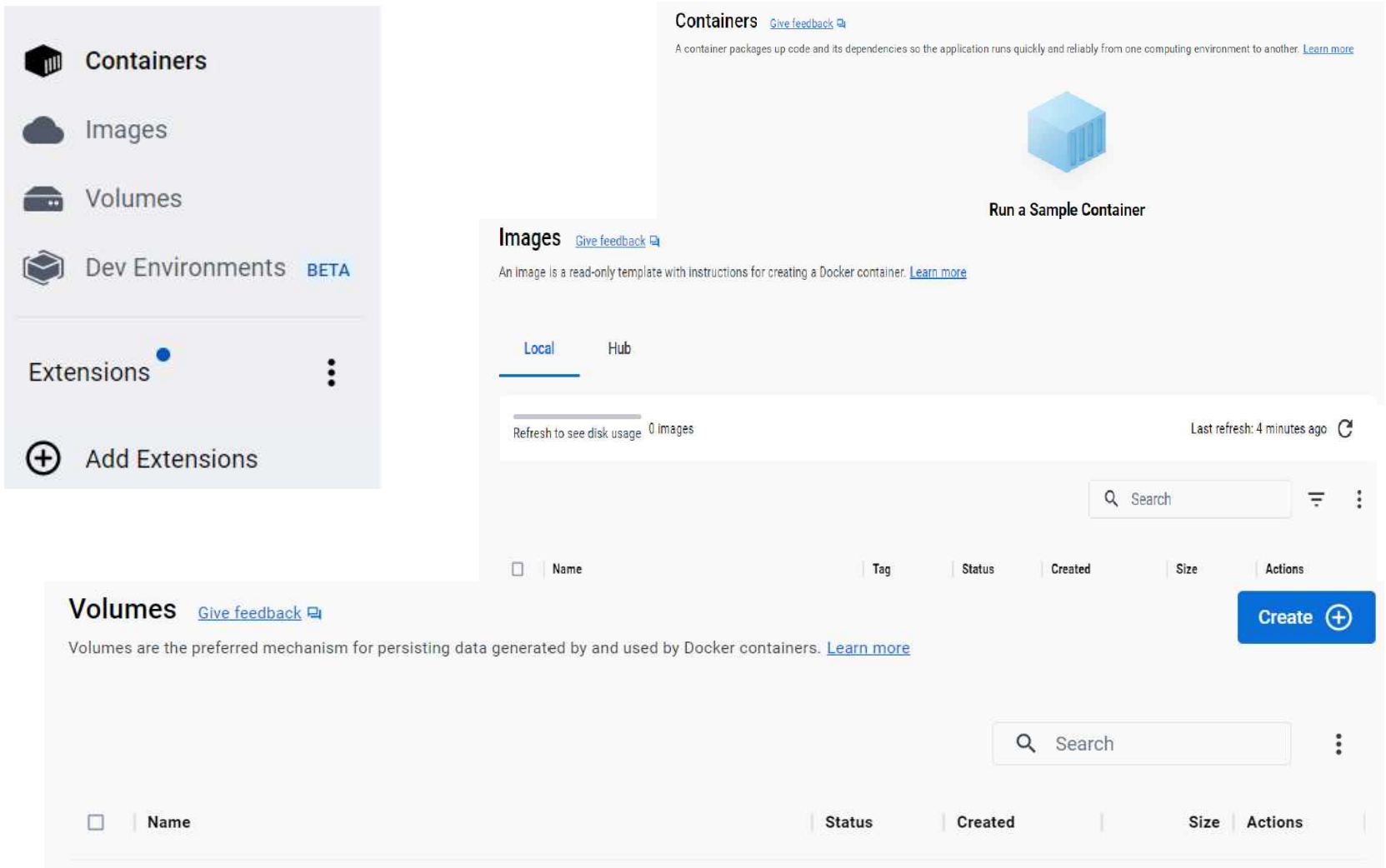


그림 3-2-10 설치 완료

2. 도커 데스크톱 화면

- 홈 화면



2. 도커 데스크톱 화면

• 도커 실행여부 (왼쪽 하단)

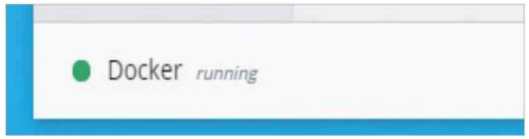


그림 3-2-16 정상 동작 중임을 나타내는 마크

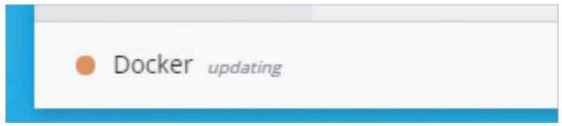
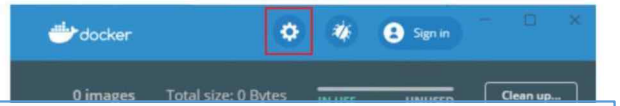
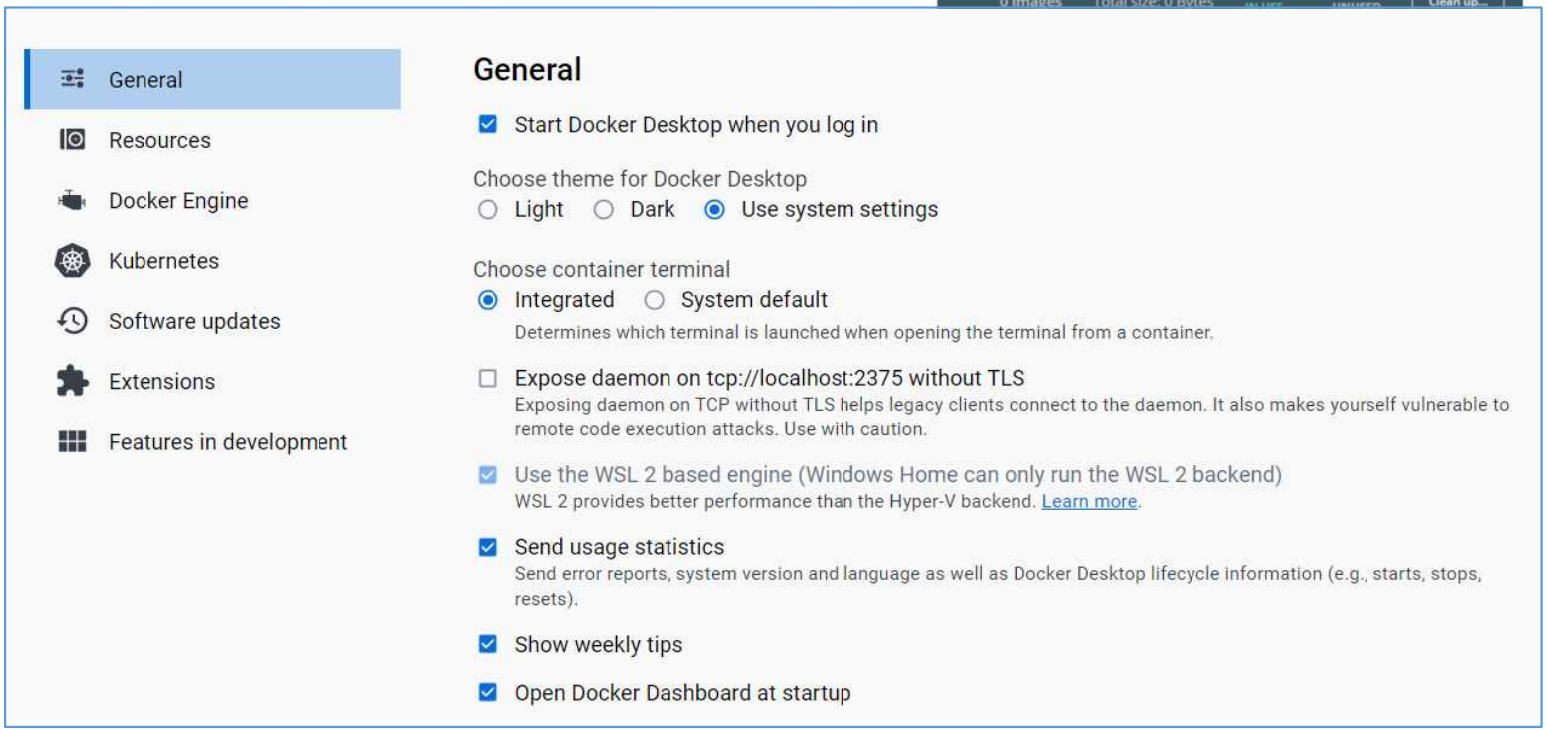


그림 3-2-17 업데이트 또는 중지 상태임을 나타내는 마크

• 도커 설정 화면 (오른쪽 상단)





2. 도커 데스크톱 화면

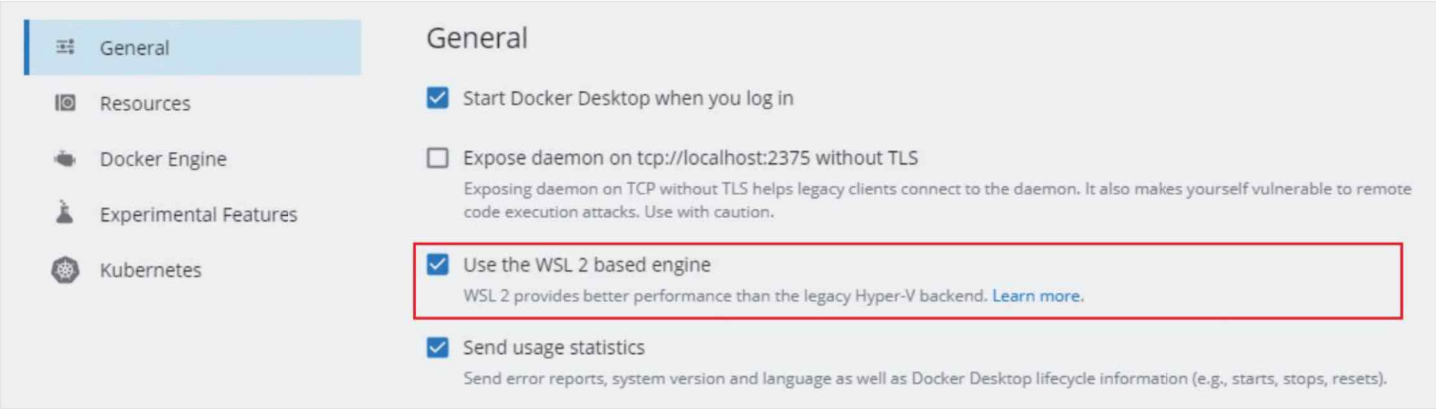


그림 3-2-19 도커 엔진이 WSL2에서 동작하는지 확인

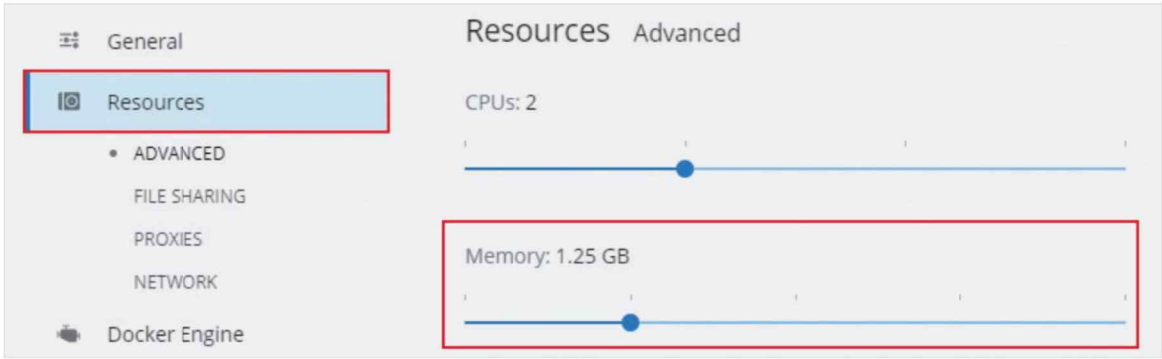


그림 3-2-20 'Resource' 탭에서 'Memory'의 설정값 조정

4장 컨테이너를 실행해보자

1절 도커 엔진 시작하기와 종료하기

1.도커 엔진 시작/종료

- 도커 엔진 자동 실행
  - 컨테이너를 실행하지않으면 컴퓨터의 리소스는 거의 차지하지 않음
- 도커엔진이 종료되면 컨테이너는 모두 정지됨
- 도커엔진은 자동으로 구동되나 컨테이너는 자동으로 구동되지 않음

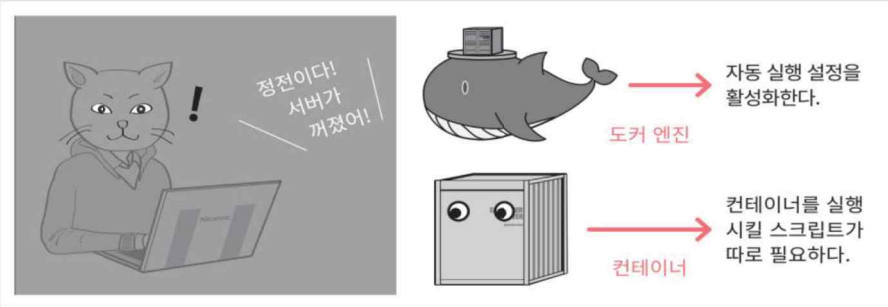


그림 4-1-1 도커 엔진은 자동 실행 설정이 가능하지만 컨테이너는 그렇지 않다.

OS	도커엔진 시작	도커엔진 종료	자동실행설정
윈도우	시작 → Docker Desktop 선택	도커 데스크톱 창 닫기	기본으로 자동 구동
macOS	파인더 → '애플리케이션' 폴더 → Docker Desktop 선택	상태막대에서 도커 클릭 -> Quit Docker Desktop 선택	상태막대에서 도커 클릭 → Preference → Start Docker Desktop when you login 선택
리눅스	sudo systemctl start docker	sudo systemctl stop docker	sudo systemctl enable docker

4장 컨테이너를 실행해보자

2절 컨테이너의 기본적인 사용방법

1.도커 기본 명령어

- 도커 명령의 구성

docker 상위명령 하위명령 [옵션] 대상 [인자]

상위명령: container, image, ., ., .

하위명령: run, pull, start, ., .

start/stop/run 명령인 경우 container 생략 가능

[옵션]: 명령의 세부 설정 시 이용
-d : 백그라운드 실행
-i or -t : 대화형 실행

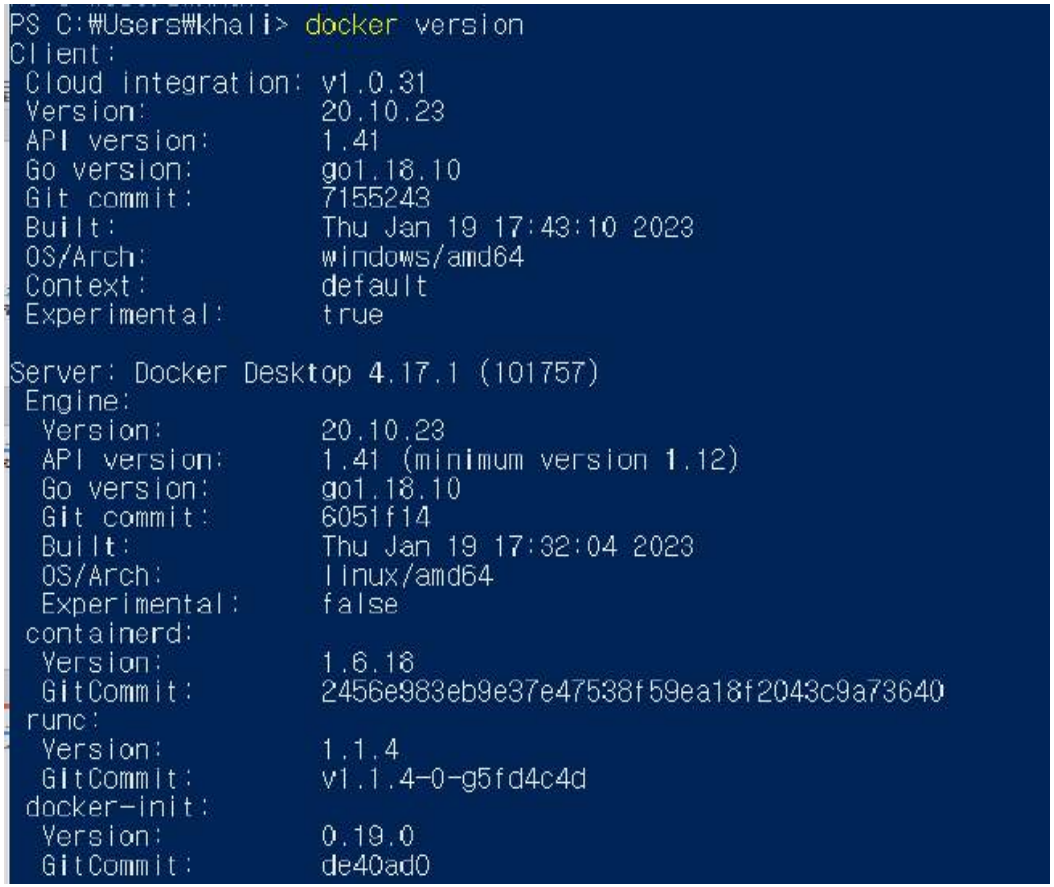
대상: 명령을 수행할 구체적인 대상

[인자]: 대상에 전달할 값 지정

1.도커 기본 명령어

<실습> 버전 확인

```
docker version
```



2.대표적인 docker 명령

- 컨테이너를 다루는 도커 명령

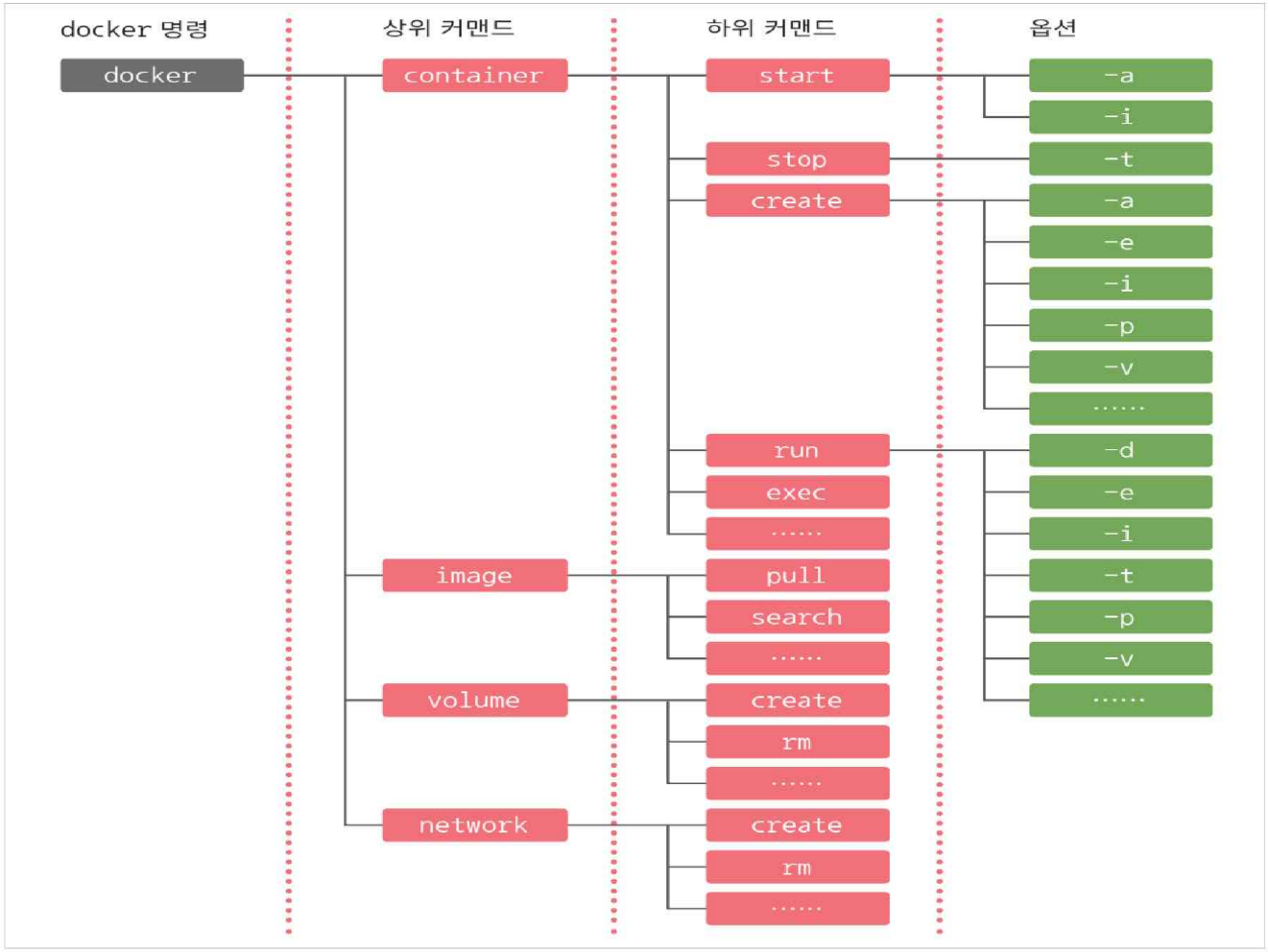


그림 4-2-1 대표적인 명령어

2.대표적인 docker 명령

• 컨테이너 제어 명령

`docker [container] 하위명령 옵션`

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
start	컨테이너를 실행	O	-i
stop	컨테이너를 정지	O	거의 사용하지 않음
create	도커 이미지로부터 컨테이너를 생성	O	--name -e -p -v
run	도커 이미지를 내려받고 컨테이너를 생성해 실행함(다운로드는 필요한 경우에만). docker image pull, docker container create, docker container start라는 세 개의 명령을 하나로 합친 것과 같다.	O	--name -e -p -v -d -i -t
rm	정지 상태의 컨테이너를 삭제	O	-f -v
exec	실행 중인 컨테이너 속에서 프로그램을 실행	O	-i -t
ls	컨테이너 목록을 출력	*1	-a
cp	도커 컨테이너와 도커 호스트 간에 파일을 복사	O	거의 사용하지 않음
commit	도커 컨테이너를 이미지로 변환	O	거의 사용하지 않음

*1: 생략형은 docker ps

2.대표적인 docker 명령

• 이미지 제어 명령

docker image 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
pull	도커 허브 등의 리포지토리에서 이미지를 내려받음	O	거의 사용하지 않음
rm	도커 이미지를 삭제	*2	거의 사용하지 않음
ls	내려 받은 이미지의 목록을 출력	X	거의 사용하지 않음
build	도커 이미지를 생성	O	-t

*2: 생략형은 docker rmi

2.대표적인 docker 명령

• 볼륨 제어 명령

```
docker volume 하위명령   옵션
```

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
create	볼륨을 생성	X	--name
inspect	볼륨의 상세 정보를 출력	X	거의 사용하지 않음
ls	볼륨의 목록을 출력	X	-a
prune	현재 마운트되지 않은 볼륨을 모두 삭제	X	거의 사용하지 않음
rm	지정한 볼륨을 삭제	X	거의 사용하지 않음

2.대표적인 docker 명령

• 네트워크 제어 명령

```
docker network 하위명령어   옵션
```

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
connect	컨테이너를 도커 네트워크에 연결	X	거의 사용하지 않음
disconnect	컨테이너의 도커 네트워크 연결을 해제	X	거의 사용하지 않음
create	도커 네트워크를 생성	X	거의 사용하지 않음
inspect	도커 네트워크의 상세 정보를 출력	X	거의 사용하지 않음
ls	도커 네트워크의 목록을 출력	X	거의 사용하지 않음
prune	현재 컨테이너가 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

2.대표적인 docker 명령

• 기타 명령어

그 밖의 상위 커맨드

상위 커맨드	내용
checkpoint	현재 상태를 일시적으로 저장한 후, 나중에 해당 시점의 상태로 되돌릴 수 있다. 현재 실험적 기능이다.
node	도커 스웜의 노드를 관리하는 기능
plugin	플러그인을 관리하는 기능
secret	도커 스웜의 비밀값 정보를 관리하는 기능
service	도커 스웜의 서비스를 관리하는 기능
stack	도커 스웜 또는 쿠버네티스에서 여러 개의 서비스를 합쳐 구성한 스택을 관리하는 기능
swarm	도커 스웜을 관리하는 기능
system	도커 엔진의 정보를 확인하는 기능

• 단독 명령어

단독 커맨드[9]

단독 커맨드	내용	주요 옵션
login	도커 레지스트리에 로그인	-u -p
logout	도커 레지스트리에 로그아웃	거의 사용하지 않음
search	도커 레지스트리를 검색	거의 사용하지 않음
version	도커 엔진 및 명령행 도구의 버전을 출력	거의 사용하지 않음

4장 컨테이너를 실행해보자

3절 컨테이너의 생성과 삭제, 실행, 정지

1.도커 생성 및 실행

• 도커 실행

```
docker [container] run  옵션
```

- 도커 생성 및 실행 기능, 필요시 이미지 다운로드 기능도 수행
- docker pull + docker create + docker start

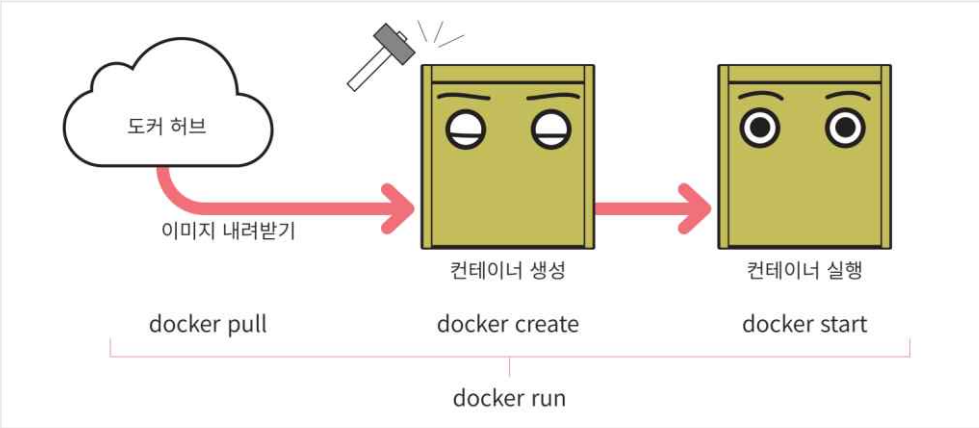


그림 4-3-1 docker run 커맨드는 세 가지 역할을 수행한다.

1.도커 생성 및 실행

• 도커 컨테이너의 생애주기

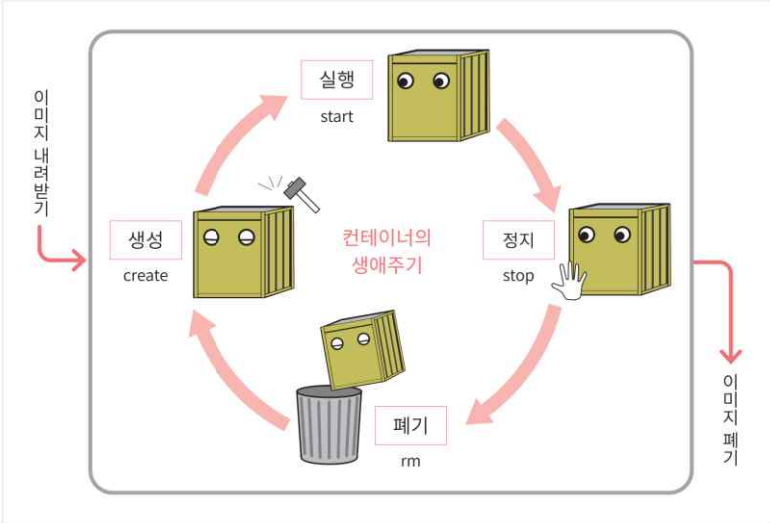


그림 4-3-2 컨테이너의 생애주기

- 동작 중인 컨테이너는 삭제 불가능
- 컨테이너 정지 후 삭제

```
docker stop & docker rm
```

1.도커 생성 및 실행

• 컨테이어 생성 및 실행

```
docker run 옵션 이미지 [인자]
```

주요 옵션

옵션 형식	내용
--name 컨테이너_이름	컨테이너 이름을 지정함
-p 호스트_포트번호:컨테이너_포트번호	포트 번호를 지정함
-v 호스트_디스크:컨테이너_디렉터리	볼륨을 마운트함
--net=네트워크_이름	컨테이너를 네트워크에 연결함
-e 환경변수_이름=값	환경변수를 설정함
-d	백그라운드로 실행함
-i	**컨테이너와 대화형 모드로 연결함**
-t	**컨테이너에 터미널 연결함**
-help	사용 방법 안내 메시지를 출력함

-p는 --publish, -v는 --volume, -e는 --env, -d는 --detach, -i는 --interactive, -t는 --tty의 생략형이다.

1.도커 생성 및 실행

- 컨테이너 정지 명령

```
docker stop 컨테이너명
```

- 컨테이너 삭제 명령

```
docker rm 컨테이너명
```

1.도커 생성 및 실행

• 컨테이너 목록 확인 명령

```
docker ps [옵션]
   or
docker container ls [옵션]
```

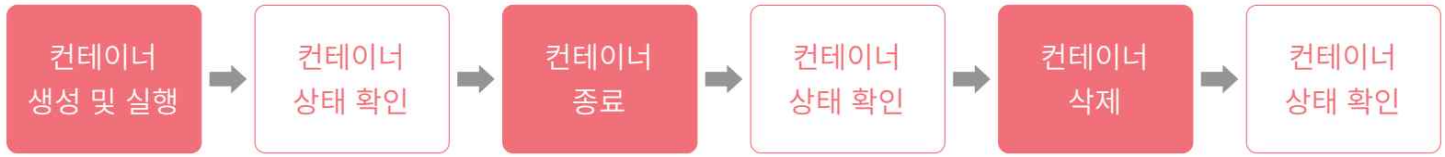
컨테이너 목록의 주요 항목

항목	내용
CONTAINER ID	컨테이너 식별자. 무작위 문자열이 할당된다. 본래는 64글자이지만 앞에서부터 12글자만 출력한다. 이 12글자만으로도 식별자 역할을 수행할 수 있다.
IMAGE	컨테이너를 만들 때 사용한 이미지의 이름
COMMAND	컨테이너 실행 시에 실행하도록 설정된 프로그램의 이름. 크게 신경 쓰지 않아도 된다.
CREATED	컨테이너 생성 후 경과된 시간
STATUS	컨테이너의 현재 상태. 실행 중이라면 'Up', 종료된 상태라면 'Exited'가 출력된다.
PORTS	컨테이너에 할당된 포트 번호. '호스트 포트 번호 -> 컨테이너 포트 번호' 형식으로 출력된다. 포트 번호가 동일할 경우 ->의 뒷부분은 출력되지 않는다.
NAMES	컨테이너 이름

1.도커 생성 및 실행

교재 102~105페이지

<실습> 컨테이너 생성, 실행, 상태 확인 및 종료



• 컨테이너 정보

항목	값
컨테이너 이름	apa000ex1
이미지 이름	httpd

옵션 항목

항목	내용
--name apa000ex1	apa000ex1이라는 이름으로 컨테이너를 생성
-d	백그라운드로 실행
httpd	아파치의 이미지 이름. 버전을 지정하지 않았으므로 가장 최신 버전(latest)이 사용된다.

4장 컨테이너를 실행해보자

4절 컨테이너의 통신

1.아파치 웹서버

- 아파치 : 웹서버를 제공하는 소프트웨어
- 컨테이너는 기본적으로 외부에서 접근 불가능한 상태로 실행
- 따라서 컨테이너 실행 시 설정 필요
  - docker run 명령의 옵션 이용
  - 실행 후 변경은 불가능함

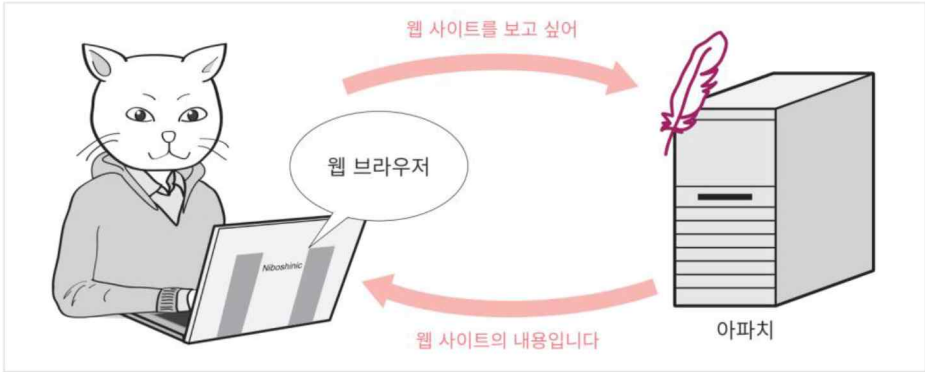


그림 4-4-1 컨테이너에서 동작하는 아파치의 웹 사이트를 열람

2.컨테이너와의 통신

- 외부와의 접속을 위해 **통신포트** 설정

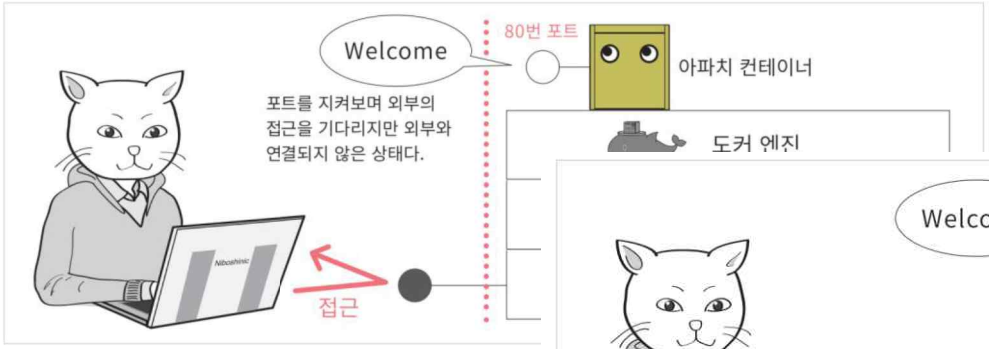


그림 4-4-2 컨테이너의 통신

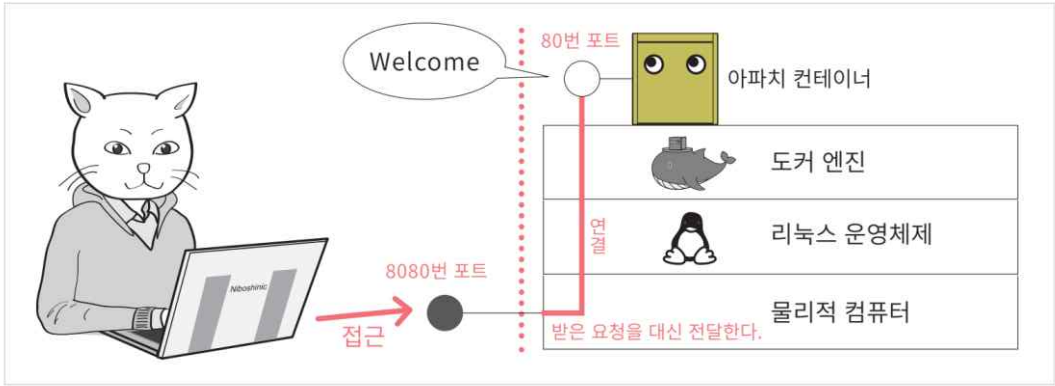


그림 4-4-3 컨테이너를 실행 중인 물리적 컴퓨터의 포트를 통해 컨테이너와 통신한다.

- 포트 설정

```
-p 호스트포트번호:컨테이너_포트번호
```

예) -p 8080:80

2.컨테이너와의 통신

- 여러 개의 웹 서버 컨테이너 구동 시 호스트_포트번호는 다르게 구성

컨테이너 A는 8080
컨테이너 B는 8081
컨테이너 C은 8082

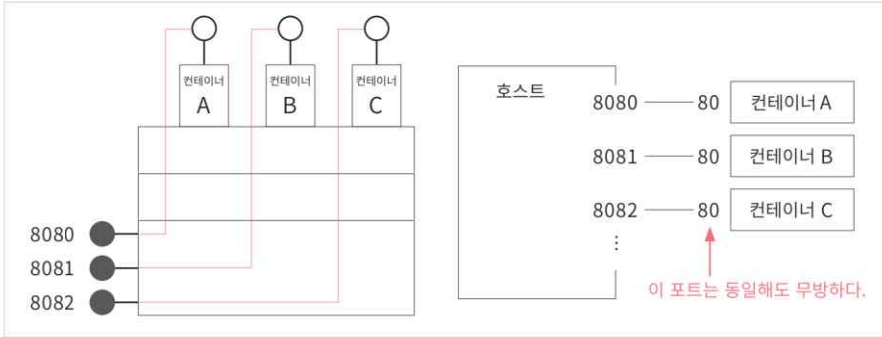


그림 4-4-4 컨테이너와 연결하는 호스트의 포트 번호를 겹치지 않게 설정한다.

- 웹 브라우저를 이용한 접근

http://localhost:8080로 접근

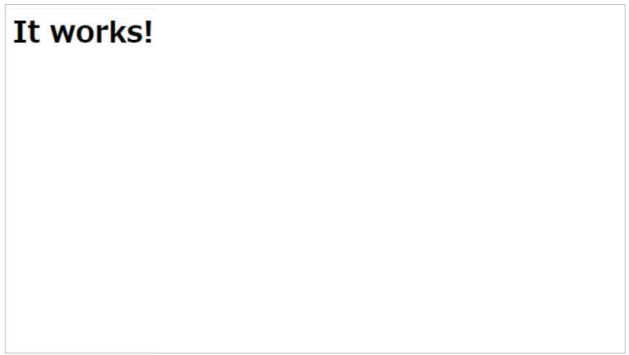
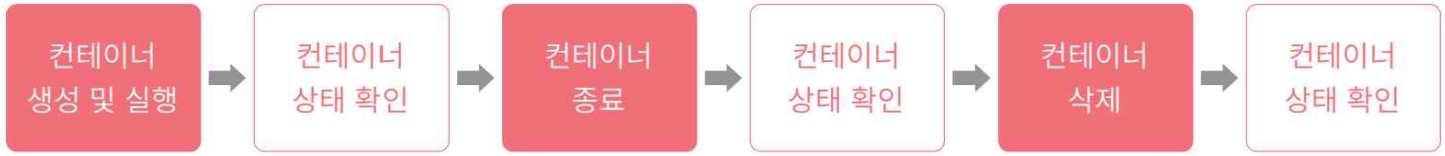


그림 4-4-5 아파치의 초기 화면

.컨테이너와의 통신

교재 110~113페이지

<실습> 통신이 가능한 컨테이너 생성



· 컨테이너 정보

항목	값
컨테이너 이름	apa000ex2
이미지 이름	httpd
포트 설정	8080:80

옵션 항목

항목	내용
--name apa000ex2	apa000ex2라는 이름으로 컨테이너를 생성
-d	백그라운드로 실행
-p 8080:80	호스트의 포트 8080을 컨테이너 포트 80으로 포워딩
httpd	아파치의 이미지 이름. 버전을 지정하지 않았으므로 최신 버전(latest)이 사용된다.

4장 컨테이너를 실행해보자

5절 컨테이너의 생성에 익숙해지기

1. 다양한 유형의 컨테이너

• 아파치 이외에 많이 사용되는 컨테이너

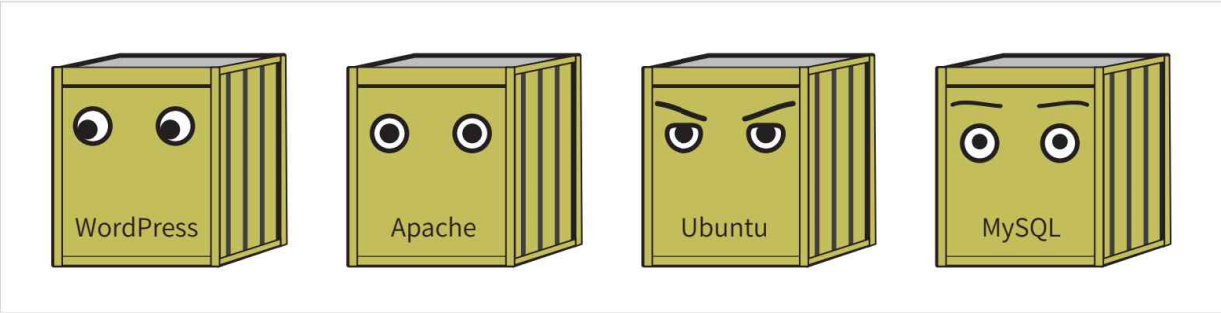


그림 4-5-1 다양한 유형의 컨테이너

• 리눅스 운영체제 컨테이너

리눅스 운영체제가 담긴 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
ubuntu	우분투	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
centos	CentOS	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
debian	데비안	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
fedora	페도라	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
busybox	BizyBox	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
alpine	알파인 리눅스	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.

1. 다양한 유형의 컨테이너

• 웹서버/데이터베이스 서버용 컨테이너

웹 서버 및 데이터베이스 서버용 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
httpd	Apache	-d로 백그라운드로 실행. -p로 포트 번호 지정
nginx	Nginx	-d로 백그라운드로 실행. -p로 포트 번호 지정
mysql	MySQL	-d를 사용. 실행 시 -e MYSQL_ROOT_PASSWORD와 같이 루트 패스워드를 지정
postgres	PostgreSQL	-d를 사용. 실행 시 -e POSTGRES_ROOT_PASSWORD와 같이 루트 패스워드를 지정
mariadb	MariaDB	-d를 사용. 실행 시 -e MYSQL_ROOT_PASSWORD와 같이 루트 패스워드를 지정

1. 다양한 유형의 컨테이너

• 프로그램 실행을 위한 런타임 및 그 외 소프트웨어

프로그래밍 언어의 런타임 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
openjdk	자바 런타임	-d를 사용하지 않고, 인자로 java 명령 등을 지정해 도구 형태로 사용한다.
python	파이썬 런타임	-d를 사용하지 않고, 인자로 python 명령 등을 지정해 도구 형태로 사용한다.
php	PHP 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공된다.
ruby	루비 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공된다.
perl	펄 런타임	-d를 사용하지 않고, 인자로 perl 명령 등을 지정해 도구 형태로 사용한다.
gcc	C/C++ 컴파일러	-d를 사용하지 않고, 인자로 gcc 명령 등을 지정해 도구 형태로 사용한다.
node	Node.js	-d를 사용하지 않고, 인자로 app 명령 등을 지정해 도구 형태로 사용한다.
registry	도커 레지스트리	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다.
wordpress	WordPress	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다. MySQL 또는 MariaDB가 필요하다. 접속에 필요한 패스워드는 -e 옵션으로 지정한다.
nextcloud	NextCloud	-d 옵션을 사용해 백그라운드로 실행한다. -p 옵션으로 포트 번호를 지정한다.
redmine	Redmine	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다. PostgreSQL 또는 MySQL이 필요하다.

1. 다양한 유형의 컨테이너

교재 116~119페이지

<실습> 아파치 컨테이너 여러 개 실행하기

- 컨테이너 정보

항목	값	값	값
컨테이너 이름	apa000ex3	apa000ex4	apa000ex5
이미지 이름	httpd	httpd	httpd
포트 설정	8081:80	8082:80	8083:80

1. 다양한 유형의 컨테이너

교재 120~123페이지

<실습> nginx 컨테이너 실행하기

• 컨테이너 정보

항목	값
컨테이너 이름	nginx000ex6
이미지 이름	nginx
포트 설정	8084:80

그림 4-5-4 nginx의 초기 화면

1. 다양한 유형의 컨테이너

교재 124~126페이지

<실습> MySQL 컨테이너 실행하기

• 컨테이너 정보

항목	값
컨테이너 이름	mysql000ex7
이미지 이름	mysql
MySQL 루트 패스워드	myrootpass

항목	내용
--name mysql000ex7	mysql000ex7이라는 이름으로 컨테이너를 생성
-dit	백그라운드에서 실행 및 키보드를 통해 컨테이너 내부의 파일 시스템을 조작
-e MYSQL_ROOT_PASSWORD=	MySQL의 루트 패스워드를 지정
mysql	MySQL 이미지 이름. 버전을 지정하지 않았으므로 최신 버전(latest)이 사용된다.

4장 컨테이너를 실행해보자

6절 이미지 삭제

1.이미지 삭제

- 컨테이너를 삭제해도 이미지는 남아있으므로 불필요한 이미지는 즉시 삭제

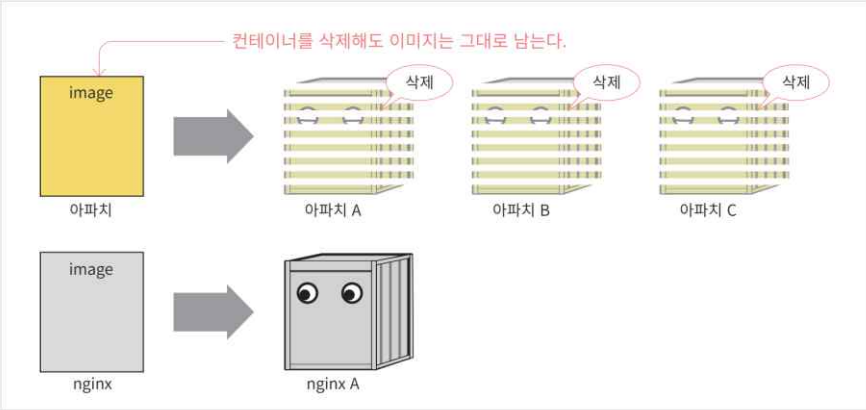


그림 4-6-1 컨테이너의 삭제와 이미지의 삭제

- 해당 이미지의 컨테이너가 남아 있으면 이미지 삭제 안됨

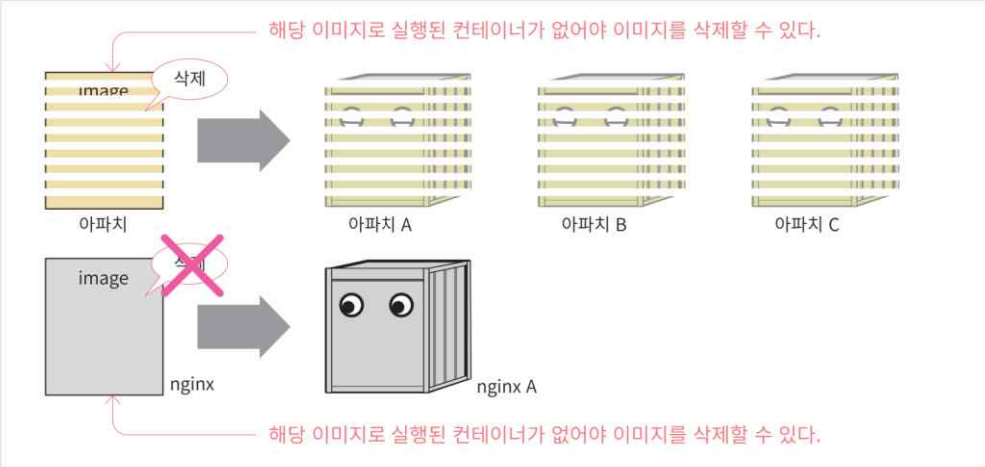


그림 4-6-2 해당 이미지로 실행된 컨테이너가 있으면 이미지를 삭제할 수 없다.

1.이미지 삭제

- 이미지 삭제 명령

```
docker image rm 이미지명 [이미지명 . . .] 또는 이미지ID
```

- 이미지 조회 명령

```
docker image ls
```

이미지 목록의 주요 항목

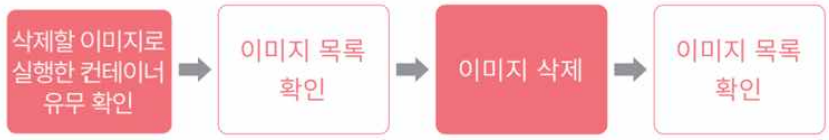
항목	내용
REPOSITORY	이미지 이름
TAG	버전 정보. 이미지를 내려받을 때 따로 지정하지 않으면 latest(최신 버전)를 내려받는다.
IMAGE ID	이미지 식별자. 본래는 64글자이지만 앞에서부터 12글자만 출력한다. 이 12글자만으로도 식별자 역할을 수행할 수 있다.
CREATED	이미지 생성 후 경과된 시간
SIZE	이미지의 전체 용량

이미지 삭제시 REPOSITORY:TAG(이미지이름:버전)으로 표현가능
교재133 COLUMN 참조

1.이미지 삭제

교재 130~133페이지

<실습> 다운받은 모든 이미지 삭제하기



5장 여러 개의 컨테이너를 연동해 실행해보자

1절 워드프레스 구축

1. 워드프레스 사이트 구축

- 워드프레스
  - 웹 사이트를 만들기 위한 소프트웨어
  - 워드프레스/아파치/데이터베이스/PHP 등 필요
  - 컨테이너 이미지
    워드프레스 공식 이미지(워드프레스/아파치/PHP) + MySQL 컨테이너

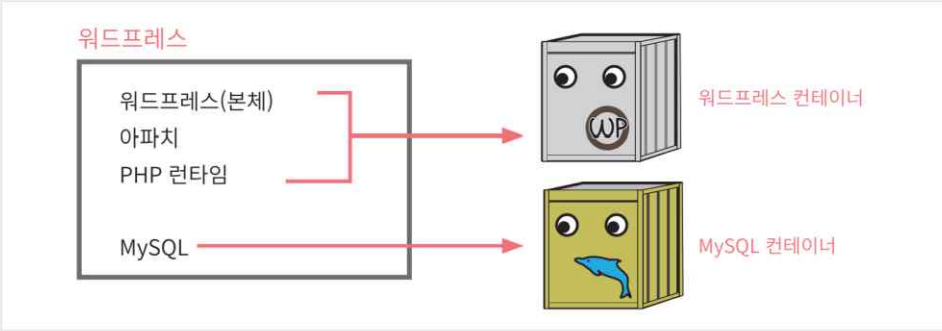


그림 5-1-1 워드프레스가 동작하려면 두 개의 컨테이너가 필요하다.

1. 워드프레스 사이트 구축

- 도커 네트워크 생성/삭제
  - 워드프레스 컨테이너와 MySQL 컨테이너의 연결을 위해 가상의 네트워크 필요
  - 네트워크 생성 명령

```
docker network create 네트워크이름
```

  - 네트워크 삭제 명령

```
docker network rm 네트워크이름
```

  - 기타 네트워크 관련 명령

주요 하위 커맨드

커맨드	내용	생략 가능 여부	주요 옵션
connect	네트워크에 컨테이너를 새로이 접속	X	거의 사용하지 않음
disconnect	네트워크에서 컨테이너의 접속을 끊음	X	거의 사용하지 않음
create	네트워크를 생성	X	거의 사용하지 않음
inspect	네트워크의 상세 정보를 확인	X	거의 사용하지 않음
ls	네트워크의 목록을 확인	X	거의 사용하지 않음
prune	현재 아무 컨테이너도 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

1.워드프레스 사이트 구축

• MySQL 컨테이너 생성 및 실행

```
docker run --name 컨테이너이름 -dit --net=네트워크이름
  -e MYSQL_ROOT_PASSWORD=MYSQL_루트_패스워드
  -e MYSQL_DATABASE=데이터베이스이름
  -e MYSQL_USER=MySQL사용자이름
  -e MYSQL_PASSWORD=MySQL_패스워드
  mysql
  --character-set-server=문자_인코딩
  --collation-server=정렬_순서
  --default-authentication-plugin=인증방식
```

옵션

항목	옵션	실습에서 사용하는 값
네트워크 이름	--net	wordpress000net1
MySQL 컨테이너 이름	--name	mysql000ex11
실행 옵션	-dit	없음
MySQL 루트 패스워드	-e MYSQL_ROOT_PASSWORD	myrootpass
MySQL 데이터베이스 이름	-e MYSQL_DATABASE	wordpress000db
MySQL 사용자 이름	-e MYSQL_USER	wordpress000kun
MySQL 패스워드	-e MYSQL_PASSWORD	wkunpass

인자

항목	인자	값	의미
문자 인코딩	--character-set-server=	utf8mb4	문자 인코딩으로 UTF8을 사용
정렬 순서	--collation-server=	utf8mb4_unicode_ci	정렬 순서로 UTF8을 따름
인증 방식	--default-authentication-plugin=	mysql_native_password	인증 방식을 예전 방식(native)으로 변경

MySQL8(8.0)으로 넘어오면서 외부에서 MySQL 접속의 인증 방식이 변경됨. 그러나 워드프레스에서는 아직 지원하지않으므로 예전 방식으로 설정

1. 워드프레스 사이트 구축

• 워드프레스 컨테이너 생성 및 실행

```
docker run --name 컨테이너이름 -dit --net=네트워크이름
  -p 포트_설정
  -e WORDPRESS_DB_HOST=데이터베이스_컨테이너_이름
  -e WORDPRESS_DB_NAME=데이터베이스_이름
  -e WORDPRESS_DB_USER=데이터베이스_사용자_이름
  -e WORDPRESS_DB_PASSWORD=데이터베이스_패스워드
  wordpress
```

옵션 항목

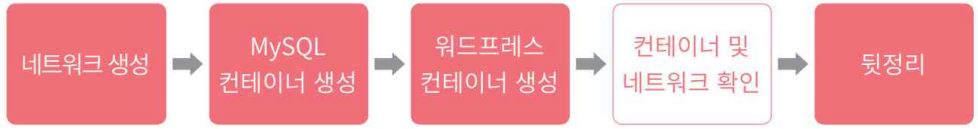
항목	옵션	설정값(임의의 이름 또는 지정된 값)
네트워크 이름	--net	wordpress000net1
워드프레스 컨테이너 이름	--name	wordpress000ex12
실행 옵션	-dit	(없음)
포트 번호 설정	-p	8085:80
데이터베이스 컨테이너 이름	-e WORDPRESS_DB_HOST	mysql000ex11
데이터베이스 이름	-e WORDPRESS_DB_NAME	wordpress000db
데이터베이스 사용자 이름	-e WORDPRESS_DB_USER	wordpress000kun
데이터베이스 패스워드	-e WORDPRESS_DB_PASSWORD	wkunpass

5장 여러 개의 컨테이너를 연동해 실행해보자

2절 워드프레스 및 MySQL 컨테이너 생성과 연동

워드프레스 & MySQL 컨테이너 생성과 연동 교재 145~148페이지

<실습>



- 생성할 네트워크 및 컨테이너 정보

항목	값
네트워크 이름	wordpress000net1
MySQL 컨테이너 이름	mysql000ex11
MySQL 이미지 이름	mysql
워드프레스 컨테이너 이름	wordpress000ex12
워드프레스 이미지 이름	wordpress

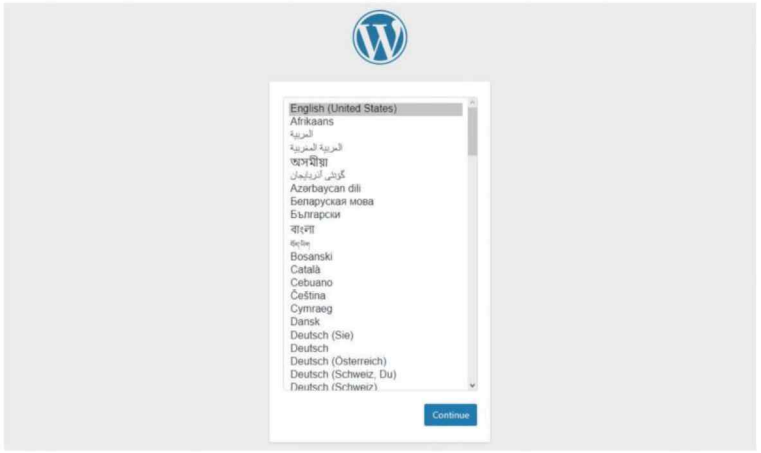


그림 5-2-1 워드프레스의 초기 화면

5장 여러 개의 컨테이너를 연동해 실행해보자

3절 명령어를 직접 작성해보자.

1.소프트웨어와 데이터베이스 관계

- LAMP = Linux + Apache + MySQL + PHP
- Apache -> Nginx, MySQL -> MariaDB/PostgreSQL등 다양한 조합이 가능해짐

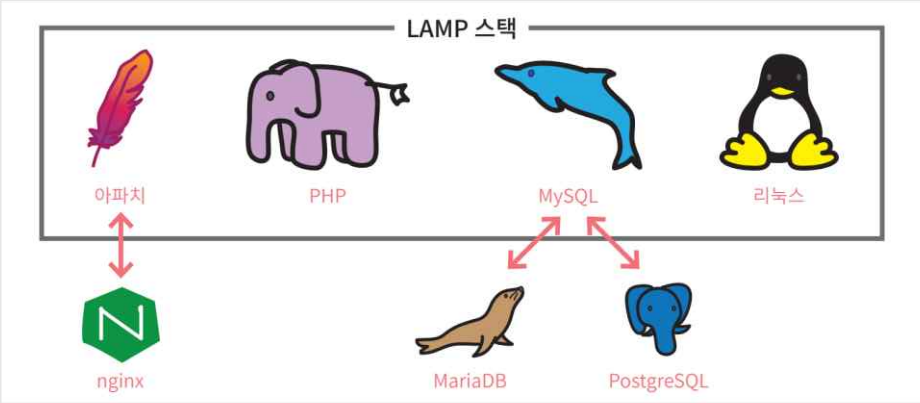


그림 5-3-1 LAMP 스택

run 명령어 직접 작성

- MySQL 컨테이너 정보

명령어에 사용할 옵션 설정값

항목	옵션	설정값
네트워크 이름	--net	wordpress000net1
MySQL 컨테이너 이름	--name	mysql000ex11
실행 옵션	-dit	(없음)
MySQL 루트 패스워드	-e MYSQL_ROOT_PASSWORD	myrootpass
MySQL 데이터베이스 이름	-e MYSQL_DATABASE	wordpress000db
MySQL 사용자 이름	-e MYSQL_USER	wordpress000kun
MySQL 패스워드	-e MYSQL_PASSWORD	wkunpass

```
docker run --name        -dit --net=
  -e MYSQL_ROOT_PASSWORD=
  -e MYSQL_DATABASE=
  -e MYSQL_USER=
  -e MYSQL_PASSWORD=
  mysql
  --character-set-server=utf8mb4
  --collation-server=utf8mb4_unicode_ci
  --default-authentication-plugin=mysql_native_password
```

run 명령어 직접 작성

- 워드프레스 컨테이너 정보

명령어에 사용할 옵션 설정값

항목	옵션	설정값
네트워크 이름	--net	wordpress000net1
워드프레스 컨테이너 이름	--name	wordpress000ex12
실행 옵션	-dit	(없음)
포트 번호 설정	-p	8085:80
데이터베이스 컨테이너 이름	-e WORDPRESS_DB_HOST	mysql000ex11
데이터베이스 이름	-e WORDPRESS_DB_NAME	wordpress000db
데이터베이스 사용자 이름	-e WORDPRESS_DB_USER	wordpress000kun
데이터베이스 패스워드	-e WORDPRESS_DB_PASSWORD	wkunpass

```
docker run --name      -dit --net=
  -p
  -e WORDPRESS_DB_HOST=
  -e WORDPRESS_DB_NAME=
  -e WORDPRESS_DB_USER=
  -e WORDPRESS_DB_PASSWORD=
  wordpress
```

직접 설정하고 컨테이너 구축해보기

MySQL 실행 시 사용할 옵션

항목	옵션	설정값(스스로 정해도 된다)
MySQL 컨테이너 이름		
실행 옵션		
MySQL 루트 패스워드		
MySQL 데이터베이스 이름		
MySQL 사용자 이름		
MySQL 패스워드		

항목	옵션	설정값 (스스로 정해도 된다)
워드프레스 컨테이너 이름		
실행 옵션		
포트 번호 설정		:80
데이터베이스 컨테이너 이름		
데이터베이스 이름		
데이터베이스 사용자 이름		
데이터베이스 패스워드		

5장 여러 개의 컨테이너를 연동해 실행해보자

4절 레드마인 및 MariaDB 컨테이너 생성 및 실행

레드마인과 MySQL 컨테이너

교재 154~156페이지

레드마인(Redmine)은 오픈 소스 프로그램으로 웹 기반의 프로젝트 관리와 버그 추적 기능을 제공하는 도구이다. 화면기반의 프로젝트 관리에 도움이 되도록 달력과 간트 차트를 제공하고 일정관리 기능을 제공한다. 또한 레드마인은 통합된 프로젝트관리 기능과 이슈추적, 여러 가지 형상 관리 기능을 제공한다.
<출처> https://ko.wikipedia.org/wiki/%EB%A0%88%EB%93%9C%EB%A7%88%EC%9D%B8

- 생성할 네트워크 및 컨테이너 정보

항목	설정값
네트워크 이름	redmine000net2
MySQL 컨테이너 이름	mysql000ex13
MySQL 이미지 이름	mysql
레드마인 컨테이너 이름	redmine000ex14
레드마인 이미지 이름	redmine

1) 네트워크 생성
2) mysql 컨테이너 생성

MySQL 컨테이너 생성 및 실행 시 옵션 항목

항목	옵션	설정값
MySQL 컨테이너 이름	--name	mysql000ex13
MySQL 루트 패스워드	-e MYSQL_ROOT_PASSWORD	myrootpass
MySQL 데이터베이스 이름	-e MYSQL_DATABASE	redmine000db
MySQL 사용자 이름	-e MYSQL_USER	redmine000kun
MySQL 패스워드	-e MYSQL_PASSWORD	rkunpass

레드마인과 MySQL 컨테이너

3) redmine 컨테이너 생성

레드마인 컨테이너 생성 및 실행 시 옵션 항목

항목	옵션	설정값
레드마인 컨테이너 이름	--name	redmine000ex14
실행 옵션	-dit	(없음)
포트 번호 설정	-p	8086:3000
데이터베이스 컨테이너 이름	-e REDMINE_DB_MYSQL	mysql000ex13
데이터베이스 이름	-e REDMINE_DB_DATABASE	redmine000db
데이터베이스 사용자 이름	-e REDMINE_DB_USERNAME	redmine000kun
데이터베이스 패스워드	-e REDMINE_DB_PASSWORD	rkunpass

4) 레드마인 동작 확인 (http://localhost:8086)

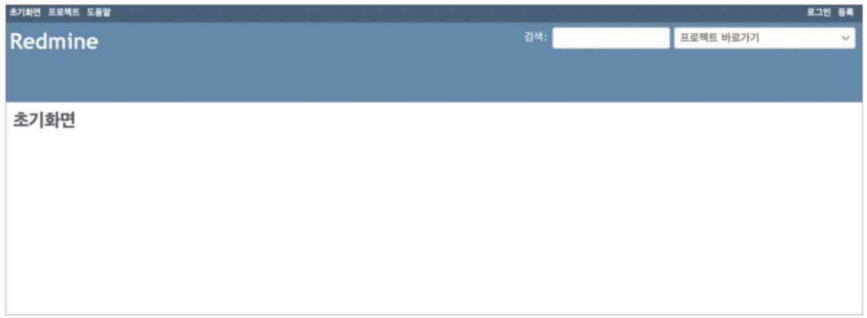


그림 5-4-1 레드마인 초기 화면

레드마인과 MariaDB 컨테이너

교재 157~159페이지

<실습>

• 생성할 네트워크 및 컨테이너 정보

항목	설정값
네트워크 이름	redmine000net3
MariaDB 컨테이너 이름	mariadb000ex15
MariaDB 이미지 이름	mariadb
레드마인 컨테이너 이름	redmine000ex16
레드마인 이미지 이름	redmine

MariaDB 컨테이너 생성 및 실행 시 옵션 항목

항목	옵션	설정값
MariaDB 컨테이너 이름	--name	mariadb000ex15
실행 옵션	-dit	(없음)
MariaDB 루트 패스워드	-e MYSQL_ROOT_PASSWORD	mariarootpass
MariaDB 데이터베이스 이름	-e MYSQL_DATABASE	redmine000db
MariaDB 사용자 이름	-e MYSQL_USER	redmine000kun
MariaDB 패스워드	-e MYSQL_PASSWORd	rkunpass

레드마인 컨테이너 생성 및 실행 시 옵션 항목

항목	옵션	설정값
레드마인 컨테이너 이름	--name	redmine000ex16
포트 번호 설정	-p	8087:3000
데이터베이스 컨테이너 이름	-e REDMINE_DB_MYSQL	mariadb000ex15

MariaDB 컨테이너 생성 및 실행 시 인자

항목	인자	설정값
문자 인코딩	--character-set-server=	utf8mb4
정렬 순서	--collation-server=	utf8mb4_unicode_ci
인증 방식	--default-authentication-plugin=	mysql_native_password

마무리

컨테이너 뒷정리

컨테이너 목록 확인	`docker ps -a`
컨테이너 종료	`docker stop 컨테이너_이름`
컨테이너 삭제	`docker rm 컨테이너_이름`

이미지 뒷정리

이미지 목록 확인	`docker image ls`
이미지 삭제	`docker image rm 이미지_이름`

네트워크 뒷정리

네트워크 목록 확인	`docker network ls`
네트워크 삭제	`docker network rm 네트워크_이름`

볼륨 뒷정리

볼륨 목록 확인	`docker volume ls`
볼륨 삭제	`docker volume rm 볼륨_이름`

6장 실전에 활용 가능한 컨테이너 사용법을 익히자

1절 실전에 필요한 지식

1.역할과 필요 지식

- 역할에 따라 알아야할 지식이 다름
  예를 들어 서버엔지니어나 보안 엔지니어라면 도커를 깊게 공부해야 함
- 학습할 내용
  - 6-2 컨테이너와 호스트 사이 파일 복사
    호스트와 컨테이너 사이의 파일 복사 방법으로 모든 사람에게 필요
  - 6-3 볼륨 마운트
    컨테이너와 파일 연동시 많이 사용(바인드 마운트), 모든 사람에게 필요
  - 6-4 컨테이너로 이미지 만들기 으로 파일 사용(볼륨 마운트), 서버 엔지니어, 프로그래머,
    컨테이너를 다른 컴퓨터로 복사하는 기능으로 서버관리자에게 필수
  - 6-5 컨테이너 개조
    사내 시스템을 컨테이너로 전환하는 업무 담당자 필수 요소, 전문성 필요
  - 6-6 도커허브로 로그인
    자신이 만든 컨테이너를 공개하기 위한 서비스
  - 7장 도커 컴포즈
    데이터베이스와 애플리케이션 컨테이너를 함께 시작해야하거나 환경을 대량으로 생성해야 할 때 유용
  - 8장 쿠버네티스
    여러 대의 서버에 켄테이너를 실행해야하는 경우 사용하는 컨테이너 오케스트레이션 도구

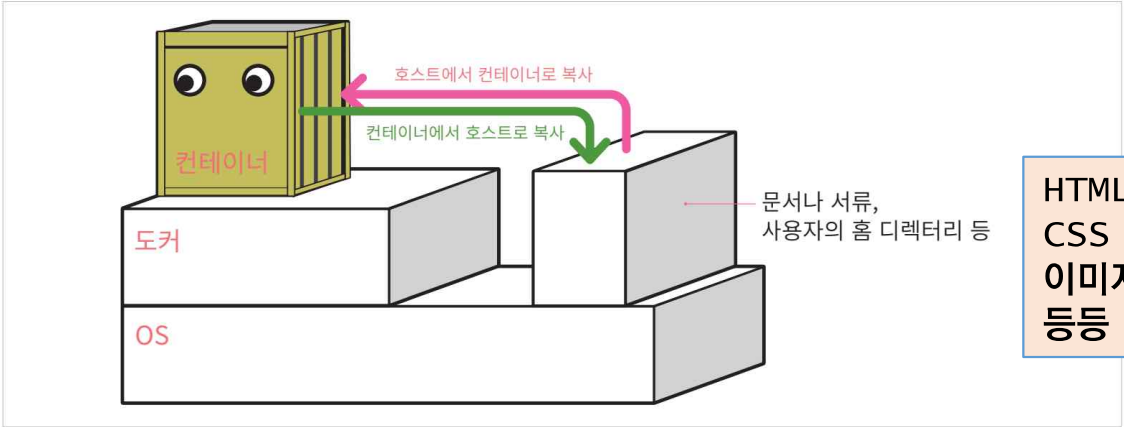
6장 실전에 활용 가능한 컨테이너 사용법을 익히자

2절 컨테이너와 호스트간 파일 복사

1.파일 복사

- 파일 복사

  컨테이너와 호스트, 호스트와 컨테이너 양방향 복사



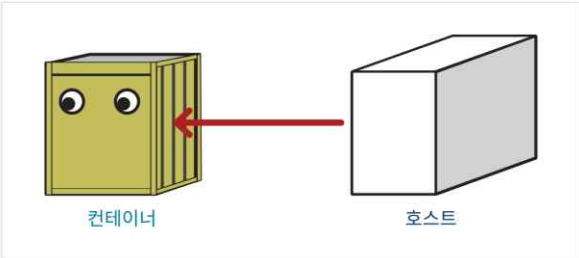
HTML 파일
CSS 파일
이미지나 텍스트 파일
등등

그림 6-2-1 호스트와 컨테이너 간에 파일 복사

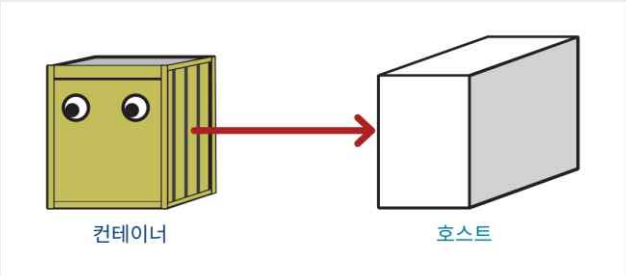
1.파일 복사

- 파일 복사 명령(docker cp, docker container cp)

```
docker cp 호스트_경로 컨테이너_이름:컨테이너_경로
```




```
docker cp 컨테이너_이름:컨테이너_경로 호스트_경로
```



1.파일 복사

• 호스트 경로의 예

경로의 예

항목	값
문서 폴더(윈도우)	C:\Users\사용자명\Documents\파일명
문서 폴더(macOS)	/Users/사용자명/Documents/파일명
리눅스의 홈 디렉터리	/home/사용자명/파일명

• index.html 파일 생성

```
<html>
<meta charset="utf-8"/>
<body>
<div>안녕하세요!</div>
</body>
</html>
```

1.파일 복사

교재 173~175페이지

<실습> 호스트 파일을 컨테이너로 복사

- 생성할 컨테이너 정보

항목	값
컨테이너 이름	apa000ex19
이미지 이름	httpd
포트 설정	8089:80

- 원본 파일 정보

항목	값
윈도우 원본 경로	C:\Users\사용자명\Documents\index.html[5]
macOS 원본 경로	/Users/사용자명/Documents/index.html
리눅스 원본 경로	/home/사용자명/index.html
컨테이너 경로	/usr/local/apache2/htdocs

1.파일 복사

- 아파치 컨테이너 생성

```
docker run --name apa000ex19  -d -p 8089:80 httpd
```

http://localhost:8089로 접속

그림 6-2-2 아파치의 초기 화면

- index.html 복사

```
docker cp index.html apa000ex19:/usr/local/apache2/htdocs/
```

http://localhost:8089로 접속

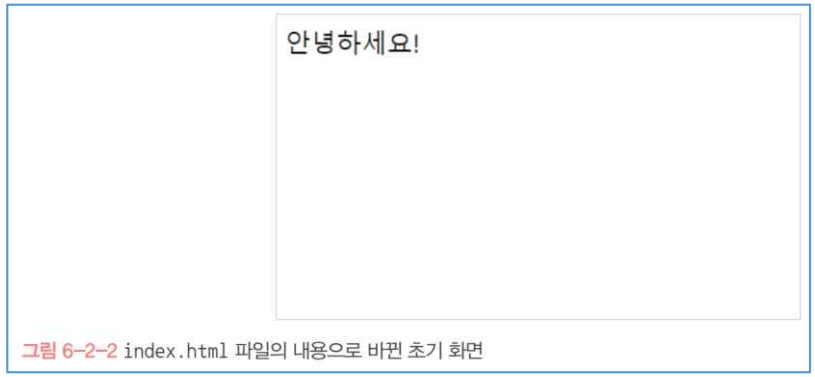


그림 6-2-2 index.html 파일의 내용으로 바뀐 초기 화면

1.파일 복사

교재 176~178페이지

<실습> 컨테이너 파일을 호스트로 복사

- 생성할 컨테이너 정보
  이전 컨테이너 활용
- 파일 복사

항목	값
윈도우 대상 경로	C:\Users\사용자명\Documents\
macOS 대상 경로	/Users/사용자명/Documents/
리눅스 대상 경로	/home/사용자명/
컨테이너 경로	/usr/local/apache2/htdocs/index.html

자신의 운영체제에 맞춰 골라 사용할 것

6장 실전에 활용 가능한 컨테이너 사용법을 익히자

3절 볼륨마운트

1. 볼륨과 마운트

- 볼륨 : 스토리지의 한 영역을 분할한 것

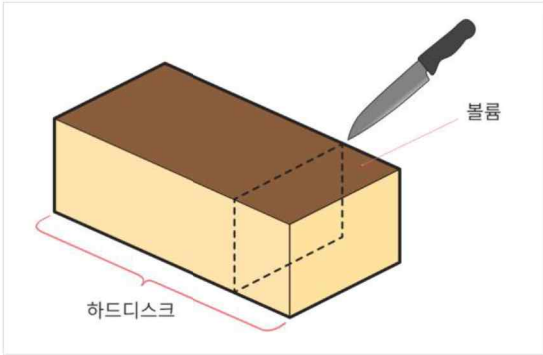


그림 6-3-1 볼륨

- 마운트 : '연결하다'의 의미

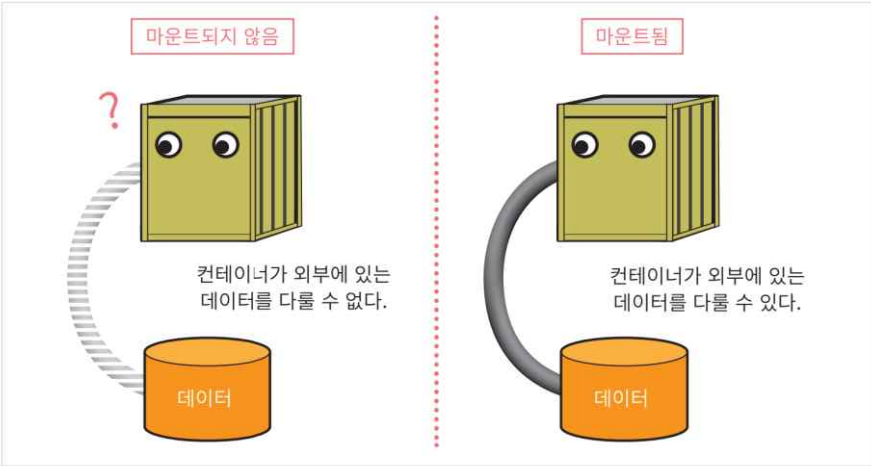


그림 6-3-2 마운트를 하면 이렇게 된다.

컨테이너는 소프트웨어 업그레이드 등을 이유로 생성과 삭제를 자주 수행하나 데이터는 외부에 저장해두어야 함
<Data Persistency>

2.스토리지 마운트 종류

- 마운트 종류
  - 볼륨 마운트
  - 바인드 마운트

- 볼륨 마운트
  - 도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너에 디스크 형태로 마운트하여 사용
  - 직접 조작이 어려우므로 주로 '임시 목적' 또는 '자주 쓰지는 않지만 지우면 안되는 파일을 저장'하는 목적으로 사용

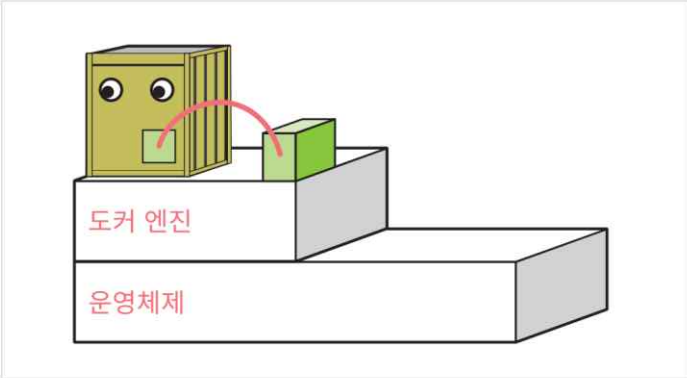


그림 6-3-3 볼륨 마운트

2.스토리지 마운트 종류

- 바인드 마운트
  - 도커 엔진에서 관리하지 않는 영역의 디렉터리를 컨테이너에 마운트하는 방식
  - 해당 디렉터리를 직접 조회할 수 있으므로 자주 사용하는 파일을 보관하는 용도로 사용

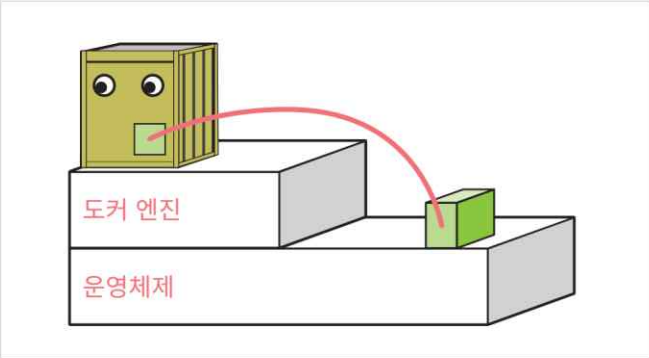


그림 6-3-4 바인드 마운트

2.스토리지 마운트 종류

- 2가지 방식의 차이점

  복잡성
  호스트에서 파일을 다룰지 여부
  환경의 의존성을 배제해야 하는지 여부

1) 볼륨 마운트
   - 도커 엔진에 의해 관리
     - 리눅스 경우 /var/lib/docker/volumns
     - 도커 데스크톱의 경우 도커엔진의 관리영역
   - 운영체제와 무관하게 일관성있는 방식으로 작업
   - 도커 제작사에서 추천하는 방식
   - 도커 컨테이너를 이용하지않고 접근 불가능
   - 호스트에서 직접 제어하면 볼륨이 깨질 수 있음
   - 백업 절차 복잡

2.스토리지 마운트 종류

2) 바인드 마운트

- 도커가 관리하지않는 영역을 이용
- 기존에 사용하는 방식으로 파일 사용 가능
- 도커 엔진과 무관하게 파일 제어
- 파일 자주 편집해야하는 경우 좋음

볼륨 마운트와 바인드 마운트의 차이

항목	볼륨 마운트	바인드 마운트
스토리지 영역	볼륨	디렉터리 또는 파일
물리적 위치	도커 엔진의 관리 영역	어디든지 가능
마운트 절차	볼륨을 생성한 후 마운트	기존 파일 또는 폴더를 마운트
내용 편집	도커 컨테이너를 통해서	일반적인 파일과 같이
백업	절차가 복잡함	일반적인 파일과 같이

3.스토리지 영역을 마운트하는 명령

- docker run 명령의 옵션 형태로 마운트

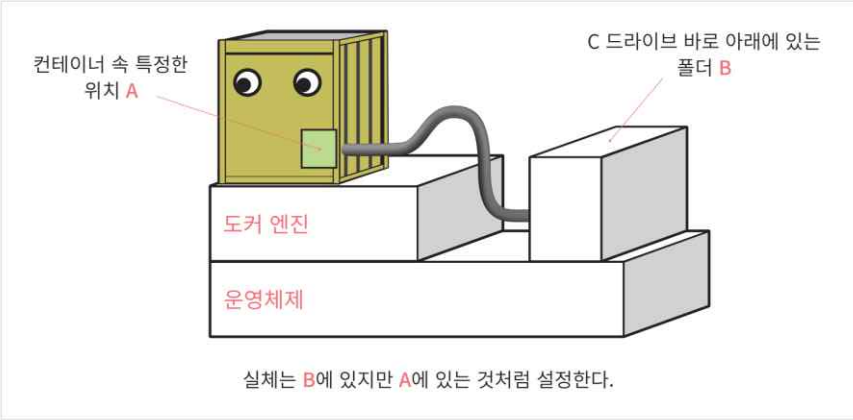


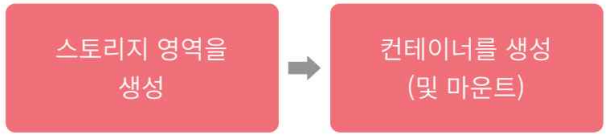
그림 6-3-5 마운트는 단축 아이콘과 같다(바인드 마운트)

<예>
폴더 B
마운트 원본 디렉터리 : C:\USERS\사용자명\Desktop\test

위치 A
마운트 대상 : /usr/local/apache2/htdocs (아파치)
/var/lib/mysql (MySQL)

3.스토리지 영역을 마운트하는 명령

- 스토리지를 마운트하는 절차



- 스토리지 영역을 생성하는 방법
  - 바인드 마운트 : 원본 파일 또는 디렉터리 먼저 생성
  - 볼륨 마운트 : 볼륨 먼저 생성

```
docker volume create 볼륨_이름

Docker volume rm 볼륨_이름
```

주요 하위 커맨드

커맨드	내용	생략형	주요 옵션
create	볼륨을 생성	X	거의 사용하지 않음
inspect	볼륨의 상세 정보를 출력	X	거의 사용하지 않음
ls	볼륨의 목록을 출력	X	거의 사용하지 않음
prune	현재 마운트되지 않은 볼륨을 모두 삭제	X	거의 사용하지 않음
rm	지정한 볼륨을 삭제	X	거의 사용하지 않음

3.스토리지 영역을 마운트하는 명령

- 스토리지 마운트 명령
  - 바인드 마운트

```
docker run . . . -v 스토리지_실제_경로:컨테이너_마운트_경로 . . .
```

  - 볼륨마운트

```
docker run . . . -v 볼륨_이름:컨테이너_마운트_경로 . . .
```

3.스토리지 영역을 마운트하는 명령

교재 186~189페이지

<실습> 바인드 마운트

• 생성할 컨테이너

항목	값
컨테이너 이름	apa00ex20
이미지 이름	httpd
포트 번호 설정	8090

• 마운트할 원본 디렉터리와 마운트 대상

항목	값
컨테이너 마운트 경로 (마운트 대상)	/usr/local/apache2/htdocs
실제 폴더 / 디렉터리 이름 (마운트 원본)	apa_folder
실제 마운트 원본 경로(윈도우)	C:\Users\사용자명\Documents\apa_folder
실제 마운트 원본 경로(macOS)	/Users/사용자명/Documents/apa_folder
실제 마운트 원본 경로(리눅스)	/home/사용자명/apa_folder

자신의 운영체제에 맞는 명령어를 선택

3.스토리지 영역을 마운트하는 명령

교재 190~192페이지

<실습> 볼륨 마운트

• 생성할 컨테이너 정보

항목	값
컨테이너 이름	apa000ex21
이미지 이름	httpd
볼륨 이름	apa000vol1(마지막은 소문자 L 뒤에 숫자 1이다)
포트 번호 설정	8091

• 마운트할 볼륨과 대상

마운트 원본과 마운트 대상 정보

항목	값
컨테이너 마운트 경로	/usr/local/apache2/htdocs
마운트 원본 볼륨	apa000vol1

<COLUMN> 볼륨 마운트에 대하여 더 알아보기

• 볼륨 마운트 확인하는 방법

- 리눅스 컨테이너를 이용하여 확인하는 방법

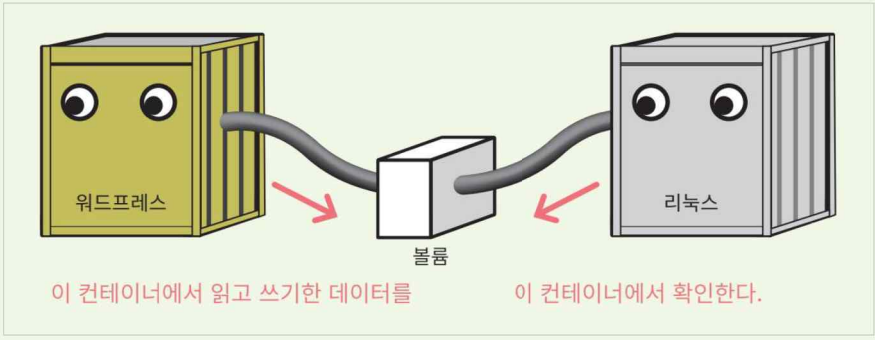


그림 6-3-8 별도의 컨테이너에서 볼륨의 내용 확인하기

- 컨테이너를 삭제하고 새 컨테이너에 볼륨을 마운트하여 확인하는 방법

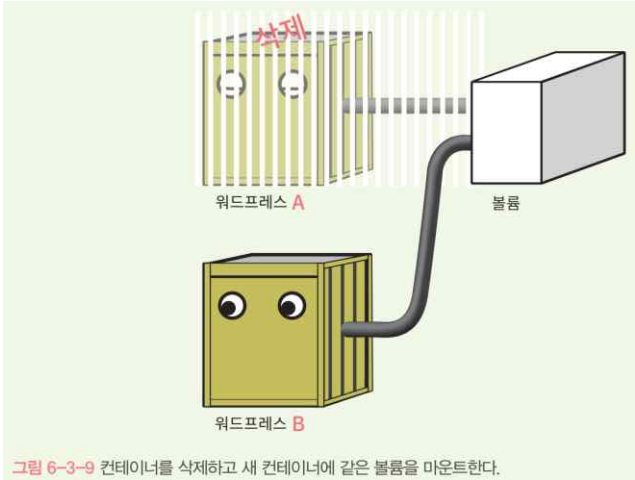


그림 6-3-9 컨테이너를 삭제하고 새 컨테이너에 같은 볼륨을 마운트한다.

<COLUMN> 볼륨 마운트에 대하여 더 알아보기

• 볼륨 백업

주 컨테이너의 종료를 확인 ➡ 리눅스 운영체제 컨테이너를 만들고 tar 명령어를 사용해 백업을 수행

• 별도의 리눅스 컨테이너를 만들어 볼륨 내용 압축 및 저장

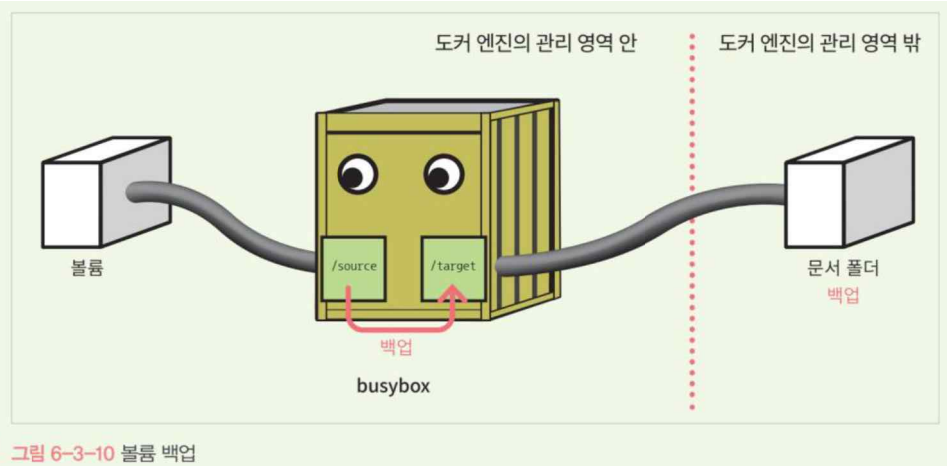


그림 6-3-10 볼륨 백업

```
docker run --rm 볼륨명:/source -v 백업_저장_폴더명:/target
busybox tar cvzf /target/백업파일이름.tar.gz -C /source .
```

실행 후 즉시 삭제

<COLUMN> 볼륨 마운트에 대하여 더 알아보기

- 볼륨 백업 정보

항목	값
볼륨 이름	apa000vol1
백업 저장 폴더 이름	C:\Users\사용자명\Documents
busybox 컨테이너의 백업 원본 디렉터리 이름	/source
busybox 컨테이너의 백업 저장 디렉터리 이름	/target
백업파일명	backup_apa

- 백업 명령

```
docker run --rm apa000vol1:/source -v
C:\Users\사용자명\Documents:/target busybox tar cvzf
/target/backup_apa.tar.gz -C /source .
```

- 복원 명령

```
docker run --rm apa000vol2:/source -v
C:\Users\사용자명\Documents:/target busybox tar xvzf
/target/backup_apa.tar.gz -C /source .
```

6장 실전에 활용 가능한 컨테이너 사용법을 익히자

4절 컨테이너로 이미지 만들기

1.컨테이너로 이미지 만드는 방법

- 나만의 이미지로 동일한 구성의 컨테이너를 대량으로 생성할 수 있음

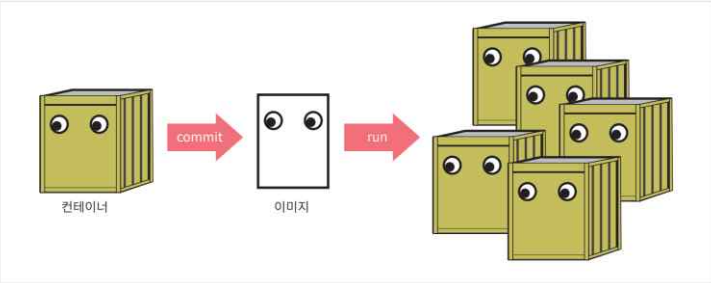


그림 6-4-1 똑같은 컨테이너를 대량으로 만들기

- 다른 서버로 쉽게 이동할 수 있음

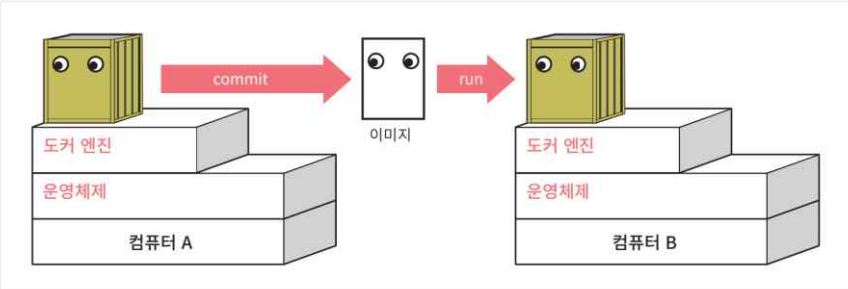


그림 6-4-2 컨테이너를 이동할 수도 있다.

1.컨테이너로 이미지 만드는 방법

- 이미지를 만드는 방법
  - commit 명령으로 만들기
  - Dockerfile 스크립트로 만들기

1) commit 명령으로 컨테이너를 이미지로 변환하기

```
docker commit 컨테이너_이름 새로운_이미지_이름
```

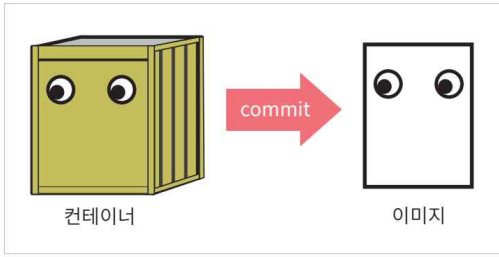


그림 6-4-3 컨테이너를 이미지로 변환

1.컨테이너로 이미지 만드는 방법

2) Dockerfile 스크립트로 이미지 생성하기

Dockerfile에 컨테이너에 넣을 이미지나 실행할 명령 등 기술

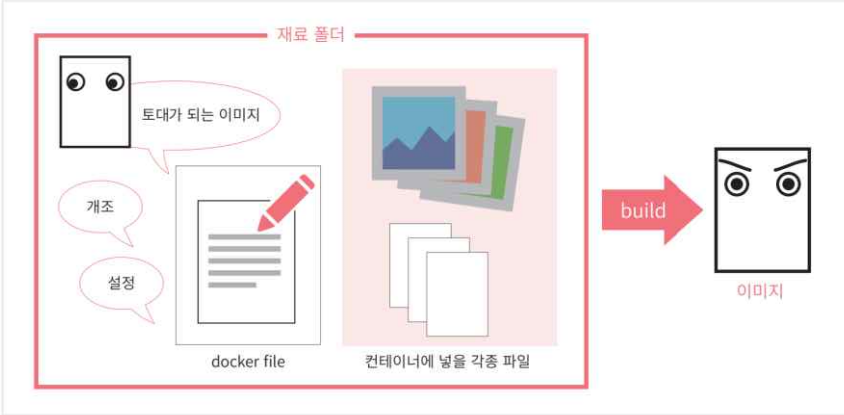


그림 6-4-4 Dockerfile 스크립트로 이미지 만들기

<Dockerfile 스크립트의 예>

```
FROM 이미지_이름
COPY 원본_경로 대상_경로
RUN 리눅스_명령어
. . .
```

```
docker build -t 생성할_이미지_이름 재료_폴더_경로
```

1.컨테이너로 이미지 만드는 방법

• Dockerfile 스크립트에 사용되는 주요 명령

주요 Dockerfile 인스트럭션

인스트럭션	내용
FROM	토대가 되는 이미지를 지정
ADD	이미지에 파일이나 폴더를 추가
COPY	이미지에 파일이나 폴더를 추가
RUN	이미지를 빌드할 때 실행할 명령어를 지정
CMD	컨테이너를 실행할 때 실행할 명령어를 지정
ENTRYPOINT	컨테이너를 실행할 때 실행할 명령어를 강제 지정
ONBUILD	이 이미지를 기반으로 다른 이미지를 빌드할 때 실행할 명령어를 지정
EXPOSE	이미지가 통신에 사용할 포트를 명시적으로 지정
VOLUME	퍼시스턴시 데이터를 저장할 경로를 명시적으로 지정
ENV	환경변수를 정의
WORKDIR	RUN, CMD, ENTRYPOINT, ADD, COPY에 정의된 명령어를 실행하는 작업 디렉터리를 지정
SHELL	빌드 시 사용할 셸을 변경
LABEL	이름이나 버전, 저작자 정보를 설정
USER	RUN, CMD, ENTRYPOINT에 정의된 명령어를 실행하는 사용자 또는 그룹을 지정
ARG	docker build 커맨드를 사용할 때 입력받을 수 있는 인자를 선언
STOPSIGNAL	docker stop 커맨드를 사용할 때 컨테이너 안에서 실행 중인 프로그램에 전달되는 시그널을 변경
HEALTHCHECK	컨테이너 헬스체크 방법을 커스터마이징

1.컨테이너로 이미지 만드는 방법

교재 201~202페이지

<실습> commit 명령으로 컨테이를 이미지로 변환

- **생성할 이미지**

항목	값
컨테이너 이름	apa000ex22
새로 만들 이미지 이름	ex22_original1 (끝부분은 소문자 L과 숫자 1이다)

1.컨테이너로 이미지 만드는 방법

교재 202~204페이지

<실습> Dockerfile 스크립트로 이미지 만들기

• 새로 만들 이미지 정보

항목	값
기반 이미지 이름	httpd
새로 만들 이미지 이름	ex22_original2
재료 폴더의 경로(윈도우)	C:\Users\사용자명\Documents\apa_folder
재료 폴더의 경로(macOS)	/Users/사용자명/apa_folder
재료 폴더의 경로(리눅스)	/home/사용자명/apa_folder

자신이 사용하는 운영체제에 맞춰 명령어를 작성할 것

• Dockerfile 스크립트 내용

Dockerfile 스크립트의 내용

인스트럭션	값
FROM	httpd
COPY	index.html /usr/local/apache2/htdocs

1.컨테이너로 이미지 만드는 방법

- <실습2> 파이썬 기반 flask 웹 프로그램(app.py)

```
from flask import Flask

app = Flask(__name__)
@app.route("/")
def hello():
    return "<h1>Hello World!</h1>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
```

Dockerfile 작성

```
FROM python
RUN pip3 install flask
WORKDIR /root
ADD app.py .

CMD ["python3", "app.py"]
```

<COLUMN> 이미지 옮기는 방법

- 이미지를 호스트 컴퓨터의 파일시스템에 tar 형식으로 저장

```
docker save -o 파일_이름.tar 이미지_이름
```

- 도커 엔진으로 가져올 때는 load 명령 이용

```
docker load -i 파일_이름.tar
```

<Container export/import>
- 컨테이너를 파일로 저장
  docker export [컨테이너명 or 컨테이너ID] > 파일_이름.tar

- 저장한 컨테이너를 도커 이미지로 읽어 들임
  docker import [파일_이름.tar or URL] - [image name[:tag name]]

6장 실전에 활용 가능한 컨테이너 사용법을 익히자

5절 컨테이너 개조

1. 컨테이너 개조

- 컨테이너 개조 방법(두 가지 방법 혼용)
  - 파일 복사와 마운트
  - 컨테이너에서 리눅스 명령 이용

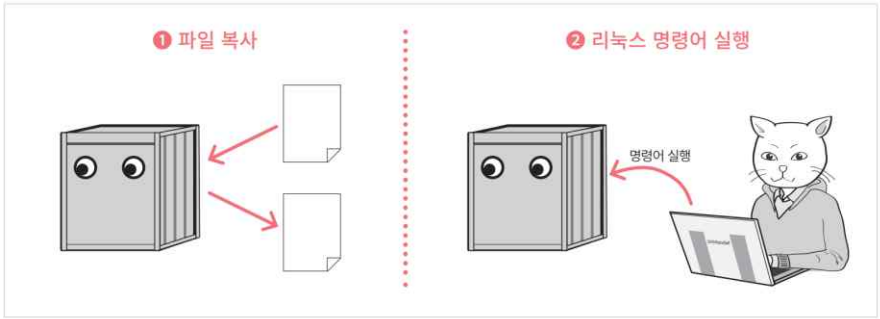


그림 6-5-1 컨테이너를 개조하는 두 가지 방법

- 컨테이너에서 리눅스 명령 실행
  - 리눅스 명령 실행을 위해 bash 쉘 필요 (/bin/bash)
  - docker run 또는 **docker exec** 명령을 이용하여 실행

    컨테이너 속에서 명령을 실행하는 명령
    실행 중인 컨테이너에 사용

  - bash를 이용하여 변경이 완료되면 docker start 명령으로 재시작 필요

1.컨테이너 개조

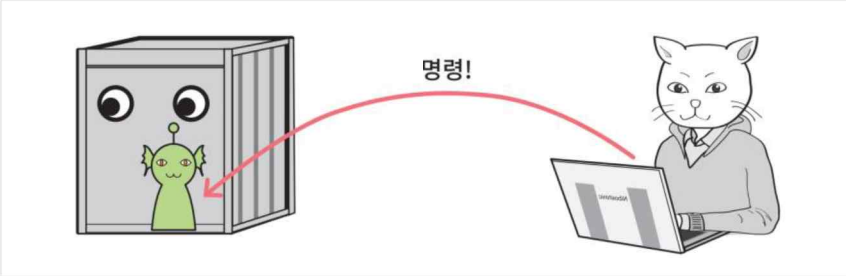


그림 6-5-2 컨테이너에 명령을 전달하는 셸

- docker exec 명령

```
docker exec [옵션] 컨테이너_이름 /bin/bash
```

- docker run 명령

```
docker run [옵션] 이미지_이름 /bin/bash
```

```
docker exec 명령 예)
  docker exec -it apa000ex23 /bin/bash


docker run 명령 예)
  docker run --name apa000ex23 -it -p 8089:80 httpd /bin/bash
```

1. 컨테이너 개조

- bash를 통해 컨테이너를 조작하는 동안에는 도커 명령을 사용할 수 없다.

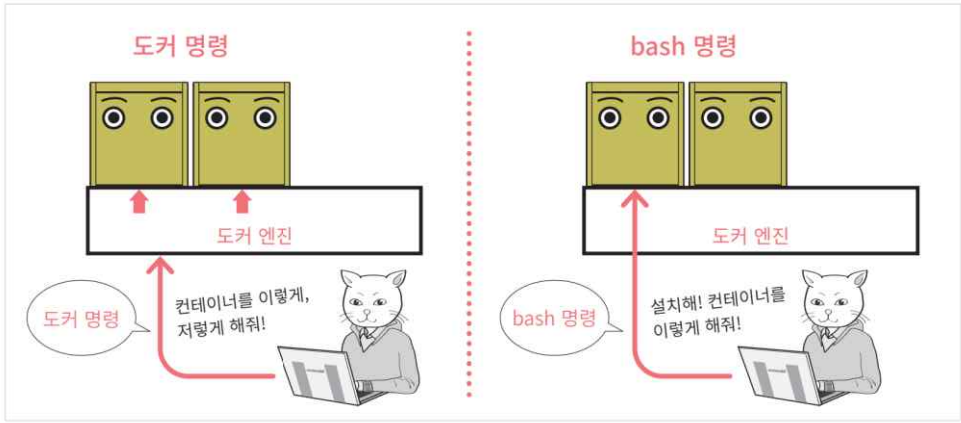


그림 6-5-3 도커 명령과 bash 명령의 차이

- 컨테이너 조작이 끝나면 'exit' 명령으로 컨테이너를 빠져나와야 함

<우분투 리눅스 컨테이너의 명령 예>

"Hello, World" 메시지 출력

```
echo "Hello, World"
```

아파치 설치

```
apt install apache2
```

MySQL 설치

```
apt install mysql-server
```

2.도커 엔진 명령과 컨테이너 내부 명령

교재 210페이지

• 도커 엔진 명령과 컨테이너 내부에서 실행하는 명령

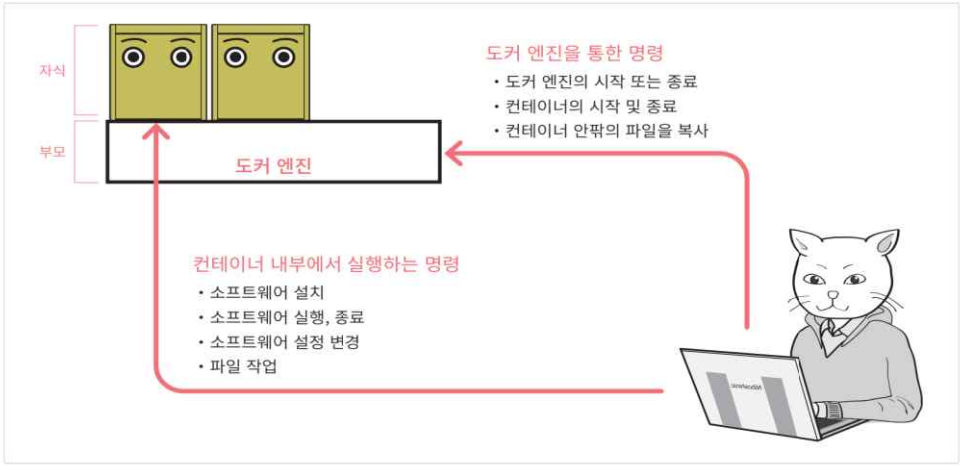


그림 6-5-4 도커 엔진을 통한 명령과 컨테이너 내부에서 실행하는 명령의 차이

• 도커와 컨테이너는 별개의 명령 사용

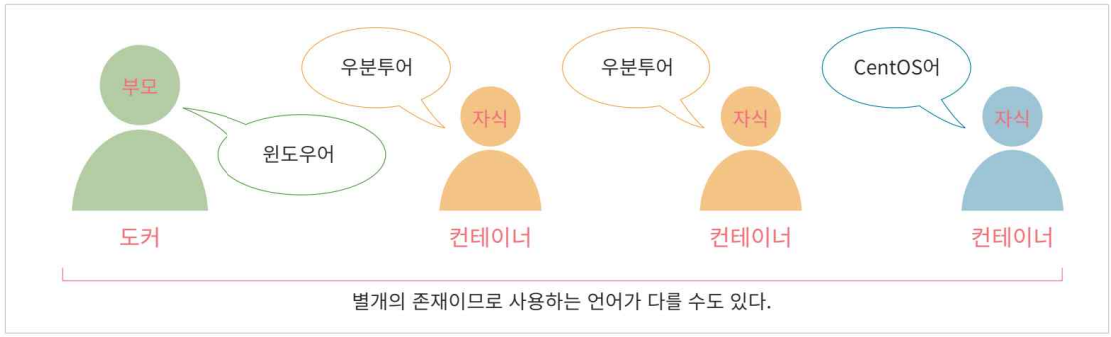


그림 6-5-5 도커와 컨테이너는 별개의 언어를 사용한다.

2.도커 엔진 명령과 컨테이너 내부 명령

- 컨테이너의 종류에 따라 내부 명령에 차이가 있음
  - 리눅스 배포본에 따라 일부 명령에 차이가 있음(패키지 관리 도구 등)

그림 6-5-7 운영체제에 따라 컨테이너 내부의 명령 스타일이 달라진다.

  - 대부분의 컨테이너는 데비안 계열을 이용

6장 실전에 활용 가능한 컨테이너 사용법을 익히자

6절 도커 허브 등록 및 로그인

1.이미지 다운로드

- 이미지 저장소 : 도커 허브 (dockerhub)

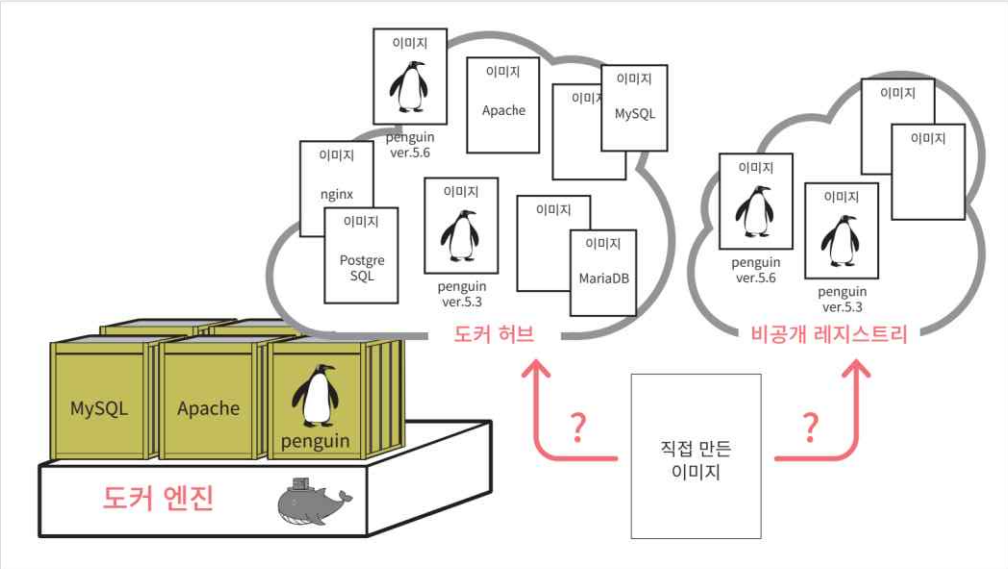


그림 6-6-1 직접 만든 이미지는 어디에 올려야 할까?

- 직접 만든 이미지를 도커 허브에 업로드할 수 있고, 비공개로 도커 허브와 같은 장소(레지스트리)를 만들 수도 있음

1.이미지 다운로드

- 도커 허브와 도커 레지스트리
  - 도커 레지스트리 : 이미지 배포하는 장소
  - 도커 허브 : 도커 제작사에서 운영하는 공식 도커 레지스트리

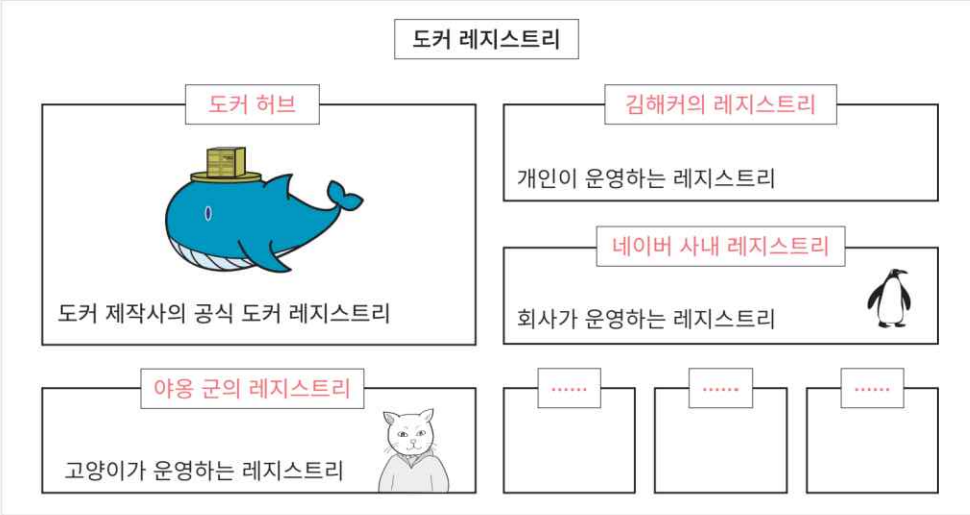


그림 6-6-2 도커 레지스트리

1.이미지 다운로드

- 레지스트리와 리포지터리
  - 레지스터리 : 이미지를 배포하는 장소
  - 리포지터리 : 레지스트리를 구성하는 단위

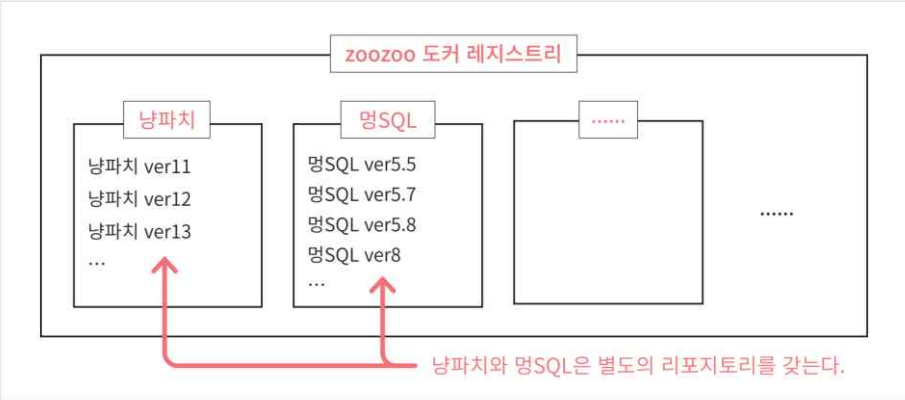


그림 6-6-3 각각의 리포지토리

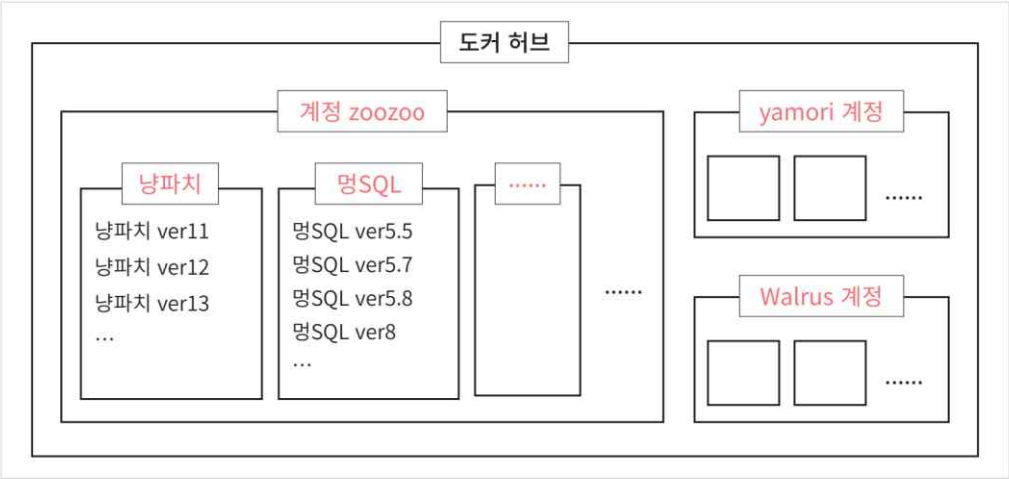


그림 6-6-4 도커 허브는 레지스트리가 여럿 모인 형태다.

2.태그와 이미지 업로드

- 이미지 이름과 태그
- 태그 : 레지스트리 주소와 버전 표기

<형식>
'레지스트리_주소(도커허브는 ID)/리포지터리_이름:버전'
버전은 생략 가능

그림 6-6-5 태그를 붙이는 방법

비공개 레지스트리의 태그 명명 예

비공개 레지스트리의 태그	레지스트리_주소/리포지토리_이름:버전
자신의 PC에 만든 레지스트리, 리포지토리 이름은 nyapacchi, ver13	localhost:5000/nyapacchi:13
레지스트리 도메인은 zoozoo.coomm, 리포지토리 이름은 nyapacchi, ver13	zoozoo.coomm/nyapacchi:13

도커 허브의 태그 명명 예

도커 허브의 태그	도커_허브_ID/리포지토리_이름:버전
도커 허브 ID는 zoozoousagi, 리포지토리 이름은 nyapacchi, ver13	zoozoousagi/nyappacchi:13

2.태그와 이미지 업로드

- 이미지에 태그를 부여하여 복제하는 명령

```
docker tag 원본_이미지_이름 레지스트리_주소/리포지터리_이름:버전
```

```
<예>
docker tag apa000ex22 zoozoo.coomm/nyapacchi:13
```

- 이미지를 업로드하는 명령

```
docker push 레지스트리_주소/리포지터리_이름:버전
```

```
<예>
docker push zoozoo.coomm/nyapacchi:13
```

3. 레지스트리 만드는 방법

- 비공개 레지스트리 만드는 방법
  - 레지스트리용 컨테이너, 즉 레지스트리용 도커를 이용하여 운영
  - 포트번호 5000번 사용
  - 레지스트리 생성 명령

```
docker run -d -p 5000:5000 registry
```

- 도커 허브 사용
  - email 주소로 쉽게 가입
  - 주소 : https://hub.docker.com

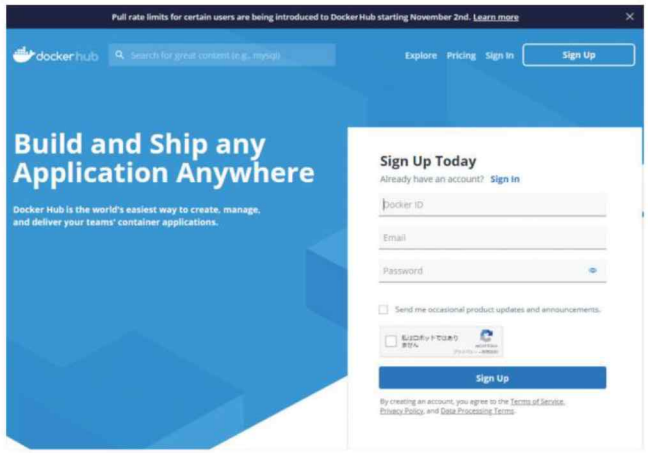


그림 6-6-6 도커 허브의 초기 화면

3. 레지스트리 만드는 방법

- 전 세계에 이미지를 공개할 수도 있고, 비공개 설정도 할 수 있음
- push 명령으로 리포지터리를 생성하면 자동으로 공개 상태(public)
- Create Repository 버튼을 클릭하여 리포지터리 생성 가능

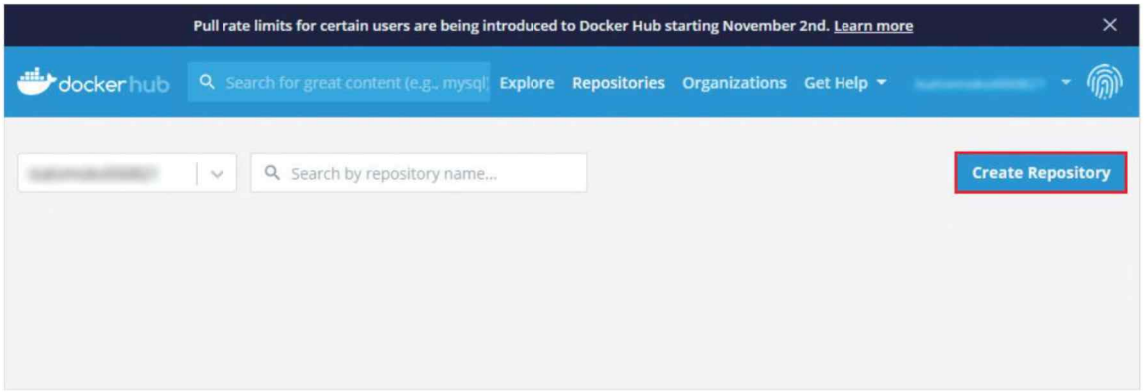


그림 6-6-7 [Create Repository] 버튼을 클릭해 리포지토리를 생성한다.

7장 도커 컴포즈를 익히자.

1절 도커 컴포즈란?

1.도커 컴포즈

- 시스템 관련 명령들을 하나의 파일에 작성하여 한번에 시스템 전체를 실행하고 종료 및 폐기까지 실행하도록 도와주는 도구

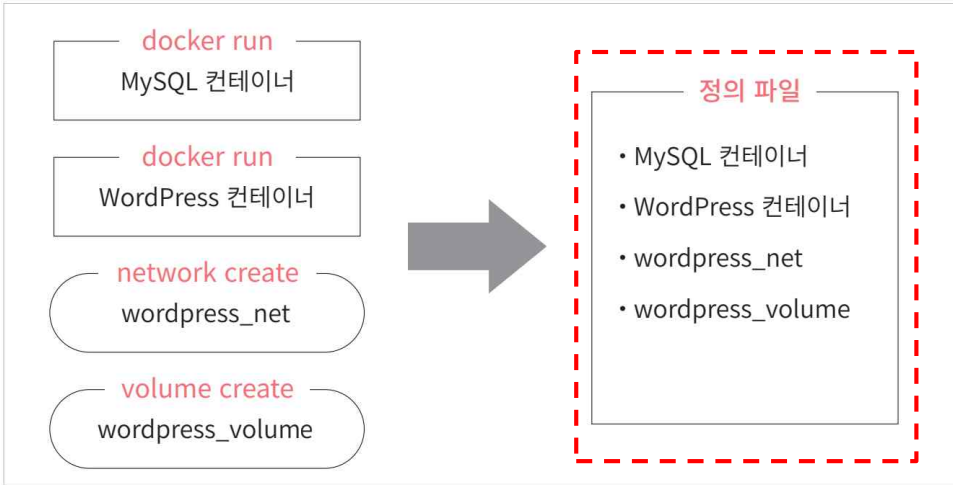


그림 7-1-1 도커 컴포즈를 사용하면 여러 개의 명령어를 하나의 정의 파일로 합쳐 실행할 수 있다.

1.도커 컴포즈

• 도커 컴포즈 구조

- YAML(YAML Ain't a Markup Language)포맷의 정의 파일

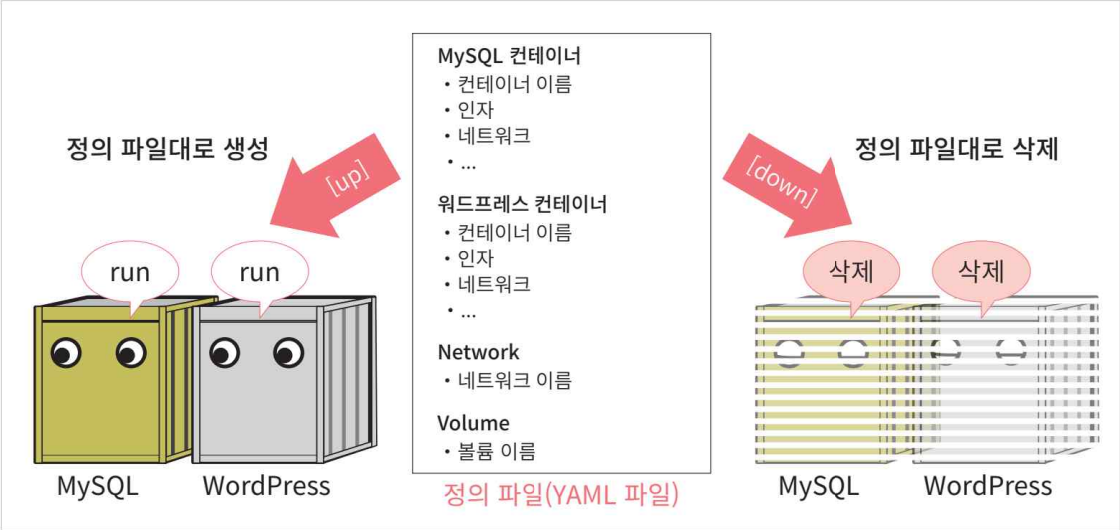


그림 7-1-2 도커 컴포즈는 시스템의 모든 정보를 정의 파일에 기재한다.

1.도커 컴포즈

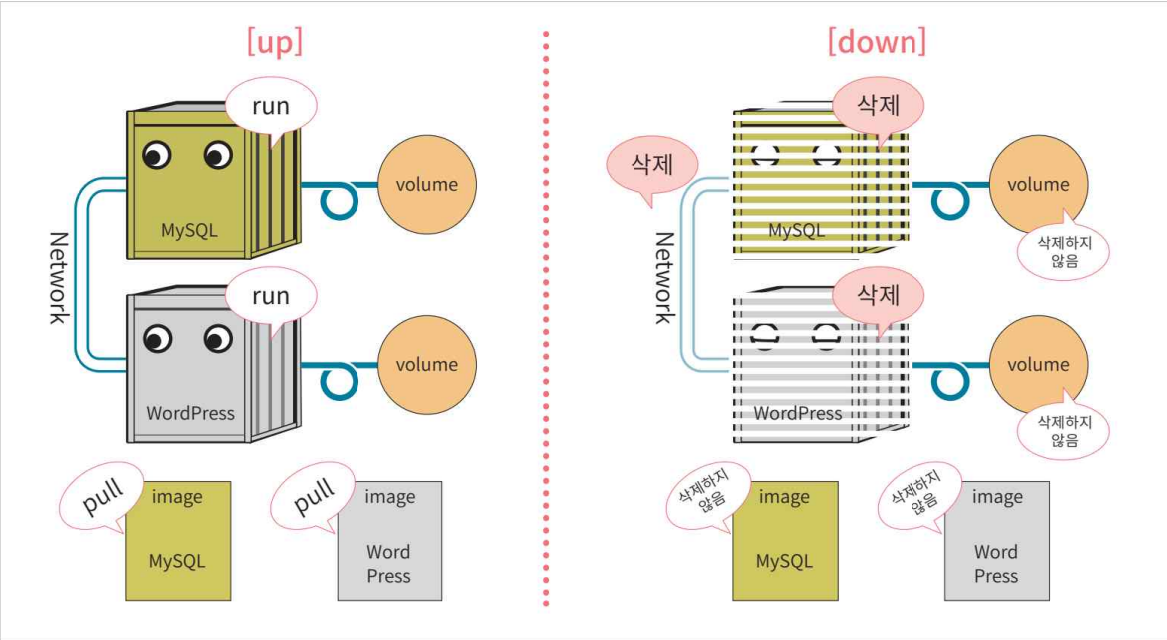


그림 7-1-3 up 커맨드로 생성 및 시작, down 커맨드로 정지 및 삭제한다.

-up 명령 : 정의 파일의 내용대로 이미지를 내려 받고 컨테이너를 생성 및 실행, 네트워크나 볼륨도 정의 가능
-down 명령 : 컨테이너와 네트워크를 정지 및 삭제
-Stop 명령 : 컨테이너와 네트워크를 삭제없이 종료만 진행

1.도커 컴포즈

- 도커 컴포즈와 Dockerfile 스크립트 차이점
  - 도커 컴포즈 : 컨테이너와 주변 환경 및 네트워크와 볼륨까지 만들 수 있다.
  - Dockerfile 스크립트 : 이미지를 만들기 위한 것으로 네트워크나 볼륨은 만들 수 없다

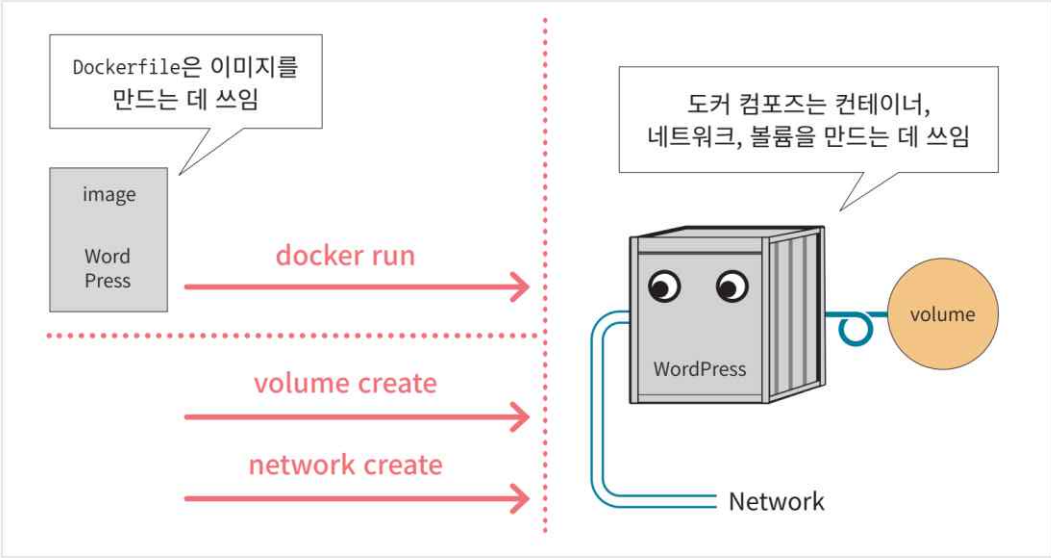


그림 7-1-4 Dockerfile 스크립트와 도커 컴포즈의 차이점

<COLUMN> 도커 컴포즈와 쿠버네티스의 차이점
쿠버네티스 : 도커 컨테이너 관리 도구
도커 컴포즈 : 컨테이너 생성 및 삭제, 관리 기능은 없음

7장 도커 컴포즈를 익히자.

2절 도커 컴포즈 설치와 사용법

1.도커 컴포즈 설치

- <실습> 도커 컴포즈 설치
- 도커 컴포즈
  - 도커 엔진과 별개의 소프트웨어, 사용법은 유사
  - 파이썬으로 작성된 프로그램

  - 도커 데스크톱에는 도커 컴포즈 함께 설치되어 있음
  - 우분투 리눅스에서 도커 컴포즈 설치

```
sudo apt install -y python3 python3-pip
sudo pip3 install docker-compose
```

  - 확인

```
docker-compose --version
```

1.도커 컴포즈 설치

• 도커 컴포즈 사용법

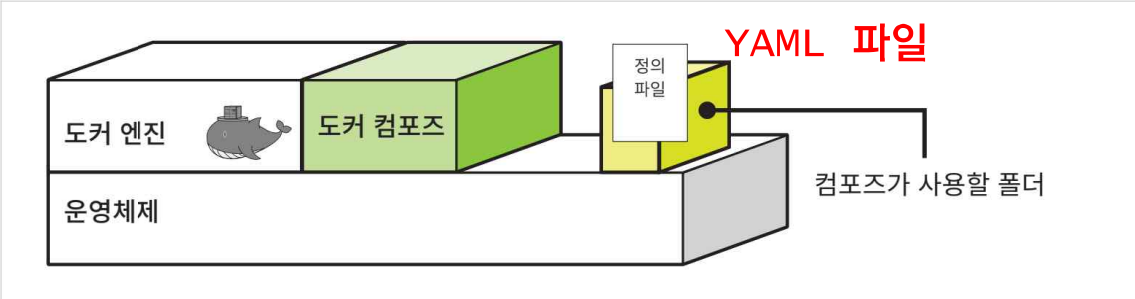


그림 7-2-1 호스트 컴퓨터에 정의 파일을 준비한다.

• 정의 파일명 : docker-compose.yml

• 도커 엔진에 의해 수행

사람이 입력하는 수동 명령을 컴포즈가 대신 전달하는 개념

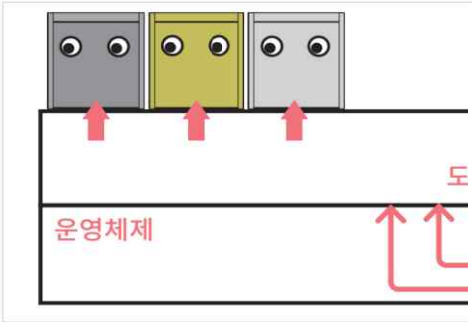


그림 7-2-2 도커 컴포즈가 명령어를 대신 입력해주는 구조

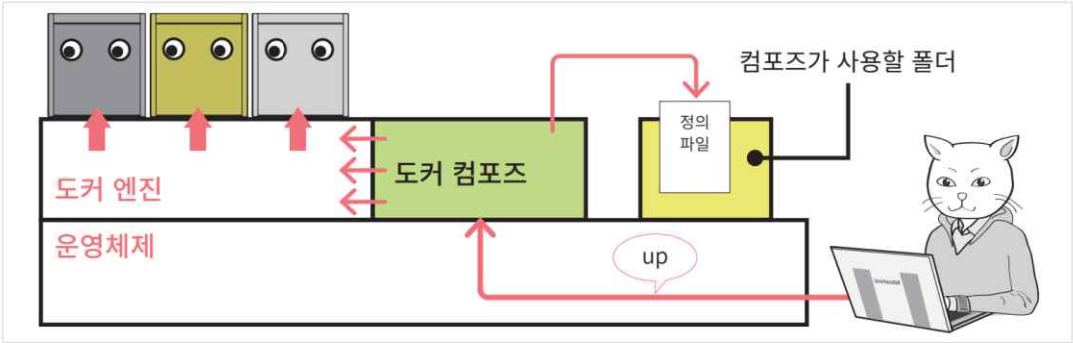


그림 7-2-3 도커 컴포즈 사용하기

1.도커 컴포즈 설치

- 정의 파일은 한 폴더에 하나만 존재
- 컨테이너 생성에 필요한 파일(이미지,HTML등)은 같은 폴더에 존재

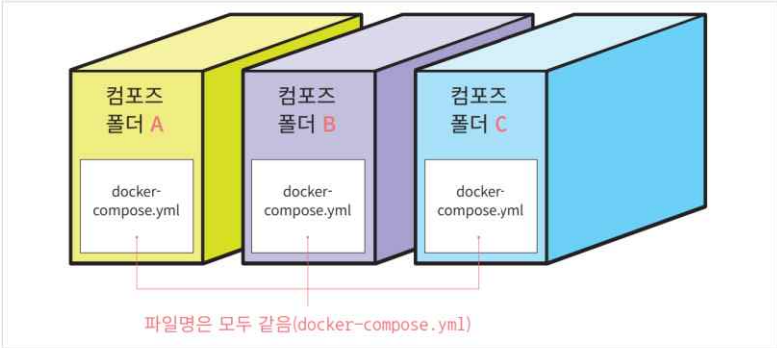


그림 7-2-4 정의 파일은 한 폴더에 하나만 있어야 한다.

<COLUMN> 서비스와 컨테이너 교재 231페이지

도커 컴포즈에서 서비스란 컨테이너가 모인 것을 의미
일반적으로 컨테이너와 서비스는 동일한 개념으로 봐도 무방

7장 도커 컴포즈를 익히자.

3절 도커 컴포즈 파일 작성방법

1.도커 컴포즈 정의 파일

교재 232페이지

- 도커 컴포즈 파일의 예1)

```
version: “3”
        https://docs.docker.com/compose/compose-file/compose-versioning/
services:
  apa000ex2:
    image: httpd
    ports:
      - 8080:80
    restart: always
```

```
docker run –name apa0002x2 –d –p 8080:80 httpd
```

1.도커 컴포즈 정의 파일

- 도커 컴포즈 파일의 예2)

```
version: “3”

services:
  wordpress000ex12:
    depends_on:
      - mysql000ex11
    image: wordpress
    networks:
      - wordpress000net1
    ports:
      - 8085:80
    restart: always
    environment:
      WORDEXPRESS_DB_HOST=mysql000ex11
      WORDEXPRESS_DB_NAME=wordexpress000db
```

```
docker run --name wordexpree000ex11 –dit
     --net=wordpress000net1
     –p 8085:80 –e WORDEXPRESS_DB_HOST=mysql000ex11
     –e WORDEXPRESS_DB_NAME=wordexpress000db
     -e WORDEXPRESS_DB_USER=  wordexpress000kun
     -e WORDEXPRESS_DB_PASSWORD=wkunpass wordpress
```

1.도커 컴포즈 정의 파일

- 파일 형식 : YAML 형식
- 파일명의 확장자 : .yml

항목	내용
정의 파일의 형식	YAML 형식
파일 이름	docker-compose.yml

다른 파일명 이용시 -f 옵션 이용하여 지정

- 컴포즈 파일 작성 방법
  - 작성 순서 : 주 항목 → 이름 추가 → 설정의 순으로 진행
  - 주 항목

```
version:      # 버전 기재

services:     # 컨테이너 관련 정보

networks :    # 네트워크 관련 정보

volumes:      # 볼륨 관련 정보
```

1.도커 컴포즈 정의 파일

- 이름 추가 시 주 항목보다 한단 들여쓰기
- 공백으로 한단 들여쓰기 사용, 한 단의 공백 개수를 동일하게 설정
- 탭 문자는 사용 못함
- 이름 뒤에는 반드시 ‘:’ 사용
- 한 줄에 이어서 설정이 필요한 경우 : 뒤에 공백문자가 1개가 반드시 필요
- 이름 뒤에 여러 개의 설정이 필요한 경우 줄을 바꿔 들여쓰기와 함께 ‘하이픈(-)+공백문자’와 함께 설정 내역 작성

그림 7-3-1 들여쓰기를 일정하게 맞춘다.

1.도커 컴포즈 정의 파일

- docker-compose.yml 템플릿

```
version: “3”
services:
  컨테이너_이름1:
    image: 이미지_이름
    networks:
      - 네트워크_이름
    ports:
      - 포트_설정

  컨테이너_이름2:
    image: 이미지_이름
  . . .
networks:
  네트워크_이름1:
  . . .
volumes:
  볼륨_이름1:
  볼륨_이름2:
  . . .
```

1.도커 컴포즈 정의 파일

- 컴포즈 파일 작성 요령

  - 첫번째 줄에 도커 컴포즈 버전 작성
  - 주 항목 services, networks, volumes 아래에 설정 내용 작성
  - 항목 간의 상하 관계는 공백을 사용한 들여쓰기로 정함
  - 들여쓰기는 같은 수의 배수만큼의 공백을 사용
  - 이름은 주 항목 아래에 들여쓰기 한 다음 작성
  - 컨테이너 설정 내용은 이름 아래 들여쓰기를 한 다음 작성
  - 여러 항목을 작성하는 경우 줄 앞에 하이픈(-)을 붙임.
  - 이름 뒤에 반드시 : 문자를 부침
  - : 문자 뒤에는 반드시 공백 문자가 와야 함.
  - # 은 주석 처리
  - 문자열은 ‘ ‘ 또는 “ “를 이용

1.도커 컴포즈 정의 파일

• 컴포즈 파일의 항목 (1of3)

주 항목

항목	내용
services	컨테이너를 정의한다.
networks	네트워크를 정의한다.
volumes	볼륨을 정의한다.

자주 나오는 정의 내용

항목	docker run 커맨드의 해당 옵션 또는 인자	내용
image	이미지 인자	사용할 이미지를 지정
networks	--net	접속할 네트워크를 지정
volumes	-v, --mount	스토리지 마운트를 설정
ports	-p	포트 설정
environment	-e	환경변수 설정
depends_on	없음	다른 서비스에 대한 의존관계를 정의
restart	없음	컨테이너 종료 시 재시작 여부를 설정

1.도커 컴포즈 정의 파일

- 컴포즈 파일의 항목 (2of3)

depends_on : 컨테이너 생성 순서나 연동 여부를 정의

예) depends_on: -namgeuk
현재 컨테이너를 namgeuk 컨테이너를 생성한 다음에 생성하라는 의미
워드프레스처럼 MySQL컨테이너가 먼저 생성되어야 하는 경우 이용

restart의 설정값

설정값	내용
no	재시작하지 않는다.
always	항상 재시작한다.
on-failure	프로세스가 0 외의 상태로 종료됐다면 재시작한다.
unless-stopped	종료 시 재시작하지 않음. 그 외에는 재시작한다.

1.도커 컴포즈 정의 파일

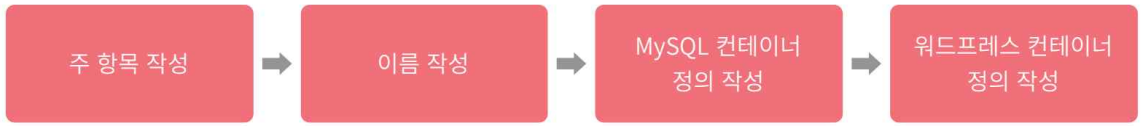
• 컴포즈 파일의 항목 (3of3)

항목	docker run 커맨드의 해당 옵션 또는 인자	내용
command	커맨드 인자	컨테이너 시작 시 기존 커맨드를 오버라이드
container_name	--name	실행할 컨테이너의 이름을 명시적으로 지정
dns	--dns	DNS 서버를 명시적으로 지정
env_file	없음	환경설정 정보를 기재한 파일을 로드
entrypoint	--entrypoint	컨테이너 시작 시 ENTRYPOINT 설정을 오버라이드
external_links	--link	외부 링크를 설정
extra_hosts	--add-host	외부 호스트의 IP 주소를 명시적으로 지정
logging	--log-driver	로그 출력 대상을 설정
network_mode	--network	네트워크 모드를 설정

1.도커 컴포즈 정의 파일

교재 240~245페이지

<실습> 컴포즈 파일 작성



- 생성할 네트워크, 볼륨 및 컨테이너 정보

항목	값
네트워크 이름	wordpress000net1
MySQL 볼륨 이름	mysql000vol11
워드프레스 볼륨 이름	wordpress000vol12
MySQL 컨테이너 이름	mysql000ex11
워드프레스 컨테이너 이름	wordpress000ex12

1.도커 컴포즈 정의 파일

• 정의 내용 (1of2)

MySQL 컨테이너(mysql000ex11)의 정의

항목	항목 이름	값
MySQL 이미지 이름	image:	mysql:5.7
사용할 네트워크	networks:	wordpress000net1
사용할 볼륨	volumes:	mysql000vol11
마운트 위치		/var/lib/mysql
재시작 설정	restart:	always
MySQL 설정	environment:	★이 붙은 항목 설정
★MySQL 루트 패스워드	MYSQL_ROOT_PASSWORD	myrootpass
★MySQL 데이터베이스 이름	MYSQL_DATABASE	wordpress000db
★MySQL 사용자 이름	MYSQL_USER	wordpress000kun
★MySQL 패스워드	MYSQL_PASSWORD	wkunpass

1.도커 컴포즈 정의 파일

• 정의 내용 (2of2)

워드프레스 컨테이너(wordpress000ex12)의 정의

항목	항목 이름	값
의존관계	depends_on:	mysql000ex11
워드프레스 이미지 이름	image:	wordpress
사용할 네트워크	networks:	wordpress000net1
사용할 볼륨	volumes:	wordpress000vol12
마운트 위치		/var/www/html
포트 번호 설정	port:	8085:80
재시작 설정	restart:	always
데이터베이스 관련 정보	environment:	★이 붙은 항목 설정
★데이터베이스 컨테이너 이름	WORDPRESS_DB_HOST	mysql000ex11
★데이터베이스 이름	WORDPRESS_DB_NAME	wordpress000db
★데이터베이스 사용자 이름	WORDPRESS_DB_USER	wordpress000kun
★데이터베이스 패스워드	WORDPRESS_DB_PASSWORD	wkunpass

1.도커 컴포즈 정의 파일

• 컴포즈 파일 배치 경로

항목	값
컴포즈 파일 배치 경로(윈도우)	C:\Users\사용자명\Documents\com_folder
컴포즈 파일 배치 경로(macOS)	/Users/사용자명/Documents/com_folder
컴포즈 파일 배치 경로(리눅스)	/home/사용자명/com_folder

• docker-compose.yml 파일 작성

1) 주항목 작성

```
version: “3”
services:
networks:
volumes:
```

1.도커 컴포즈 정의 파일

2) 이름 작성

3) MySQL 컨

4) 워드프레스

5) 파일 저장

<COLUMN> MYSQL8.0 사용하는 방법 참조

교재 245페이지

```
version: "3"
services:
  mysql000ex11:
    image: mysql:5.7
    networks:
      - wordpress000net1
    volumes:
      - mysql000vol11:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: myrootpass
      MYSQL_DATABASE: wordpress000db
      MYSQL_USER: wordpress000kun
      MYSQL_PASSWORD: wkunpass
  wordpress000ex12:
    depends_on:
      - mysql000ex11
    image: wordpress
    networks:
      - wordpress000net1
    volumes:
      - wordpress000vol2:/var/www/html
    ports:
      - 8085:80
    environment:
      WORDPRESS_DB_HOST: mysql000ex11
      WORDPRESS_DB_NAME: wordpress000db
      WORDPRESS_DB_USER: wordpress000kun
      WORDPRESS_DB_PASSWORD: wkunpass
networks:
  wordpress000net1:
volumes:
```

7장 도커 컴포즈를 익히자.

4절 도커 컴포즈 실행

1.도커 컴포즈 명령

- 컨테이너와 주변환경을 생성하는 명령

```
docker-compose -f 정의_파일_이름 up 옵션
```

옵션 항목

옵션	내용
-d	백그라운드로 실행
--no-color	화면 출력 내용을 흑백으로 함
--no-deps	링크된 서비스를 실행하지 않음
--force-recreate	설정 또는 이미지가 변경되지 않더라도 컨테이너를 재생성
--no-create	컨테이너가 이미 존재할 경우 다시 생성하지 않음

사용 예)

```
docker-compose -f C:\Users\사용자명\Documents\com_folder\docker-compose.yml up -d
```

현재 작업 디렉터리를 컴포즈용으로 사용할 때에는 따로 콤포즈 파일을 지정하지 않아도 됨(교재 249 COLUMN 참조)

```
docker-compose up -d
```

1.도커 컴포즈 명령

• 컨테이너와 네트워크 삭제 명령

```
docker-compose -f 정의_파일_이름 down 옵션
```

옵션 항목

옵션	내용
--rmi 종류	삭제 시에 이미지도 삭제한다. 종류를 all로 지정하면 사용했던 모든 이미지가 삭제된다. local로 지정하면 커스텀 태그가 없는 이미지만 삭제한다.
-v, --volumes	volumes 항목에 기재된 볼륨을 삭제한다. 단, external로 지정된 볼륨은 삭제되지 않는다.
--remove-orphans	컴포즈 파일에 정의되지 않은 서비스의 컨테이너도 삭제한다.

• 컨테이너를 종료하는 명령

```
docker-compose -f 정의_파일_이름 stop
```

1.도커 컴포즈 명령

• docker-compose 주요 명령 목록

커맨드	내용
up	컨테이너를 생성하고 실행한다.
down	컨테이너와 네트워크를 종료하고 삭제한다.
ps	컨테이너 목록을 출력한다.
config	컴포즈 파일을 확인하고 내용을 출력한다.
port	포트 설정 내용을 출력한다.
logs	컨테이너가 출력한 내용을 화면에 출력한다.
start	컨테이너를 시작한다.
stop	컨테이너를 종료한다.
kill	컨테이너를 강제 종료한다.
exec	명령어를 실행한다.
run	컨테이너를 실행한다.

create	컨테이너를 생성한다.
restart	컨테이너를 재실행한다.
pause	컨테이너를 일시 정지한다.
unpause	컨테이너의 일시 정지를 해제한다.
rm	종료된 컨테이너를 삭제한다.
build	컨테이너에 사용되는 이미지를 빌드 혹은 재빌드한다.
pull	컨테이너에 사용되는 이미지를 내려받는다.
scale	컨테이너의 수를 지정한다.
events	컨테이너로부터 실시간으로 이벤트를 수신한다.
help	도움말 화면을 출력한다.

1.도커 컴포즈 명령

교재 251~253페이지

<실습> 도커 컴포즈 실행

- 실습에 사용할 컴포즈 파일

항목	값
윈도우	C:\Users\사용자명\Documents\com_folder\docker-compose.yml
macOS	/Users/사용자명/Documents/com_folder/docker-compose.yml
리눅스	/home/사용자명/com_folder/docker-compose.yml

- 컴포즈 파일 실행 및 확인

```
docker-compose up -d
```

```
$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED              STATUS
         PORTS                    NAMES
522eba9c9c6c   wordpress   "docker-entrypoint.s…"   53 seconds ago       Up 49 s
econds   0.0.0.0:8085->80/tcp   com_folder-wordpress000ex12-1
dd44395b2617   mysql:5.7   "docker-entrypoint.s…"   About a minute ago   Up 50 s
econds   3306/tcp, 33060/tcp    com_folder-mysql000ex11-1
```

1.도커 컴포즈 명령

교재 251~253페이지

- 웹브라우저에서 워드프레스 접근

  http://localhost:8085

그림 7-4-2 워드프레스의 초기 화면

- 컨테이너와 네트워크 종료 및 삭제

```
docker-compose down -d
```

```
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS   PORTS   NAMES
```

8장 쿠버네티스를 익히자

1절 쿠버네티스란?

1.쿠버네티스(Kubernetes)란?

- 컨테이너 오케스트레이션 도구

  시스템 전체를 통괄하고, 여러 컨테이너를 지휘(관리)하는 일

- 'K8s' 라고도 부름

그림 8-1-1 쿠버네티스

- 쿠버네티스는 대규모 시스템 관리에 적합

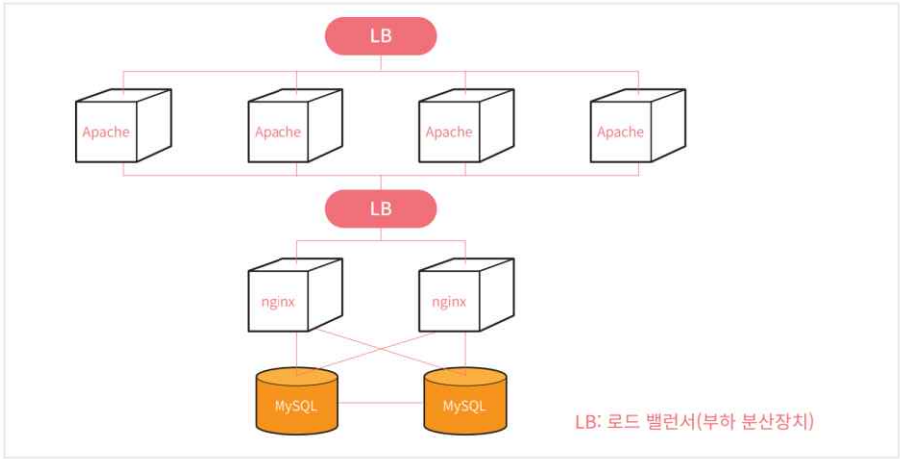


그림 8-1-2 쿠버네티스는 대규모 시스템에 적용되는 경우가 많다.

1.쿠버네티스(Kubernetes)란?

- 프로그램 개발자가 쿠버네티스를 활발하게 사용할 일은 많지 않지만 시스템 개발시 쿠버네티스로 어떤 일을 할 수 있는지 알아야 장점을 잘 활용할 수 있다.

- 쿠버네티스는 여러 대의 컨테이너가 여러 대의 서버에 걸쳐 실행되는 것을 전제로 한다.

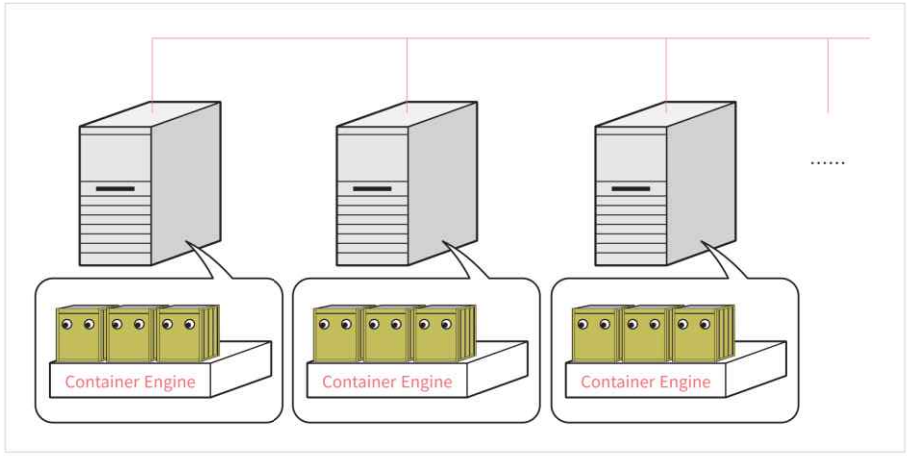


그림 8-1-3 쿠버네티스는 여러 대의 물리적 서버가 존재한다는 것을 전제로 한다.

- 번거로운 컨테이너 생성 및 관리의 수고를 덜어주는 도구

도커 컴포즈와 유사하게 정의 파일(메니페스트 파일)만 작성하면 모든 물리 서버에 컨테이너를 생성하고 관리해 줌

8장 쿠버네티스를 익히자

2절 마스터 노드와 워커 노드

1.클러스터의 구성-마스터 노드와 워커노드

- 쿠버네티스 구성
  - 마스터 노드 : 제어 노드
  - 워커 노드 : 실제 동작 담당
- 마스터 노드와 워커 노드의 역할

마스터 노드 역할	워커 노드 역할
컨테이너 실행 안함 워커 노드에서 실행되는 컨테이너 관리 컨테이너 엔진 설치 안됨	실제 서버 역할 컨테이너가 실제 동작하는 서버 컨테이너 엔진 설치

<참고>
쿠버네티스 1.2버전부터 컨테이너 엔진으로 도커 이외에도 containerd/crio/rtk 등 추천(2021년1월 기준)

- 클러스터
  - 마스터 노드와 워커 노드로 구
  - 마스터 노드에 설정된 내용에 따라 사람의 개입없이 자율적으로 워커 노드가 관리

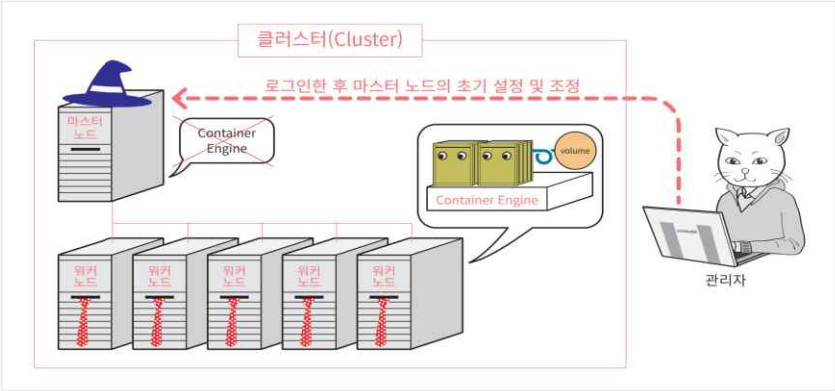


그림 8-2-1 클러스터는 설정 파일의 내용에 따라 자율적으로 동작한다.

1.클러스터의 구성-마스터 노드와 워커노드

- 쿠버네티스 설치
  - 도커 엔진과는 별개로 쿠버네티스 소프트웨어와 CNI(가상 네트워크 드라이버)를 설치 필요
  - 대표적인 CNI 소프트웨어 : 플란넬(flannel), 칼리코(calico), AWS VPC CNI 등

마스터 노드	워커 노드
etcd 데이터베이스 (컨테이너 상태 관리용) kubectl 도구	도커와 같은 컨테이너 엔진

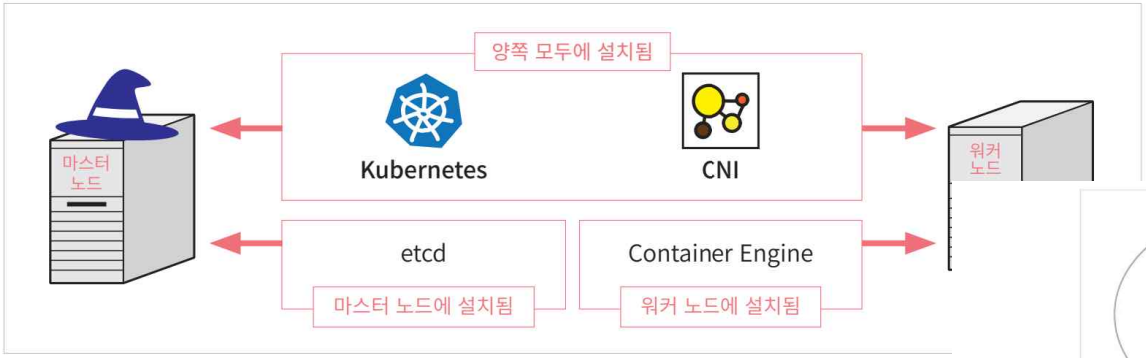


그림 8-2-2 마스터 노드, 워커 노드에는 각기 다른 소프트웨어가 설치된다.

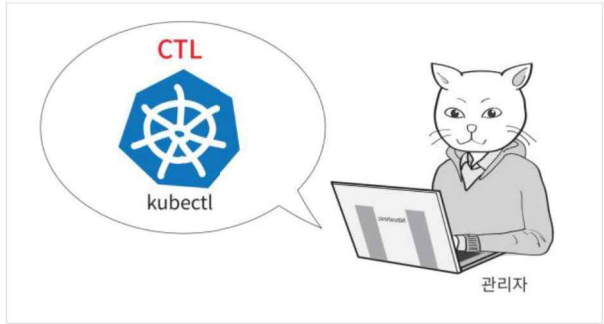


그림 8-2-3 관리자의 컴퓨터에는 kubectl을 설치해야 한다.

1.클러스터의 구성-마스터 노드와 워커노드

- Control-plane(제어판)
  - 마스터 노드에 설치
  - 워커 노드 관리용
  - 쿠버네티스 안에 포함되어 있음

마스터 노드측 control-plane의 구성

항목	내용
kube-apiserver	외부와 통신하는 프로세스, kubectl로부터 명령을 전달받아 실행한다.
kube-controller-manager	컨트롤러를 통합 관리, 실행한다.
kube-scheduler	파드를 워커 노드에 할당한다.
cloud-controller-manager	클라우드 서비스와 연동해 서비스를 생성한다.
etcd	클러스터 관련 정보 전반을 관리하는 데이터베이스

- kube-let
  - 워커 노드에 설치
  - 마스터 노드의 kube-scheduler와 연동하여 워커 노드에 컨테이너 또는 볼륨을 배치하고 실행
  - Kubernetis 소프트웨어 안에 포함되어 있음

워커 노드의 구성

항목	내용
kube-let	마스터 노드에 있는 kube-scheduler와 연동하며 워커 노드에 파드를 배치하고 실행한다. 또 실행 중인 파드의 상태를 정기적으로 모니터링하며 kube-scheduler에 통지한다.
kube-proxy	네트워크 통신의 라우팅 메커니즘

1.클러스터의 구성-마스터 노드와 워커노드

• 쿠버네티스 동작 구조

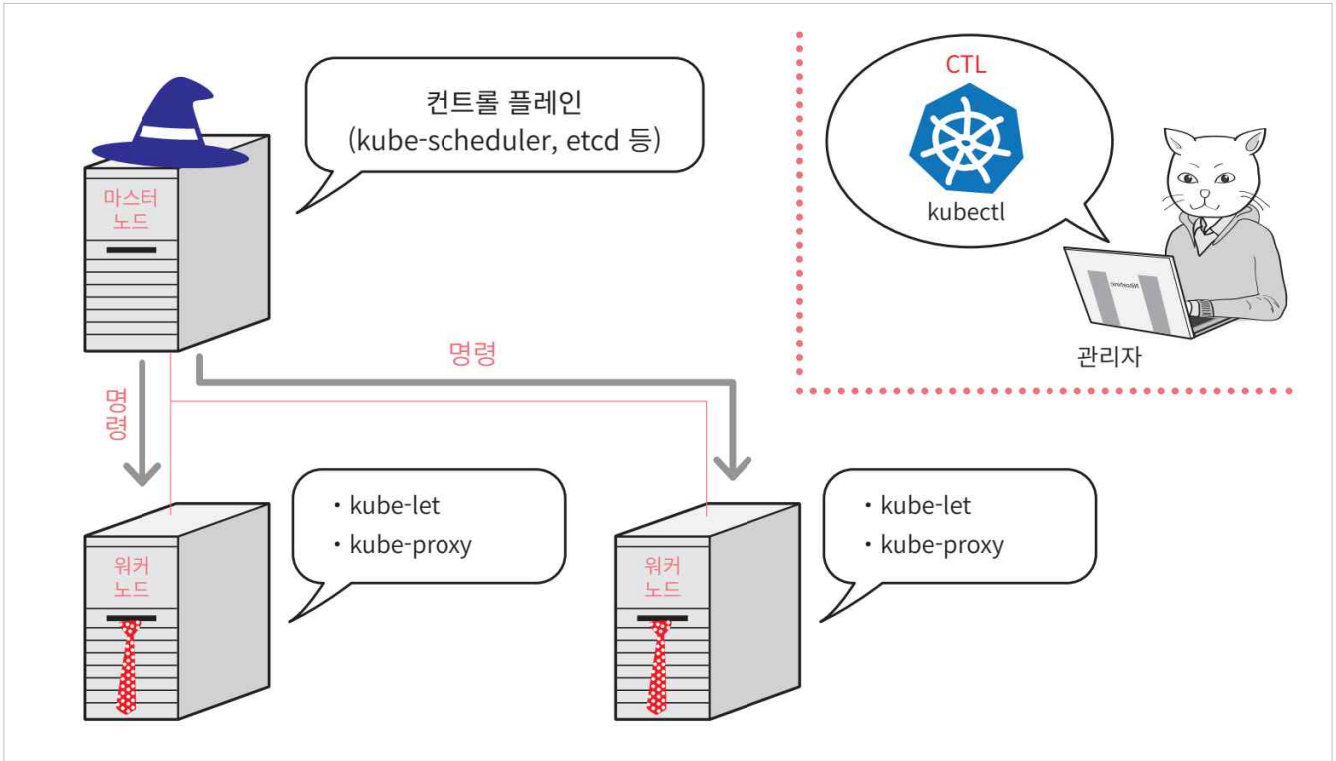


그림 8-2-4 쿠버네티스의 동작 구조

2.쿠버네티스는 항상 '바람직한' 상태를 유지한다

- 쿠버네티스에서는 일일이 명령을 통해 컨테이너를 생성/삭제하는 것이 아니라 '바람직한' 상태를 YAML 파일에 정의하고 자동으로 컨테이너를 생성 및 삭제하면서 이 상태를 유지하도록 함

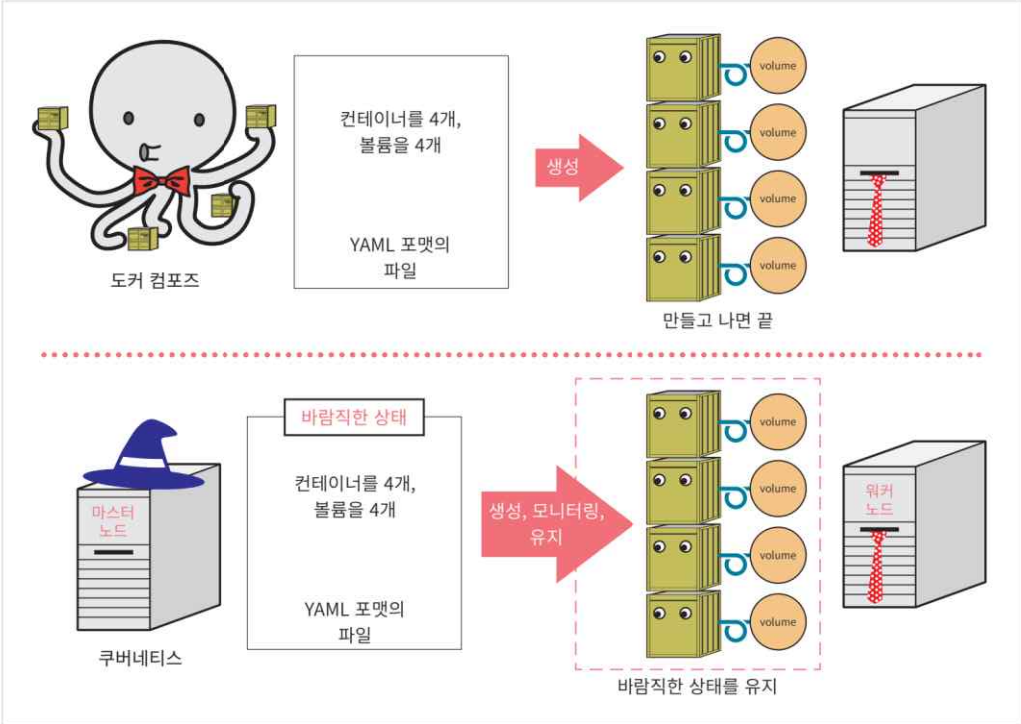


그림 8-2-5 도커 컴포즈와 쿠버네티스의 차이점

도커 컴포즈와의 차이점 : 여러 대의 물리 서버에 걸쳐 시스템을 구성할 수 있음

2.쿠버네티스는 항상 '바람직한' 상태를 유지한다

- '바람직한 상태' 유지의 예

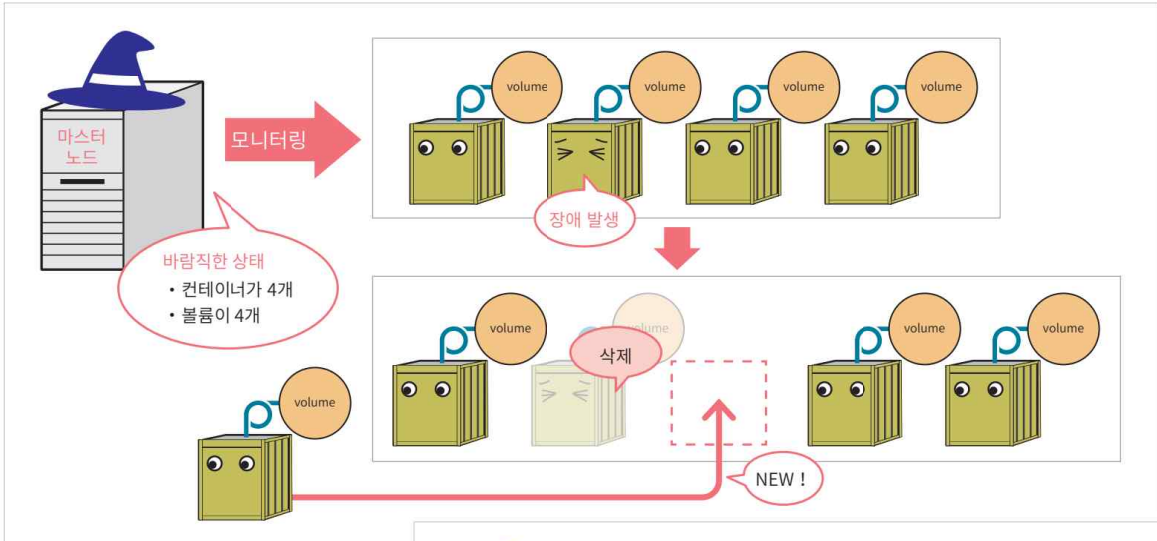


그림 8-2-6 컨테이너가 하나 망가지면 해당

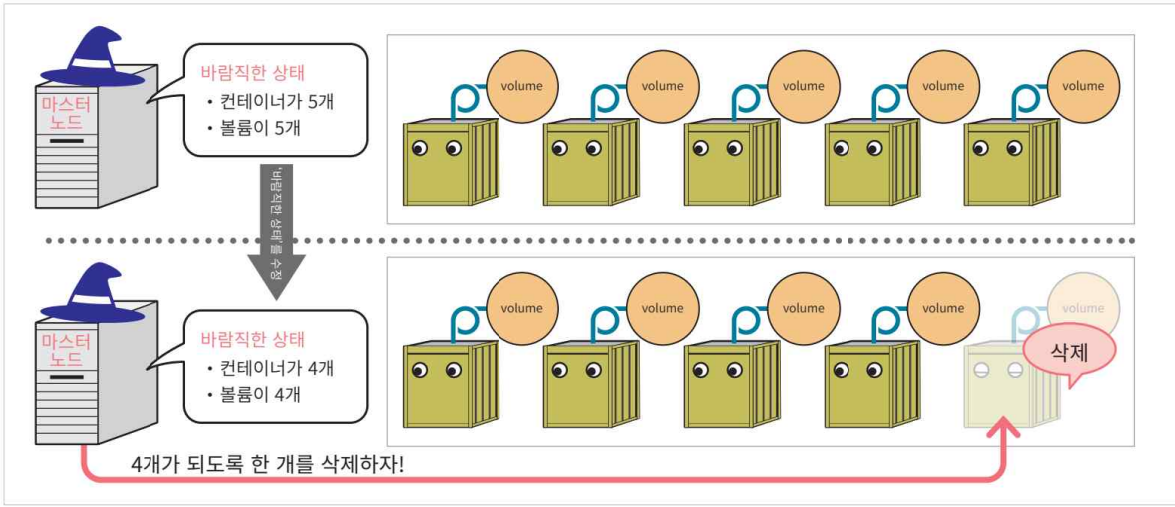


그림 8-2-7 바람직한 상태가 바뀌어도 삭제 또는 생성을 통해 정해진 상태를 유지한다.

2.쿠버네티스는 항상 '바람직한' 상태를 유지한다

- 컨테이너 삭제
  - 삭제 명령 대신 정의 파일에서 '바람직한 상태'를 수정

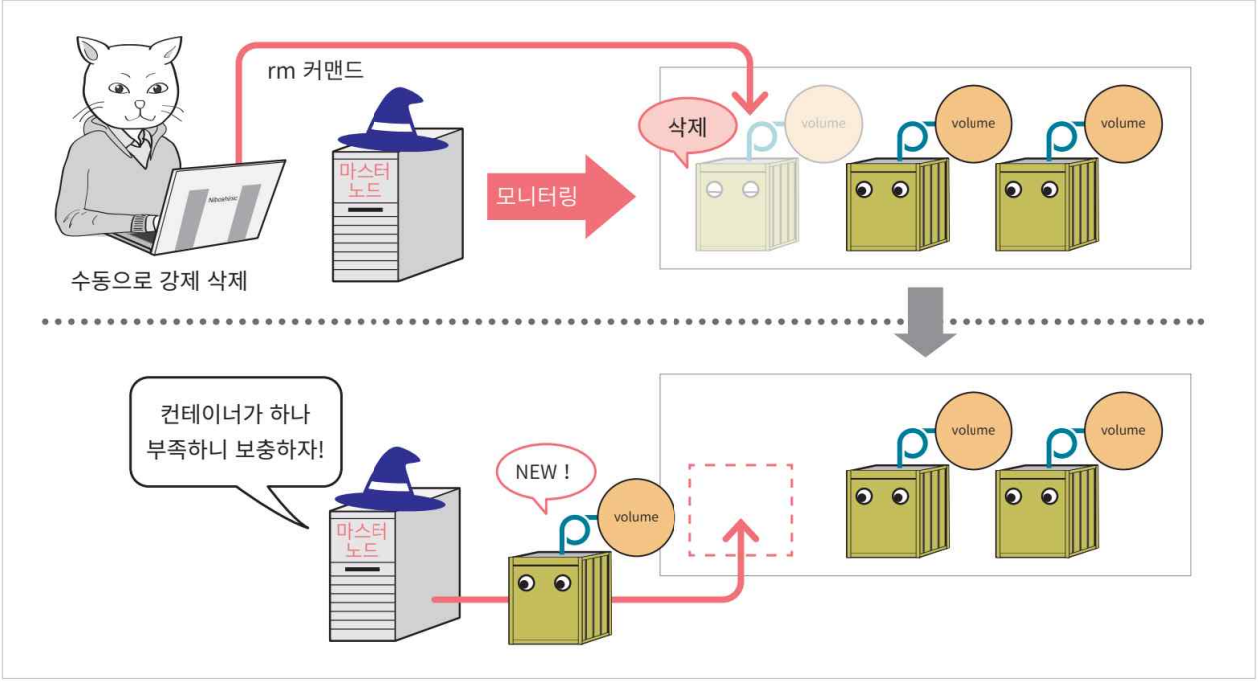


그림 8-2-8 관리자가 직접 컨테이너를 삭제해도 쿠버네티스가 대체 컨테이너를 생성한다.

- 쿠버네티스의 목표는 정의된 '바람직한' 상태를 유지하는 것이다.

8장 쿠버네티스를 익히자

3절 쿠버네티스의 구성과 용어

1.쿠버네티스 구성과 관련된 용어

- 파드(pod)
  - 쿠버네티스에서 관리하는 컨테이너 단위
  - 컨테이너와 볼륨을 하나로 묶은 것(볼륨이 없는 경우도 있음)
  - 1개의 컨테이너 또는 여러 개의 컨테이너로 구성될 수도 있음

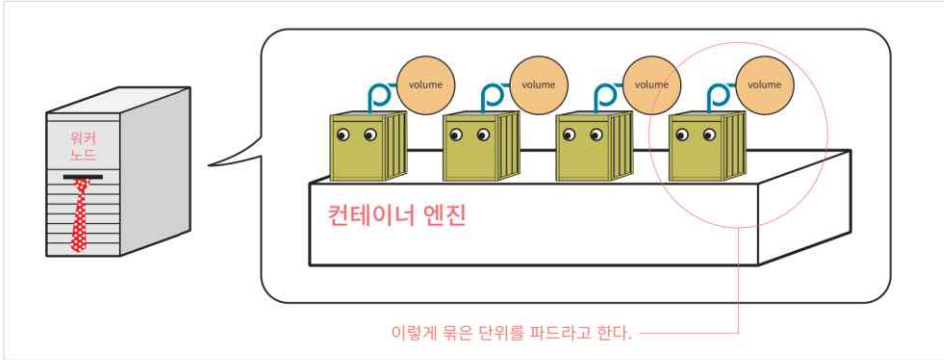


그림 8-3-1 파드는 컨테이너와 볼륨을 묶은 것이다.

1.쿠버네티스 구성과 관련된 용어

- 서비스(service)
  - 여러 개의 파드를 하나로 관리
  - 서비스가 관리하는 파드는 모두 동일한 구성으로 이루어짐.

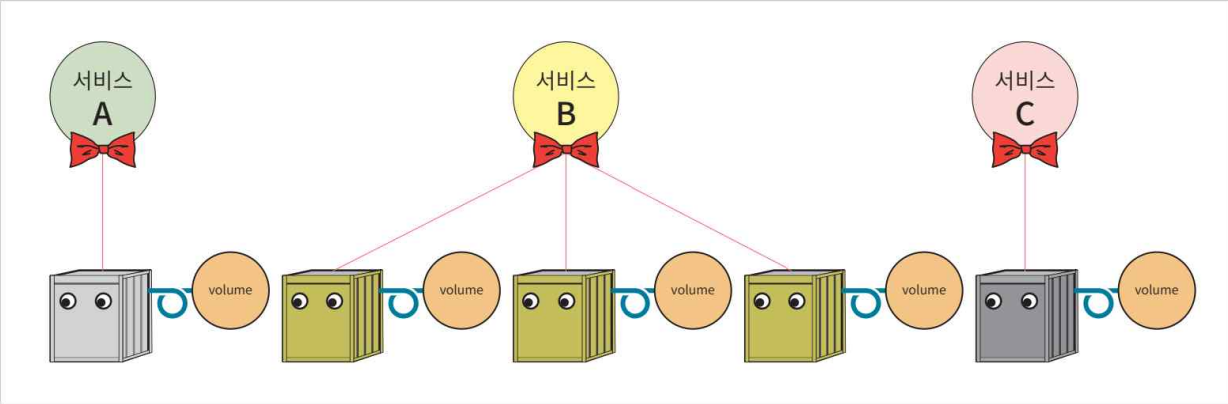


그림 8-3-2 같은 종류의 파드를 하나의 서비스가 관리한다.

  - 서비스의 역할 : 로드밸런서(부하분산) 역할
  - 각 서비스는 자동으로 고정된 IP 주소(clusterIP)를 부여받으며, 이 주소로 통신
  - 여러 개의 파드로 구성되어도 IP 주소는 1개만 부여

1.쿠버네티스 구성과 관련된 용어

- 서비스(service) (2of2)
  - 서비스가 분배하는 통신은 한 워크 노드 안으로 국한됨
  - 여러 워커 노드간 분배는 실제로 로드밸런서나 인그레스(ingress)가 담당

HTTP/HTTPS응용 계층에서 동작하는 리버스 프락시, 로드밸런서와 유사

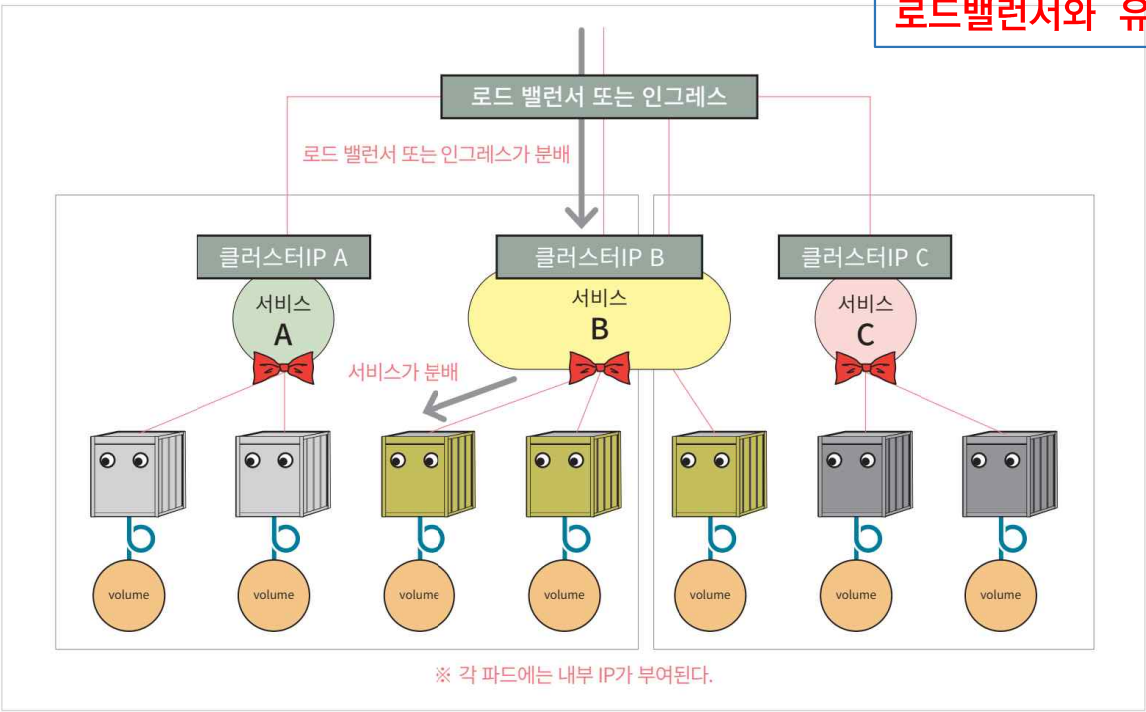


그림 8-3-4 서비스는 각 파드에 요청을 배분한다.

1.쿠버네티스 구성과 관련된 용어

- 레플리카세트
  - 파드의 수를 관리하는 역할
  - 장애등으로 파드가 종료됐을 때 부족한 파드를 보충하거나 정의 파일의 파드 수가 감소하면 파드 수를 실제로 감소시킴

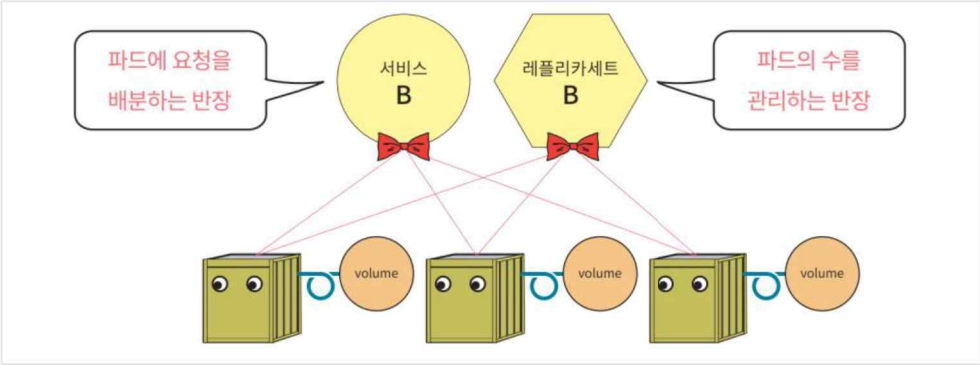


그림 8-3-5 레플리카세트는 파드의 수를 관리한다.

  - 레플리카 : 레플리카세트가 관리하는 동일한 구성의 파드
    파드의 수를 조정하는 것을 '레플리카의 수를 조정'
    또는 '레플리카의 수를 결정'한다고 표현
  - 디플로이먼트와 함께 사용

1.쿠버네티스 구성과 관련된 용어

- 디플로이먼트
  - 파드의 디플로이(배포)를 관리하는 요소
  - 파드가 사용하는 이미지 등 파드에 대한 정보를 가지고 있음
  - 레플리카세트를 관리

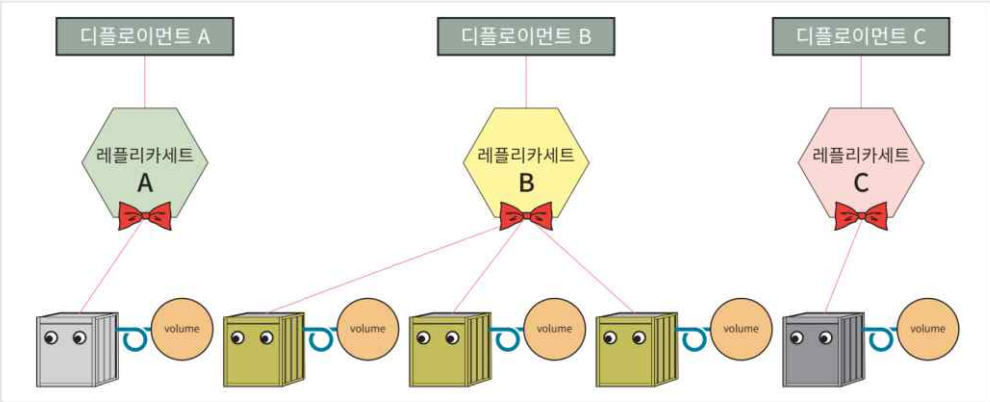


그림 8-3-7 디플로이먼트는 파드의 배포를 관리한다.

1.쿠버네티스 구성과 관련된 용어

• 기타 쿠버네티스 리소스

주요 쿠버네티스 리소스

리소스 이름	내용
파드(pods)	파드. 컨테이너와 볼륨을 합친 것.
파드템플릿(podtemplates)	배포 시 파드의 형틀 역할
레플리케이션컨트롤러(replicationcontrollers)	레플리케이션을 제어
리소스쿼터(resourcequotas)	쿠버네티스 리소스의 사용량 제한을 설정
비밀값(secrets)	키 정보를 관리
서비스어카운트(serviceaccounts)	리소스를 다루는 사용자를 관리
서비스(services)	파드에 요청을 배분
데몬세트(daemonsets)	워커 노드마다 하나의 파드를 생성
디플로이먼트(deployments)	파드의 배포를 관리
레플리카세트(replicasets)	파드의 수를 관리
스테이트풀세트(statefulsets)	파드의 배포를 상태를 유지하며 관리
크론잡(cronjobs)	지정된 스케줄대로 파드를 실행
잡(jobs)	파드를 한번 실행

1.쿠버네티스 구성과 관련된 용어

• 쿠버네티스 용어정리

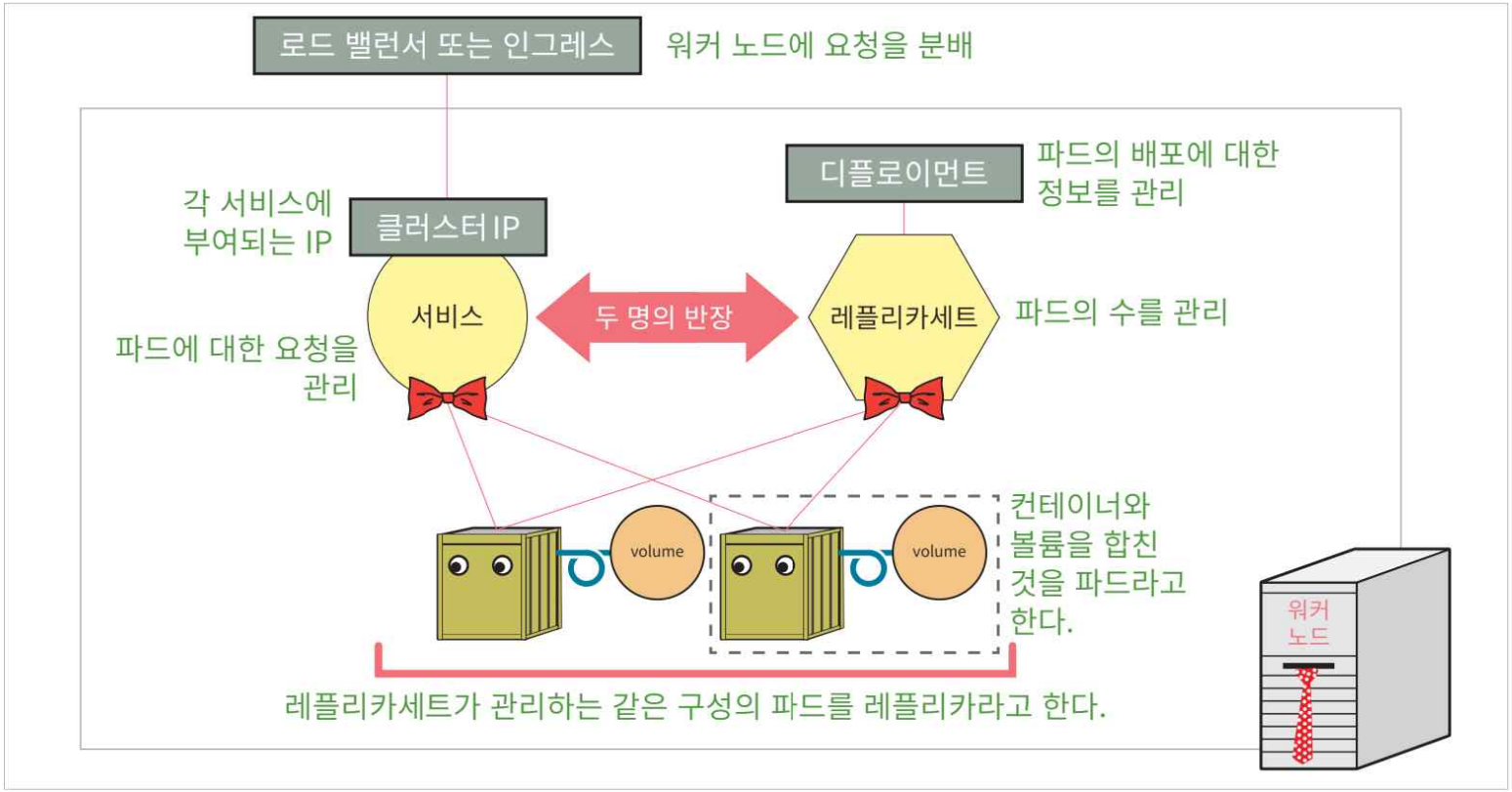


그림 8-3-8 용어 정리

1.쿠버네티스 구성과 관련된 용어

• 오브젝트와 인스턴스

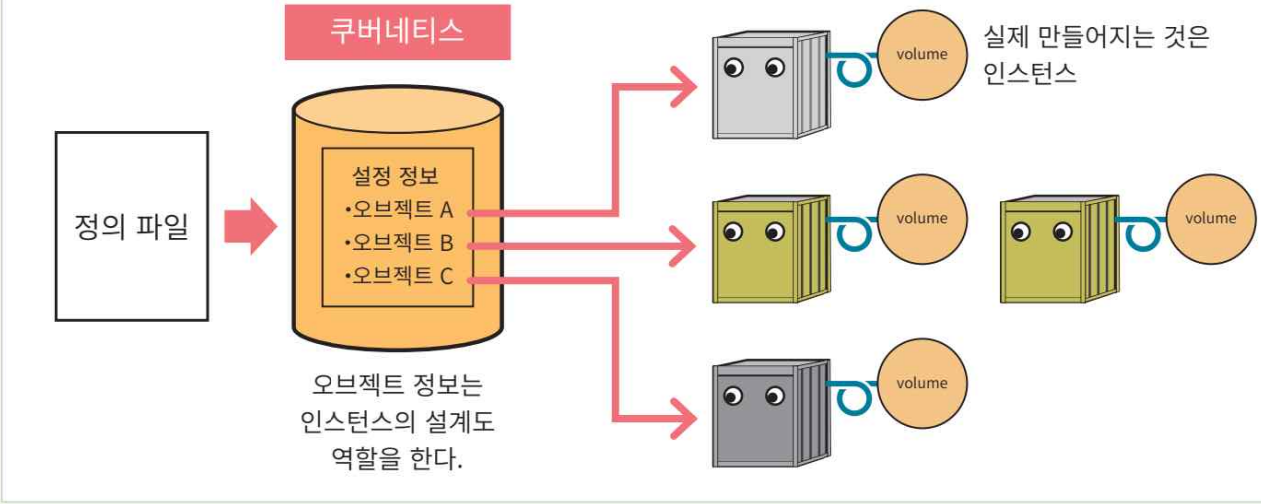


그림 8-3-9 etcd에 등록된 설정 정보를 따라 인스턴스를 생성한다.

8장 쿠버네티스를 익히자

4절 쿠버네티스 설치 및 사용법

1.쿠버네티스 종류

- 쿠버네티스
  - 클라우드 네이티브 컴퓨팅 재단(Cloud Native Computing Foundation, CNCF)에서 정한 표준
  - 원래 구글에서 개발, CNCF에 기부하여 오픈소스로 전환

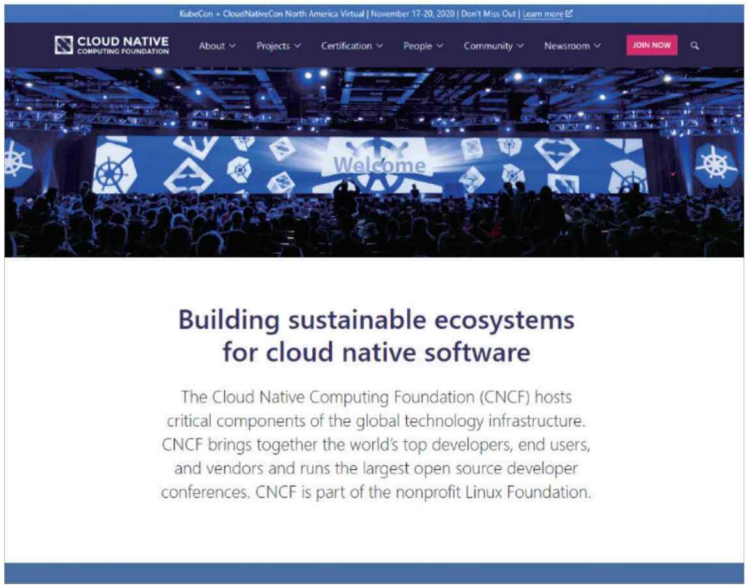


그림 8-4-1 CNCF 공식 웹 사이트(https://www.cncf.io/)

  - 관리 기능을 강화한 버전이나 크기를 줄인 버전 등 서드파티 버전이 발표되고 있음
  - AWS나 애저 및 GCP같은 클라우드 서비스에서 커스터마이징된 쿠버네티스 서비스 제공

1.쿠버네티스 종류

- 호환성이 검증된 소프트웨어 및 서비스

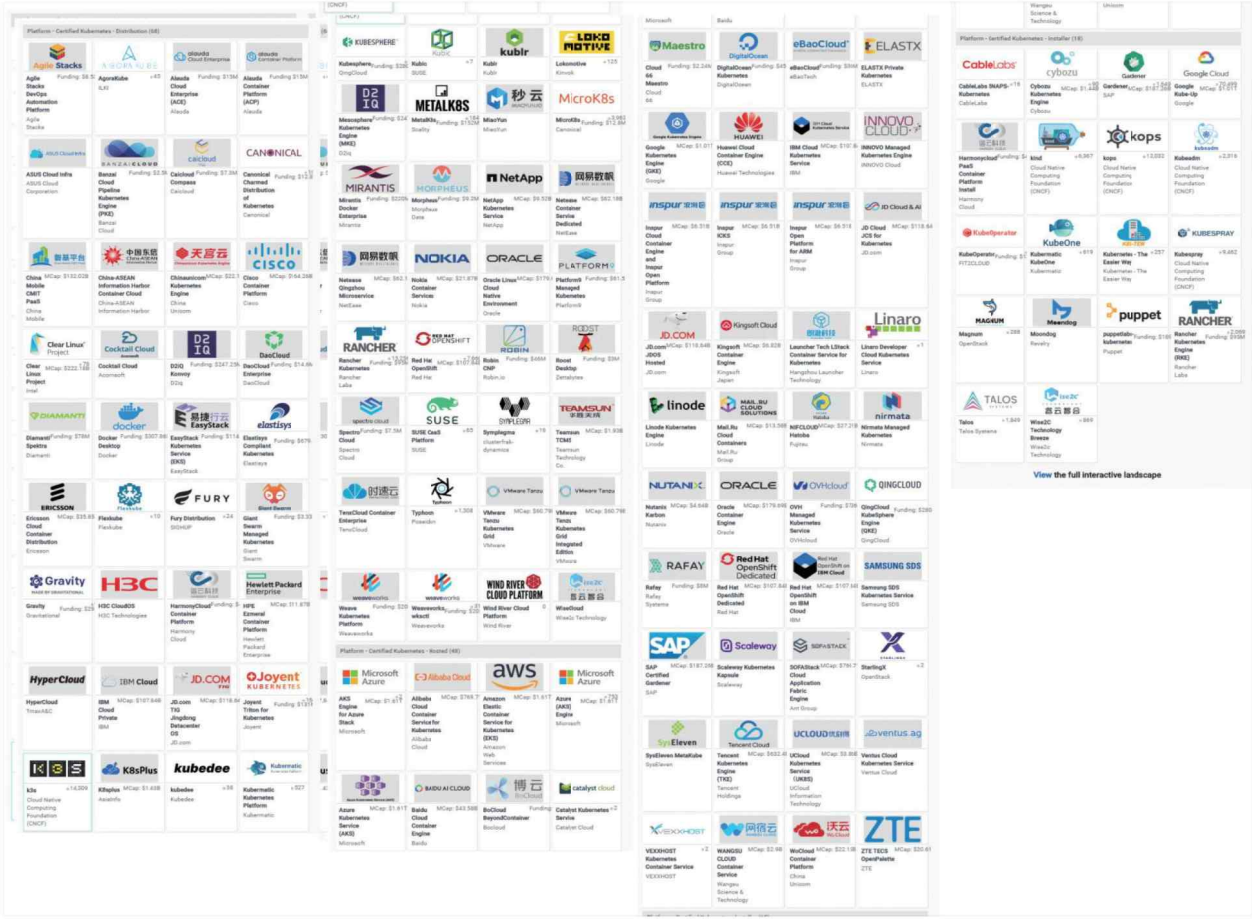


그림 8-4-2 Certified Kubernetes 웹 페이지(https://www.cncf.io/certification/software-conformance/의 내용을 기초로 작성).

1.쿠버네티스 종류

- 원조 쿠버네티스와 클라우드 버전
  - 원조 쿠버네티스를 많이 사용하기는 하지만 직접 구축이 용이하지 않음
  - 시스템 통합 전문가에게 의뢰하는 일이 많음
  - 클라우드 버전의 경우 전무적인 지식이 없어도 이용가능
  - 예) AWS Ec2나 Fargate를 워커 노드로 구성하고 EKS를 사용해 관리, 서버 대수만큼 비용 부과

- minikube
  - 컴퓨터 한 대에 마스터 노드와 워커 노드를 구축하여 쿠버네티스 기능을 사용할 수 있음

1.쿠버네티스 종류

- 도커 데스크톱과 minikube
  - 도커 데스크톱의 설정에서 Kubernetes 를 체크하면 바로 사용 가능

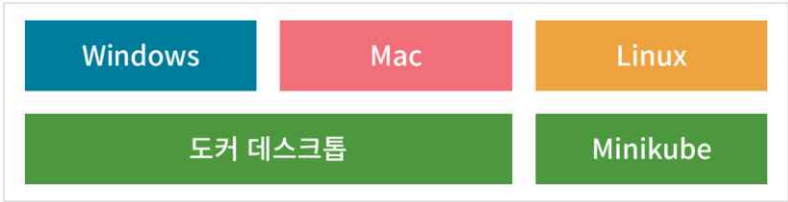


그림 8-4-3 학습 목적에 적합한 쿠버네티스

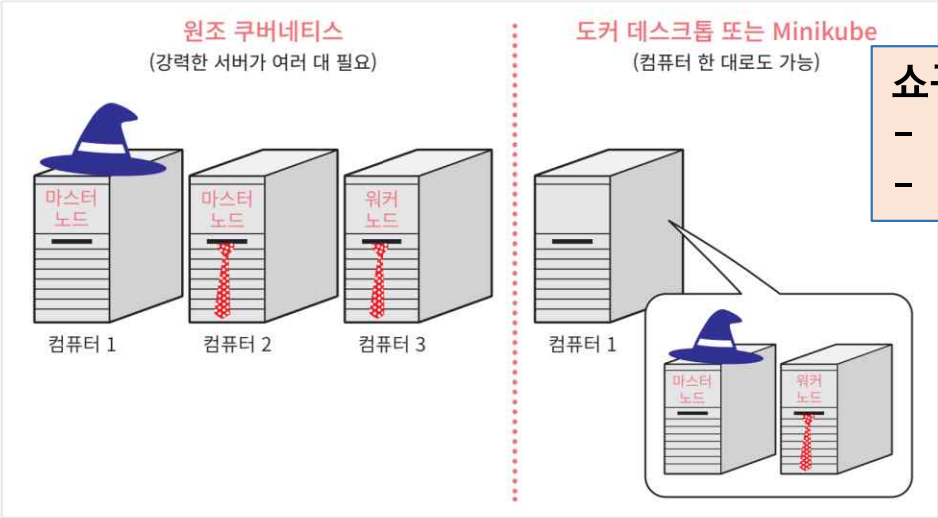


그림 8-4-4 도커 데스크톱이나 Minikube를 사용하면 컴퓨터 한 대로도 쿠버네티스를 배울 수 있다.

쇼규모의 쿠버네티스는 학습용
- 명령어 연습
- 메니페스트 파일 작성 연습

1.쿠버네티스 종류

<실습> 도커 데스크톱의 쿠버네티스 준비

- 도커 데스크톱의 설정 메뉴 선택, [Kubernetes] 탭에서 'Enable Kubernetes' 선택

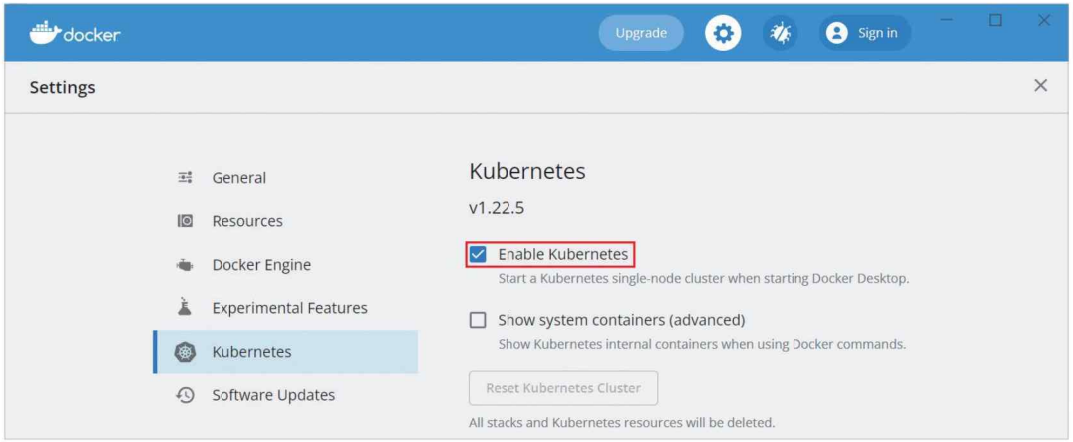


그림 8-4-5 쿠버네티스 활성화

1.쿠버네티스 종류

- 쿠버네티스 클러스터 설치

  Kubernetis Cluster Install 대화창이 나타나면 Install 을 선택하여 쿠버네티스 클러스터 설치

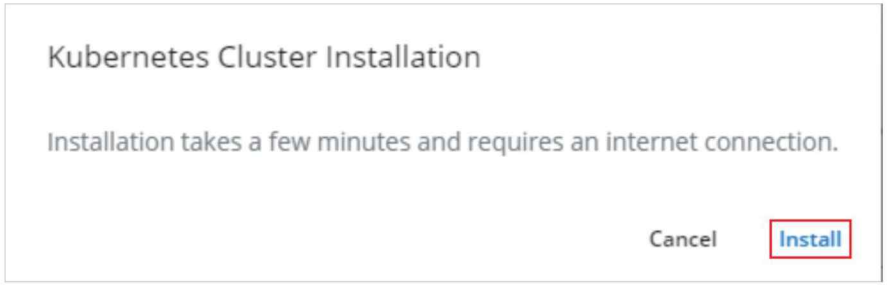


그림 8-4-6 쿠버네티스 클러스터 설치

- 준비완료 - 쿠버네티스 실행

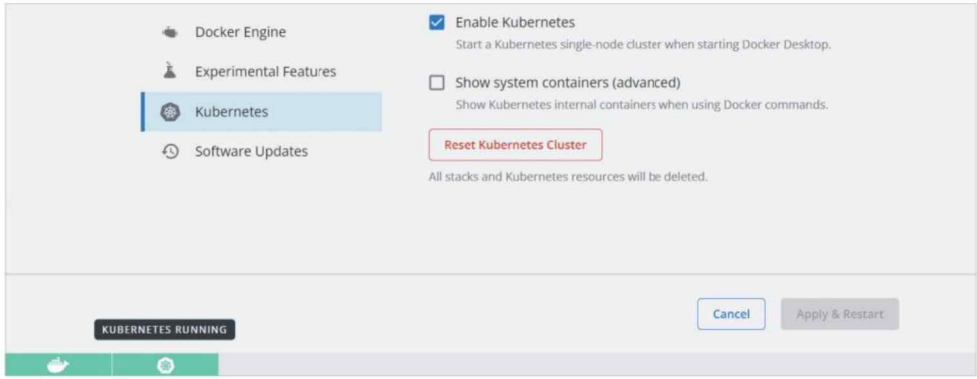


그림 8-4-7 Kubernetes의 상태가 'running'으로 바뀐다.

8장 쿠버네티스를 익히자

5절 메니페스트 파일 작성

1. 매니페스트 파일이란?

- 매니페스트 파일을 쿠버네티스에 업로드하면, 그 내용을 etcd 데이터베이스에 '바람직한 상태'로 등록하고 서버를 그 상태로 유지

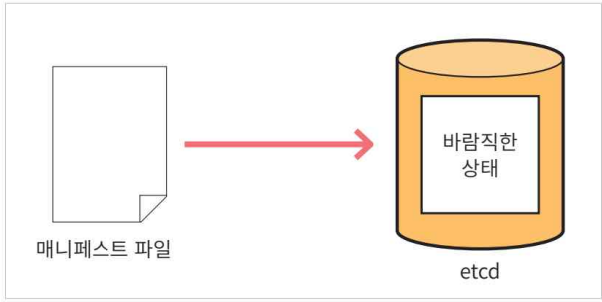


그림 8-5-1 매니페스트 파일을 작성해 etcd에 등록한다.

- 매니페스트 파일 작성
  - YAML 또는 JSON 형식으로 작성
  - 파드나 서비스에 대한 설정
  - 파일명은 정해져 있지 않음

쿠버네티스 정의 파일

항목	내용
파일 형식	YAML 형식(.yml)
파일 이름	임의의 이름

1. 매니페스트 파일이란?

- 리소스 단위로 작성
  - 리소스 : 파드, 서비스, 디플로이먼트, 레플리카세트 등

그림 8-5-2 초보자가 다루는 주요 리소스는 두 가지다.

  - 디플로이먼트 리소스에 파드와 레플리카세트의 기능이 포함되어있어 주로 디플로이먼트 리소스와 서비스 리소스를 많이 사용한다. 예를 들어 아파치 파드를 만들려면 '아파치 디플로이먼트'와 '아파치 서비스' 리소스를 작성한다

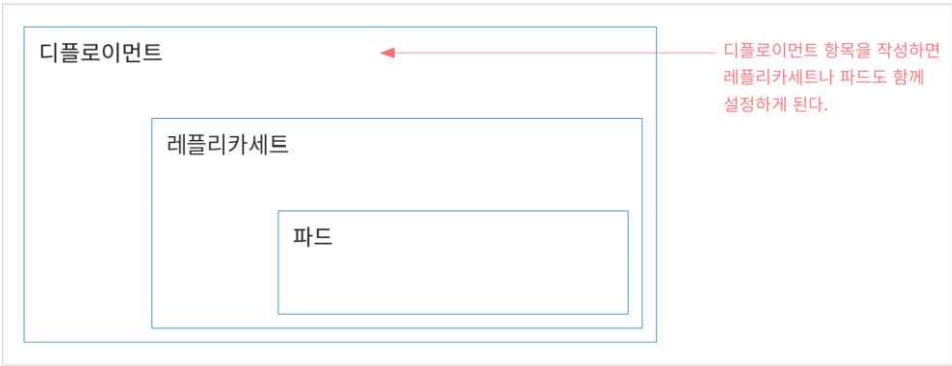


그림 8-5-3 '레플리카세트'와 '파드'는 '디플로이먼트'에서 설정할 수 있다.

1. 매니페스트 파일이란?

- 매니페스트 파일 분할
  - 리소스 단위로 분할 가능
  - 하나의 파일로 작성하는 경우에는 각 리소스를 '---'로 구분
  - 리소스 단위로 나눌 때는 파일명에 리소스 구별이 되도록 이름 지정

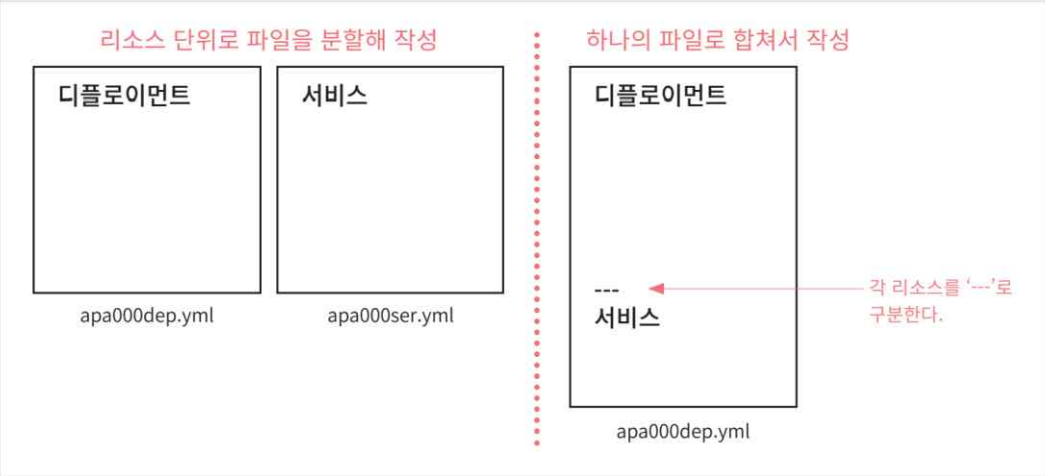


그림 8-5-4 매니페스트 파일은 리소스 단위로 분할하거나 한 파일로 작성할 수 있다.

2. 매니페스트 파일의 내용

- 매니페스트 파일의 주 항목

```
apiVersion:          # API 그룹 및 버전
kind:                # 리소스 유형
metadata:            # 메타데이터
spec:               # 리소스 내용
```

- 리소스 설정(API 그룹 및 유형)
  - 리소스를 정의하려면 API그룹과 리소스 유형을 지정해야 함.

자주 사용되는 리소스의 API 그룹 및 리소스 유형

리소스	API 그룹 / 버전	리소스 유형
파드	core/v1(v1으로 축약 가능)	Pod
서비스	core/v1(v1으로 축약 가능)	Service
디플로이먼트	apps/v1	Deployment
레플리카세트	apps/v1	ReplicaSet

<리소스 유형의 최신 정보 확인>
$ kubectl api-resources 명령 또는 공식 사이트에서 확인

2. 매니페스트 파일의 내용

- 메타데이터와 스펙
  - 메타데이터 : 리소스의 이름이나 레이블을 작성
  - 스펙 : 리소스의 내용을 정의, 항목은 리소스의 유형에 따라 달라짐

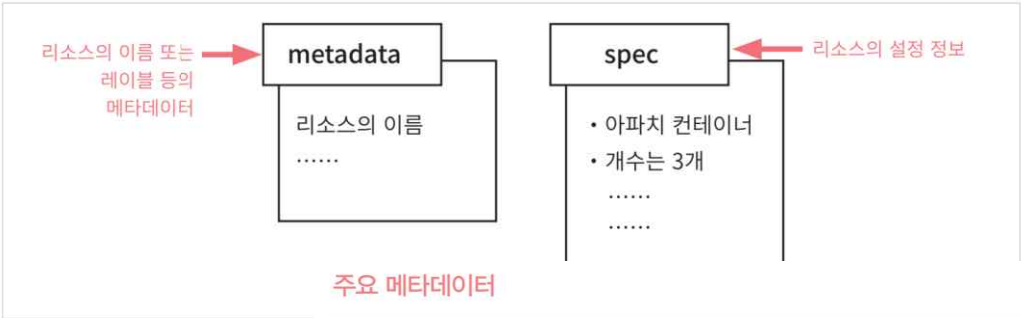


그림 8-5-5 메타데이터와 스펙

주요 메타데이터

항목	내용
name	리소스의 이름. 문자열로 된 유일 식별자.
namespace	리소스를 세분화한 DNS 호환 레이블
uid	유일 식별자
resourceVersion	리소스 버전
generation	생성 순서를 나타내는 번호
creationTimestamp	생성 일시
deletionTimestamp	삭제 일시
labels	임의의 레이블
anotation	리소스에 설정할 값. 선택 대상은 되지 못한다.

2. 매니페스트 파일의 내용

- 레이블과 셀렉터
  - 레이블
    - 키-값 쌍 형태로 구성
    - 파드나 서비스와 같은 리소스에 붙일수 있음
    - 메타데이터로 설정

  - 셀렉터
    - 특정 레이블이 부여된 파드만 선택하여 설정할 때 이용하는 기능

그림 8-5-6 레이블로 특정 리소스를 선택할 수 있다.

3. 메타데이터와 스펙 작성(1)-파드

- 리소스 유형이나 설정 내용에 따라 작성 내용이 상이
  - 파드
    - metadata: 항목에 name, labels 작성
    - spec: 항목에 containers에 컨테이너 구성 정보 작성 (필요시 volumes 항목 작성)

```
apiVersion:
kind:
metadata:
  name:              # 파드 이름
  labels:            # 레이블
spec:
  containers:        # 컨테이너 구성
  - name:            # 컨테이너 이름
    image:           # 이미지
    ports:           # 포트 설
```

```
name
image
ports
```

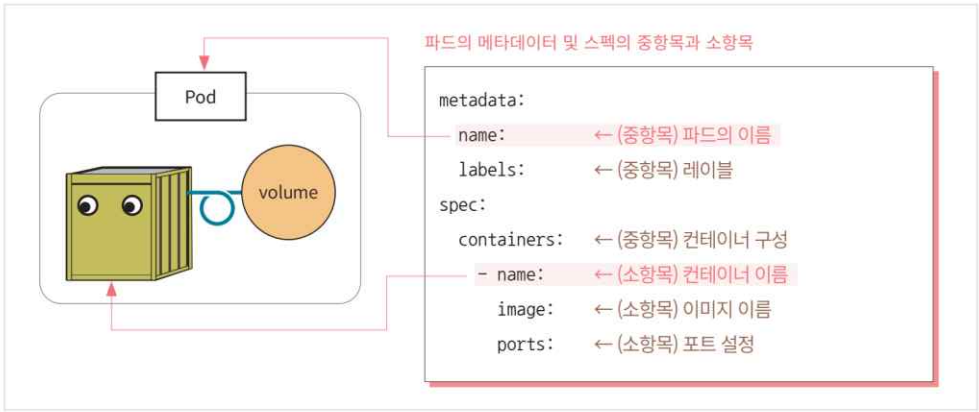


그림 8-5-7 metadata의 name 항목과 containers의 name 항목은 다른 것이다.

3. 메타데이터와 스펙 작성(1)-파드

• 파드 작성 항목

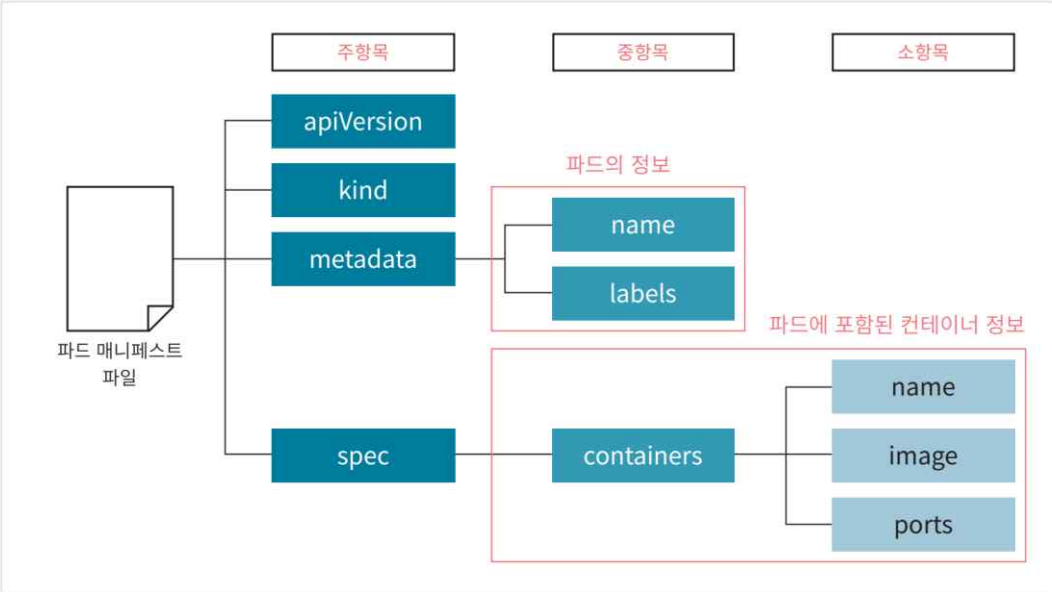


그림 8-5-8 파드 매니페스트 파일의 작성 항목

3. 메타데이터와 스펙 작성(1)-파드

교재 292~295페이지

- <실습> 매니페스트 파일 작성(1)-파드

그림 8-5-9 우선 파드의 매니페스트 파일을 작성하자.

- 생성할 파드와 컨테이너 정보

항목	값
API 그룹/버전	v1(그룹 생략)
리소스 유형	Pod
파드 이름	apa000pod
레이블	app: apa000kube
컨테이너 이름	apa000ex91
이미지 이름	httpd
포트 설정	containerPort: 80

- 생성할 매니페스트 파일명 및 경로

항목	값
파일 이름	apa000pod.yml
파일 경로(윈도우)	C:\Users\사용자명\Documents\kube_folder
파일 경로(macOS)	/Users/사용자명/Documents/kube_folder
파일 경로(리눅스)	/home/사용자명/kube_folder

3.메타데이터와 스펙 작성(1)-파드

교재 292~295페이지

- apa000pod.yml 파일 작성

1) 주 항목 작성

```
apiVersion:
kind:
metadata:
spec:
```

```
apiVersion: v1
kind: Pod
metadata:
  name: apa000po
  labels:
    app: apa000ku
spec:
```

```
apiVersion: v1
kind: Pod
metadata:
  name: apa000pod
  labels:
    app: apa000kube
spec:
  containers:
    - name: apa000ex91
      image: httpd
      ports:
      - containerPort: 80
```

2) 파일 저장

4. 메타데이터와 스펙 작성-디플로이먼트

- 디플로이먼트를 위한 매니페스트 파일 형식

```
apiVersion:
kind:
metadata:
  name:                # 디플로이먼트 이름
spec:
  selector:            # 셀렉터 설정
    matchLabels:       # 셀렉터가 선택할 관리 대상 레이블
  replicas:            # 레플리카 설정
  template:            # 템플릿(파드 정보)
    metadata:          # 파드의 메타데이타 설정
    spec:              # 파드의 스펙 설정
```

- 셀렉터(selector) 설정
  - 디플로이먼트가 특정한 레이블이 지정된 파드를 관리할 수 있도록 설정
  - matchLabels: 항목에 레이블 설정 (template 항목 중 metadata 항목에 설정된 내용)

- 레플리카(replica) 설정
  - 파드의 레플리카에 대한 관리
  - 파드의 수를 '몇 개로 유지할 것인지 설정 (0은 삭제 의미)

continue

4.메타데이터와 스펙 작성-디플로이먼트

- 템플릿(template) 작성
  - 생성할 파드 정보(파드 설정의 메타데이터와 스펙 항목과 유사)
  - 파드의 수가 늘어나면 레이블로 관리하는 경우가 많기 때문에 일반적으로 '파드_이름'은 설정하지 않음

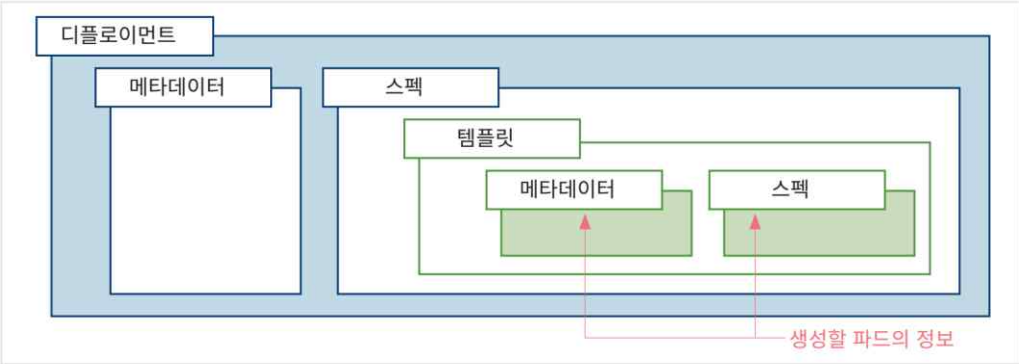


그림 8-5-10 디플로이먼트 정의 파일의 얼개

continue

4.메타데이터와 스펙 작성-디플로이먼트

• 디플로이먼트 작성 항목

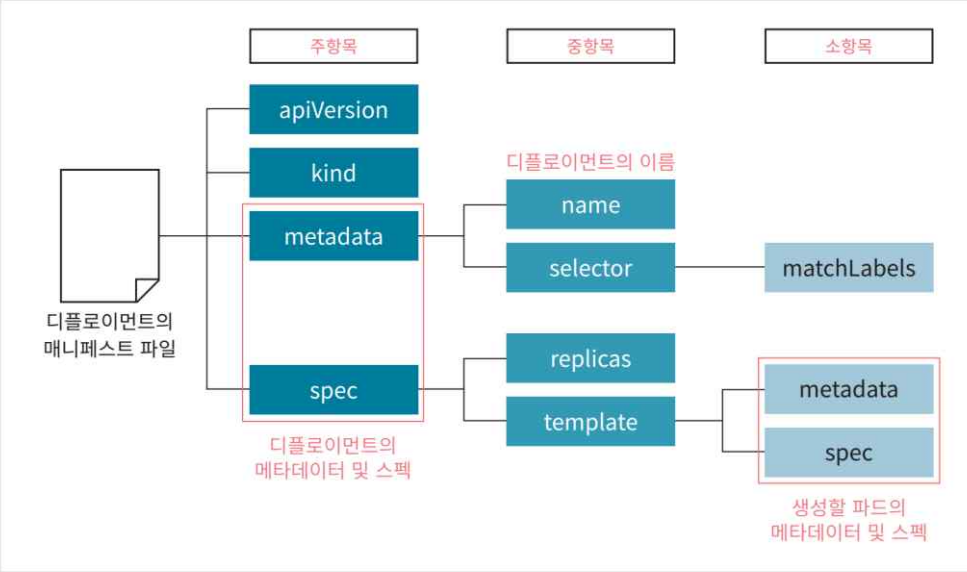


그림 8-5-11 디플로이먼트 매니페스트 파일의 기재 항목 및 내용

4. 메타데이터와 스펙 작성-디플로이먼트

교재 298~301페이지

<실습> 매니페스트 파일 작성(2)-디플로이먼트

그림 8-5-12 디플로이먼트의 매니페스트 파일을 작성한다.

• 생성할 디플로이먼트와 컨테이너 정보

항목	값
API 그룹 / 버전	apps/v1
리소스 유형	Deployment
파드 이름	apa000dep
셀렉터로 대상을 지정할 레이블	app: apa000kube
레플리카(파드) 수	3
컨테이너 이름	apa000ex91
이미지 이름	httpd
컨테이너 포트 설정	containerPort: 80

항목	값
매니페스트 파일 이름	apa000dep.yml

파일의 위치는 293쪽을 참조

continue

4.메타데이터와 스펙 작성-디플로이먼트

- apa000dep.yml 파일 작성

1) 주 항목 설정

```
apiVersion: apps/v1
kind: Deployment
metadata:
spec:
```

2) metadata 항목 설정

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apa000dep
spec:
```

continue

4.메타데이터와 스펙 작성-디플로이먼트

3) spec 항목 설정 : 셀렉터 및 레플리카 수 설정

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apa000dep
spec:
  selector:
    matchLabels:
      app: apa000kub
  replica: 3
```

4) Spec의 template 항

```
 . . .
spec:
  selector:
    matchLabels:
      app: apa000kube
  replica: 3
  template:
    metadata:
      labels:
        app: apa000kube
    spec:
      containers:
      - name: apa000ex91
        image: httpd
        ports:
        - containerPort: 80
```

5. 메타데이타와 스펙 작성-서비스

- 서비스 매니페스트 파일
  - 서비스의 역할 : 파드에 들어오는 요청을 관리하는 것(통신과 관련)
  - 서비스 항목

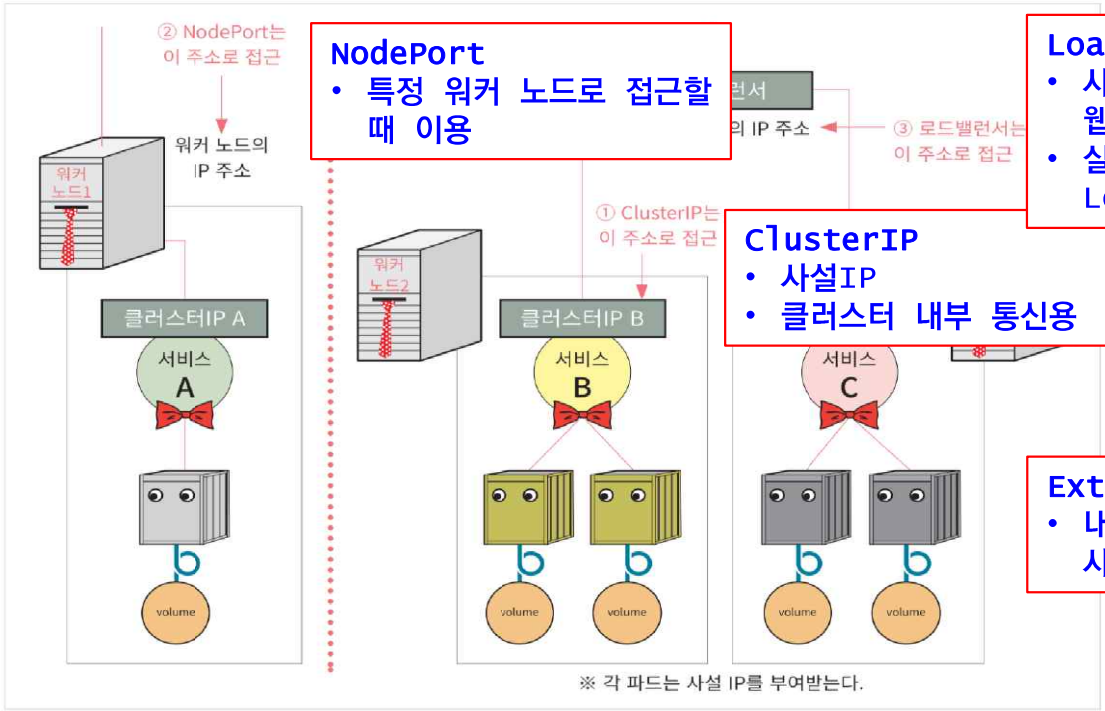
```
apiVersion:
kind:
metadata:
  name:                 # 서비스 이름
spec:
  type:                 # 서비스의 유형
  ports:                # 포트 설정
  - port:               # 서비스 포트
    targetPort:         # 컨테이너 포트
    protocol:           # 통신에 사용되는 프로토콜
    nodePort:           # 워커 노드의 포트
  selector:             # 셀렉터 설정
```

5.메타데이타와 스펙 작성-서비스

- 유형(type) 설정
  - 서비스에 어떤 유형의 IP 주소(또는 DNS)로 접근할지를 설정

	유형 이름	내용
①	ClusterIP	클러스터IP를 통해 서비스에 접근하도록 함(외부에서는 접근 불가)
②	NodePort	워커 노드의 IP를 통해 서비스에 접근하도록 함
③	LoadBalancer	로드밸런서의 IP를 통해 서비스에 접근하도록 함
④	ExternalName	파드에서 서비스를 통해 외부로 나가기 위한 설정

- 유형 설



NodePort
- 특정 워커 노드로 접근할 때 이용

LoadBalancer
- 사용자들이 외부에서 웹사이트 접근할 때 이용
- 실무에서 대부분 LoadBalancer로 설정

ClusterIP
- 사설IP
- 클러스터 내부 통신용

ExternalName
- 내부에서 외부로 접근 시 사용

그림 8-5-13 유형 설정 항목과 의미

5. 메타데이타와 스펙 작성-서비스

• 포트 설정

	항목	내용
①	port	서비스의 포트
②	nodePort	워커 노드의 포트
③	targetPort	컨테이너 포트

nodePort에는 30000~32767 사이 설정 가능

• 포트 설정 항목의 의미

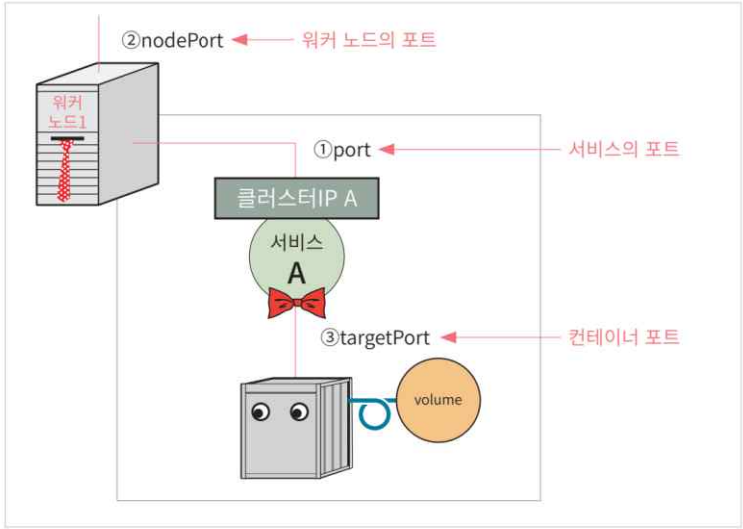


그림 8-5-14 포트 설정 항목의 의미

5.메타데이타와 스펙 작성-서비스

- 셀렉터 설정
  - 서비스가 특정 레이블이 부여된 파드를 선택적으로 관리하기위한 설정
  - 레이블은 파드나 디플로이먼트에서 컨테이너 부분에 설정한 레이블 사용
  - 서비스에서는 'matchLabels:' 항목을 사용하지 않음
    (디플로이먼트에서는 'matchLabels:' 항목이 필수)

5. 메타데이타와 스펙 작성-서비스

교재 304~307페이지

• <실습> 매니페스트 파일 작성(3)-서비스

그림 8-5-15 서비스의 매니페스트 파일을 작성한다.

• 생성할 서비스와 컨테이너 정보

항목	값
API 그룹/버전	v1
리소스 유형	Service
서비스 이름	apa000ser
셀렉터로 대상을 지정할 레이블	app: apa000kube
서비스 유형	NodePort
서비스 포트	8099
컨테이너 포트	80
노드 포트	30080
프로토콜	TCP
매니페스트 파일 이름	apa000ser.yml

continue

5.메타데이타와 스펙 작성-서비스

- apa000ser.yml 작성
  1) 주 항목 및 metadata 설정

```
apiVersion: v1
kind: Service
metadata:
  name: apa000ser
spec:
```

  2) spec 항목 설정

```
apiVersion: v1
kind: Service
metadata:
  name: apa000ser
spec:
  type: NodePort
  ports:
  - port: 8099
    targetPort: 80
    protocol: tcp
    NodePort: 30080
  selector:
    app: apa000kube
```

8장 쿠버네티스를 익히자

6절 쿠버네티스 명령

1.쿠버네티스 명령

• 쿠버네티스 제어 명령

Kubectl 명령 옵션

주요 kubectl 커맨드

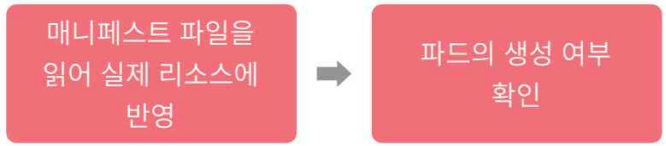
커맨드	내용
create	리소스를 생성
edit	리소스를 편집
delete	리소스를 삭제
get	리소스의 상태를 출력
set	리소스의 값을 설정
apply	리소스의 변경 사항을 반영

커맨드	내용
describe	상세 정보를 확인
diff	'바람직한 상태'와 '현재 상태'의 차이를 확인
expose	여러 파드에 부하를 분산하는 새로운 서비스 오브젝트를 생성
scale	레플리카 수를 변경
autoscale	자동 스케일링을 적용
rollout	롤아웃을 수행
exec	컨테이너에서 명령을 실행
run	컨테이너에서 명령을 한번 실행
attach	컨테이너에 접속
cp	컨테이너에 파일을 복사
logs	컨테이너의 로그를 화면에 출력
cluster-info	클러스터의 상세 정보를 화면에 출력
top	CPU, 메모리, 스토리지 등 시스템 자원을 확인

1.쿠버네티스 명령

교재 310~311페이지

• <실습> 매니페스트 파일로 파드 생성(1)-디플로이먼트



• 사용할 파일

항목	값
파일 이름	apa000dep.yml

• 사용할 명령

커맨드	내용	옵션
apply	리소스의 변경 사항을 반영	-f
get	리소스의 상태를 화면에 출력	

continue

1.쿠버네티스 명령

- 디플로이먼트 매니페스트 파일 적용하기

```
Kubectl apply -f apa000dep.yml
```

```
$ kubectl apply -f apa000dep.yml
deployment.apps/apa000dep created
```

- 파드 생성 목록 확인

```
Kubectl get pods
```

```
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
apa000dep-84d9f4dbf8-d2g65    1/1     Running   0          2m26s
apa000dep-84d9f4dbf8-pxh78    1/1     Running   0          2m26s
apa000dep-84d9f4dbf8-schdh    1/1     Running   0          2m26s
```

1.쿠버네티스 명령

교재 311~313페이지

• <실습> 매니페스트 파일로 파드 생성(2)-서비스

• 사용할 파일

항목	값
파일 이름	apa000ser.yml

• 사용할 명령

커맨드	내용	옵션
apply	리소스의 변경 사항을 반영	-f
get	리소스의 상태를 화면에 출력	

continue

1.쿠버네티스 명령

- 서비스 매니페스트 파일 적용하기

```
Kubectl apply -f apa000ser.yml
```

```
$ kubectl apply -f apa000ser.yml
service/apa000ser created
```

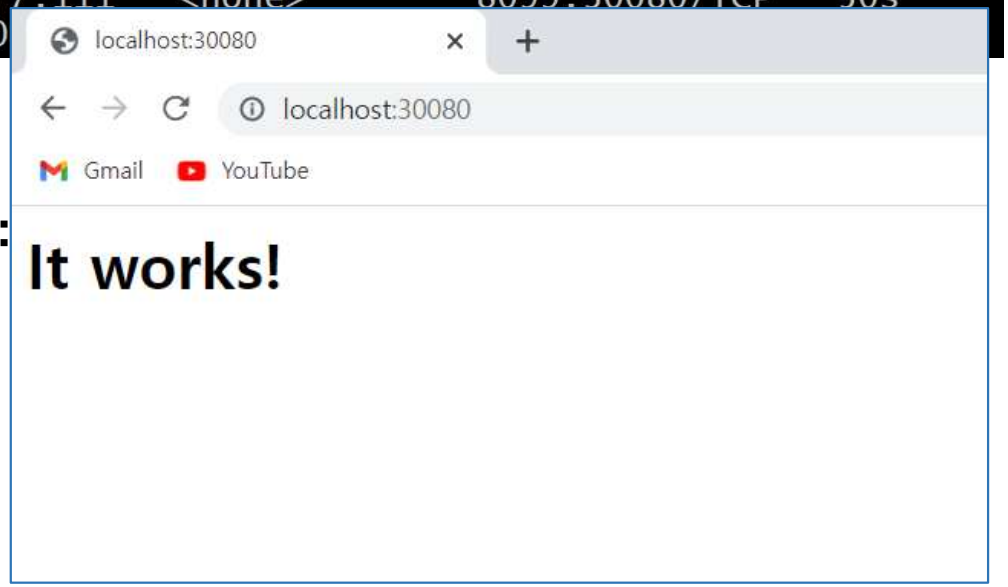
- 서비스 생성 목록 확인

```
Kubectl  get services
```

```
$ kubectl get services
NAME          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
apa000ser     NodePort    10.96.77.111    <none>        8099:30080/TCP   50s
kubernetes    ClusterIP   10.96.0
```

- 웹 브라우저에서 접근

  <URL> http://localhost:



8장 쿠버네티스를 익히자

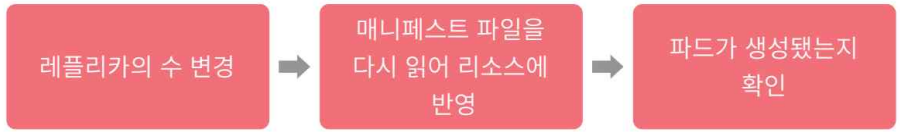
7절 쿠버네티스를 연습하자

1.매니페스트 파일로 파드 개수 증가

교재 314~316페이지

- 쿠버네티스는 매니페스트 파일을 etcd 데이터베이스에 읽어들이고 '바람직한 상태'를 유지하는 기능을 수행한다.
- 매니페스트 파일을 수정하여 다시 읽어들이면 데이터베이스의 내용을 수정하여 파드의 수나 종류 및 상태를 변경할 수 있다.

<실습>



- 사용할 파일

항목	값
매니페스트 파일 이름	apa000dep.yml

- 사용할 명령

커맨드	내용	옵션
apply	리소스의 변경 사항을 반영	-f
get	리소스의 상태를 화면에 출력	

continue

1.매니페스트 파일로 파드 개수 증가

- 디플로이먼트 매니페스트 파일 수정

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apa000dep
spec:
  selector:
    matchLabels:
      app: apa000kube
  replicas: 3 → 5
  template:
    metadata:
      labels:
        app: apa000kube
    spec:
      containers:
      - name: apa000ex91
        image: httpd
        ports:
        - containerPort: 80
```

continue

1. 매니페스트 파일로 파드 개수 증가

- 디플로이먼트 매니페스트 파일 적용하기

```
Kubectl apply -f apa000dep.yml
```

```
$ kubectl apply -f apa000dep.yml
deployment.apps/apa000dep configured
```

- 파드 생성 목록 확인

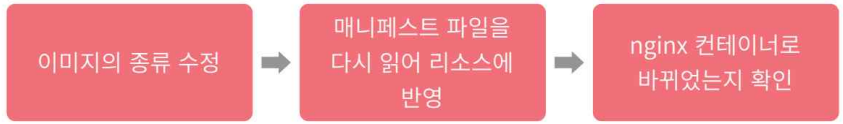
```
Kubectl get pods
```

```
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
apa000dep-84d9f4dbf8-d2g65    1/1     Running   0          31m
apa000dep-84d9f4dbf8-l2wvf    1/1     Running   0          13s
apa000dep-84d9f4dbf8-pxh78    1/1     Running   0          31m
apa000dep-84d9f4dbf8-schdh    1/1     Running   0          31m
apa000dep-84d9f4dbf8-tqrz4    1/1     Running   0          13s
```

2. 매니페스트 파일로 파드 이미지 변경

교재 316~318페이지

<실습> 매니페스트 파일로 아파치를 nginx로 변경



• 사용할 파일

항목	값
매니페스트 파일 이름	apa000dep.yml

• 사용할 명령

커맨드	내용	옵션
apply	리소스의 변경 사항을 반영	-f
get	리소스의 상태를 화면에 출력	

continue

2.매니페스트 파일로 파드 이미지 변경

- 디플로이먼트 매니페스트 파일 수정

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apa000dep
spec:
  selector:
    matchLabels:
      app: apa000kube
  replicas: 5
  template:
    metadata:
      labels:
        app: apa000kube
    spec:
      containers:
      - name: apa000ex91
        image: httpd ⟶ nginx
        ports:
        - containerPort: 80
```

continue

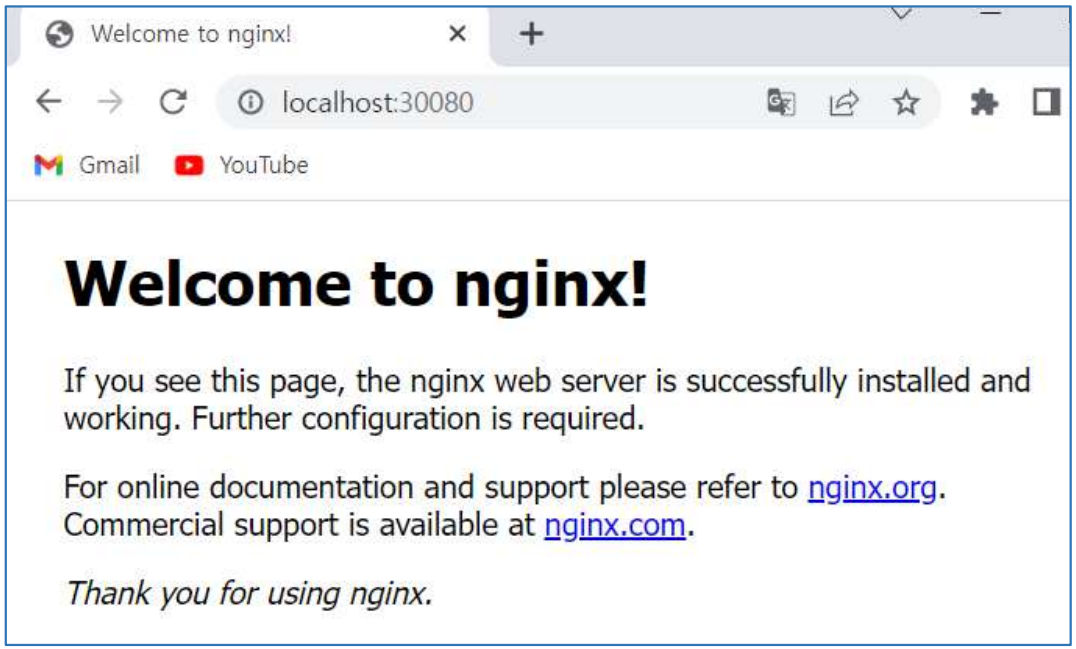
2.매니페스트 파일로 파드 이미지 변경

- 디플로이먼트 매니페스트 파일 적용하기

```
Kubectl apply -f apa000dep.yml
```

```
$ kubectl apply -f apa000dep.yml
deployment.apps/apa000dep configured
```

- nginx 웹 서비스 확인



3.파드 자동 복구 확인

교재 316~318페이지

<실습> 수동으로 파드 삭제 후 자동 복구 확인

• 사용할 파일

항목	값
매니페스트 파일 이름	apa000dep.yml

• 사용할 명령

커맨드	내용	옵션
delete	리소스를 삭제	-f
get	리소스의 상태를 화면에 출력	

continue

3.파드 자동 복구 확인

- get 명령으로 파드 목록 확인

```
Kubectl get pods
```

```
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
apa000dep-7cbc9bd694-68kqk    1/1     Running   0          6m3s
apa000dep-7cbc9bd694-6nbxt    1/1     Running   0          6m3s
apa000dep-7cbc9bd694-qdgx9    1/1     Running   0          5m52s
apa000dep-7cbc9bd694-qrc4m    1/1     Running   0          5m49s
apa000dep-7cbc9bd694-v7nbd    1/1     Running   0          6m3s
```

- delete 명령을 이용하여 파드 삭제

```
Kubectl delete pod <파드_이름>
```

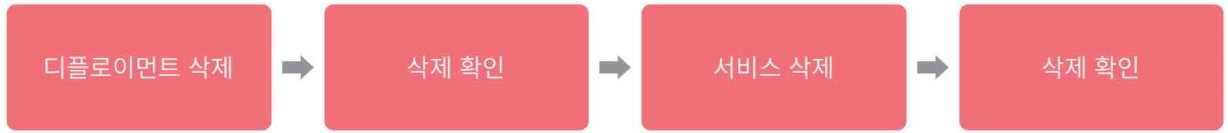
```
$ kubectl delete pod apa000dep-7cbc9bd694-68kqk
pod "apa000dep-7cbc9bd694-68kqk" deleted
```

```
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
apa000dep-7cbc9bd694-6nbxt    1/1     Running   0          8m53s
apa000dep-7cbc9bd694-qdgx9    1/1     Running   0          8m42s
apa000dep-7cbc9bd694-qrc4m    1/1     Running   0          8m39s
apa000dep-7cbc9bd694-v7nbd    1/1     Running   0          8m53s
apa000dep-7cbc9bd694-z7kfz    1/1     Running   0          39s
```

4.디플로이먼트와 서비스 삭제

교재 320~322페이지

- <실습>



- 사용할 명령

커맨드	내용	옵션
delete	리소스를 삭제	-f
get	리소스의 상태를 화면에 출력	

- delete 명령으로 디플로이먼트 삭제 & 확인

```
Kubectl delete -f apa000dep.yml
```

```
$ kubectl delete -f apa000dep.yml
deployment.apps "apa000dep" deleted
```

```
Kubectl get  deployment
```

```
$ kubectl get deployment
No resources found in default namespace.
```

continue

4.디플로이먼트와 서비스 삭제

- 서비스 삭제 & 확인

```
Kubectl delete -f apa000ser.yml
```

```
$ kubectl delete -f apa000ser.yml
service "apa000ser" deleted
```

```
Kubectl get service
```

```
$ kubectl get service
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   17h
```